

Physlib Monthly Report

1–30 April 2025

Contributors: Jeremy Lindsay, Joseph Tooby-Smith, Matteo Cipollina, Pietro Monticone, Zhi Kai Pong

Reviewers: Joseph Tooby-Smith, Tomáš Skřivan

Generated 14 August 2026 from commit `a30f2f3`.

Abstract

Physlib is a community library of formalised physics for the [Lean 4](#) proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month’s additions to it.

It covers 1–30 April 2025 on branch `master` of [leanprover-community/physlib](#), between commits `cac586d` and `a30f2f3`. Over that period 5 contributors landed 152 commits, changing 24,701 lines across 261 files and adding 795 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at [GitHub compare](#).

Contents

1	Introduction	2
1.1	Contributors	2
1.2	Reviewers	2
1.3	Modified subfolders	2
1.4	New declarations by kind	3
2	Details by file	3
2.1	<code>PhysLean.ClassicalMechanics.HarmonicOscillator.Basic</code>	3
2.2	<code>PhysLean.ClassicalMechanics.HarmonicOscillator.Solution</code>	4
2.3	<code>PhysLean.ClassicalMechanics.Momentum.Basic</code>	4
2.4	<code>PhysLean.ClassicalMechanics.Space.Basic</code>	5
2.5	<code>PhysLean.ClassicalMechanics.Space.VectorIdentities</code>	6
2.6	<code>PhysLean.ClassicalMechanics.Time.Basic</code>	9
2.7	<code>PhysLean.ClassicalMechanics.VectorFields</code>	9
2.8	<code>PhysLean.Cosmology.FLRW.Basic</code>	10
2.9	<code>PhysLean.Electromagnetism.Basic</code>	12
2.10	<code>PhysLean.Electromagnetism.FieldStrength.Basic</code>	12
2.11	<code>PhysLean.Electromagnetism.Homogeneous</code>	13
2.12	<code>PhysLean.Electromagnetism.LorentzAction</code>	15
2.13	<code>PhysLean.Electromagnetism.MaxwellEquations</code>	15
2.14	<code>PhysLean.Mathematics.FDerivCurry</code>	16
2.15	<code>PhysLean.Meta.Informal.Post</code>	19
2.16	<code>PhysLean.Meta.Informal.SemiFormal</code>	19
2.17	<code>PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.Basic</code>	19
2.18	<code>PhysLean.QFT.PerturbationTheory.WickAlgebra.Basic</code>	20
2.19	<code>PhysLean.QFT.PerturbationTheory.WickAlgebra.Grading</code>	26

2.20	PhysLean.QFT.PerturbationTheory.WickAlgebra.NormalOrder.Basic	30
2.21	PhysLean.QFT.PerturbationTheory.WickAlgebra.NormalOrder.Lemmas	32
2.22	PhysLean.QFT.PerturbationTheory.WickAlgebra.NormalOrder.WickContractions	37
2.23	PhysLean.QFT.PerturbationTheory.WickAlgebra.StaticWickTheorem	38
2.24	PhysLean.QFT.PerturbationTheory.WickAlgebra.SuperCommute	39
2.25	PhysLean.QFT.PerturbationTheory.WickAlgebra.TimeContraction	45
2.26	PhysLean.QFT.PerturbationTheory.WickAlgebra.TimeOrder	47
2.27	PhysLean.QFT.PerturbationTheory.WickAlgebra.Universality	50
2.28	PhysLean.QFT.PerturbationTheory.WickAlgebra.WicksTheoremNormal	51
2.29	PhysLean.QFT.PerturbationTheory.WickContraction.Perm	53
2.30	PhysLean.Relativity.Lorentz.Group.Basic	54
2.31	PhysLean.Relativity.Lorentz.Group.Orthochronous	54
2.32	PhysLean.Relativity.Lorentz.Group.Proper	54
2.33	PhysLean.Relativity.Lorentz.Group.Restricted	55
2.34	PhysLean.Relativity.PauliMatrices.Basic	55
2.35	PhysLean.Relativity.SpaceTime.Basic	56
2.36	PhysLean.Relativity.SpaceTime.TimeSlice	58
2.37	PhysLean.Relativity.Special.TwinParadox.Basic	59
2.38	PhysLean.Relativity.Tensors.Basic	59
2.39	PhysLean.Relativity.Tensors.ComplexTensor.Basic	61
2.40	PhysLean.Relativity.Tensors.ComplexTensor.Lemmas	62
2.41	PhysLean.Relativity.Tensors.ComplexTensor.Metrics.Basic	62
2.42	PhysLean.Relativity.Tensors.ComplexTensor.OfRat	66
2.43	PhysLean.Relativity.Tensors.ComplexTensor.Units.Basic	67
2.44	PhysLean.Relativity.Tensors.Constructors	71
2.45	PhysLean.Relativity.Tensors.Contraction.Basic	76
2.46	PhysLean.Relativity.Tensors.Contraction.Basis	77
2.47	PhysLean.Relativity.Tensors.Contraction.Products	79
2.48	PhysLean.Relativity.Tensors.Contraction.Pure	79
2.49	PhysLean.Relativity.Tensors.Dual	81
2.50	PhysLean.Relativity.Tensors.Elab	82
2.51	PhysLean.Relativity.Tensors.MetricTensor	87
2.52	PhysLean.Relativity.Tensors.OfInt	88
2.53	PhysLean.Relativity.Tensors.Product	89
2.54	PhysLean.Relativity.Tensors.RealTensor.Basic	90
2.55	PhysLean.Relativity.Tensors.RealTensor.Metrics.Basic	90
2.56	PhysLean.Relativity.Tensors.RealTensor.ToComplex	91
2.57	PhysLean.Relativity.Tensors.RealTensor.Vector.Basic	92
2.58	PhysLean.Relativity.Tensors.RealTensor.Vector.Causality.Basic	93
2.59	PhysLean.Relativity.Tensors.RealTensor.Vector.Causality.LightLike	94
2.60	PhysLean.Relativity.Tensors.RealTensor.Vector.Causality.TimeLike	94
2.61	PhysLean.Relativity.Tensors.TensorSpecies.Basic	96
2.62	PhysLean.Relativity.Tensors.UnitTensor	96
2.63	PhysLean.StringTheory.FTheory.SU5U1.Charges	97
2.64	PhysLean.StringTheory.FTheory.SU5U1.Examples	98
2.65	PhysLean.StringTheory.FTheory.SU5U1.Matter	101
2.66	PhysLean.StringTheory.FTheory.SU5U1.NoExotics.Basic	105
2.67	PhysLean.StringTheory.FTheory.SU5U1.NoExotics.FiveBar	106
2.68	PhysLean.StringTheory.FTheory.SU5U1.NoExotics.Ten	109
2.69	PhysLean.StringTheory.FTheory.SU5U1.PhenConstraints.Basic	110
2.70	scripts.MetaPrograms.check_dup_tags	113

2.71	<code>scripts.MetaPrograms.local_stats</code>	113
2.72	<code>scripts.MetaPrograms.make_tag</code>	113
2.73	<code>scripts.MetaPrograms.TODO_to_yaml</code>	114

1 Introduction

During Apr 2025, 5 contributors submitted 152 commits that changed 24,701 lines ([+11,577](#) / [-13,124](#)) across 261 files spanning 22 top-level areas of the repository. Of those, [48](#) files were newly added and [40](#) were removed outright. This report catalogues 795 newly added Lean declarations: 558 lemmas, 154 defs, 21 instances, 20 abbrevs, 16 semiformal results, 13 informal lemmas, 6 theorems, 4 structures, 2 informal definitions, and 1 inductive. We're delighted to recognize the following first-time contributors this month: [Jeremy Lindsay](#), [Matteo Cipollina](#), and [Zhi Kai Pong](#).

See the [full compare diff](#) and the [list of commits](#) on GitHub for the raw source.

1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

Contributor	Lines Changed	Commits
Jeremy Lindsay	247	7
Joseph Tooby-Smith	25,683	63
Matteo Cipollina	4,144	16
Pietro Monticone	190	4
Zhi Kai Pong	2,854	62

1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

Reviewer	PRs Reviewed
Joseph Tooby-Smith	20
Tomáš Skřivan	1

1.3 Modified subfolders

Subfolder	Files	New	Deleted	Additions	Deletions
PhysLean/Relativity	112	+24	-37	+6,963	-10,711
PhysLean/QFT	50	+1	-3	+406	-1,410
PhysLean/StringTheory	8	+8	—	+1,426	-0
PhysLean/ClassicalMechanics	7	+5	—	+851	-73
PhysLean/Electromagnetism	6	+2	—	+460	-314
PhysLean/Mathematics	5	+1	—	+345	-16
docs/_data	2	—	—	+176	-176
scripts/MetaPrograms	5	+3	—	+280	-37
PhysLean/Particles	43	—	—	+148	-155
PhysLean.lean	1	—	—	+140	-136
PhysLean/Meta	6	+1	—	+117	-6
lakefile.toml	1	—	—	+69	-36
PhysLean/Cosmology	1	+1	—	+99	-0
README.md	1	—	—	+17	-14

.github/workflows	2	+1	—	+26	-0
CONTRIBUTING.md	1	—	—	+4	-21
PhysLean/CondensedMatter	1	+1	—	+24	-0
lake-manifest.json	1	—	—	+9	-9
PhysLean/QuantumMechanics	5	—	—	+6	-9
scripts	1	—	—	+9	-0
lean-toolchain	1	—	—	+1	-1
.github/ISSUE_TEMPLATE	1	—	—	+1	-0

1.4 New declarations by kind

Kind	Count
lemma	558
def	154
instance	21
abbrev	20
semiformal_result	16
informal_lemma	13
theorem	6
structure	4
informal_definition	2
inductive	1

2 Details by file

2.1 PhysLean.ClassicalMechanics.HarmonicOscillator.Basic

[PhysLean/ClassicalMechanics/HarmonicOscillator/Basic.lean](#) on GitHub.

semiformal_result force

[\[source\]](#)

The force of the classical harmonic oscillator defined as $-dU(x)/dx$ where $U(x)$ is the potential energy.

```
semiformal_result "6YBYP" force (S : HarmonicOscillator) (x : Time → ℝ) : Time → ℝ
```

semiformal_result force_is_linear

[\[source\]](#)

The force on the classical harmonic oscillator is $-k \cdot x$.

```
semiformal_result "6YB2U" force_is_linear (x : Time → ℝ) :  
  force S x = - S.k • x
```

semiformal_result «semiformal_result@13b4716f»

[\[source\]](#)

The definition of the equation of motion for the classical harmonic oscillator defined through the Euler-Lagrange equations.

```
semiformal_result "6ZTP5" EquationOfMotion (x : Time → ℝ) : Prop
```

semiformal_result equationOfMotion_iff_newtons_second_law[\[source\]](#)

The equations of motion are satisfied if and only if Newton's second law holds.

```
semiformal_result "6YBEI" equationOfMotion_iff_newtons_second_law (x : Time → ℝ) :
  EquationOfMotion x ↔ ∀ t, force S x t = S.m * deriv (fun t' => deriv x t') t
```

semiformal_result ExtremaOfAction[\[source\]](#)

The proposition on a trajectory which is true if that trajectory is an extrema of the action.

semiformal implementation notes: - This is not expected to be easy to define.

```
semiformal_result "6YBIG" ExtremaOfAction (x : Time → ℝ) : Prop
```

semiformal_result equationOfMotion_iff_extremaOfAction[\[source\]](#)

A trajectory $x : \mathbb{R} \rightarrow \mathbb{R}$ satisfies the equation of motion if and only if it is an extrema of the action.

Implementation note: This result depends on other semi-formal results which will need defining before this.

```
semiformal_result "6YBQH" equationOfMotion_iff_extremaOfAction (x : Time → ℝ) :
  EquationOfMotion x ↔ ExtremaOfAction x
```

2.2 PhysLean.ClassicalMechanics.HarmonicOscillator.Solution

[PhysLean/ClassicalMechanics/HarmonicOscillator/Solution.lean](#) on GitHub.

semiformal_result sol_equationOfMotion[\[source\]](#)

The solutions for any initial condition solve the equation of motion.

```
semiformal_result "6YATB" sol_equationOfMotion (IC : InitialConditions) :
  EquationOfMotion (S.sol IC)
```

semiformal_result sol_unique[\[source\]](#)

The solutions to the equation of motion for a given set of initial conditions are unique.

Semiformal implmentation: - One may needed the added condition of smoothness on x here. - EquationOfMotion needs defining before this can be proved.

```
semiformal_result "6VZJO" sol_unique (IC : InitialConditions) (x : Time → ℝ) :
  EquationOfMotion x ∧ x 0 = IC.x₀ ∧ deriv x 0 = IC.v₀ →
  x = S.sol IC
```

2.3 PhysLean.ClassicalMechanics.Momentum.Basic

[PhysLean/ClassicalMechanics/Momentum/Basic.lean](#) on GitHub.

abbrev Momentum[\[source\]](#)

The type `Space d` represents d dimensional Euclidean space. The default value of d is 3. Thus `Space = Space 3`.

```
abbrev Momentum (d : ℕ := 3)
```

2.4 PhysLean.ClassicalMechanics.Space.Basic

[PhysLean/ClassicalMechanics/Space/Basic.lean](#) on GitHub.

def basis[\[source\]](#)

The standard basis of `Space` based on `Fin d`.

```
noncomputable def basis : OrthonormalBasis (Fin d) ℝ (Space d)
```

def coord[\[source\]](#)

The standard coordinate functions of `Space` based on `Fin d`.

The notation $\S \mu p$ can be used for this.

```
noncomputable def coord (μ : Fin d) (p : Space d) : ℝ
```

def deriv[\[source\]](#)

Given a function $f : \text{Space } d \rightarrow M$ the derivative of f in direction μ .

```
noncomputable def deriv [AddCommGroup M] [Module ℝ M] [TopologicalSpace M]
  (μ : Fin d) (f : Space d → M) : Space d → M
```

lemma deriv_eq[\[source\]](#)

```
lemma deriv_eq [AddCommGroup M] [Module ℝ M] [TopologicalSpace M]
  (μ : Fin d) (f : Space d → M) (x : Space d) :
  deriv μ f x = fderiv ℝ f x (EuclideanSpace.single μ (1:ℝ))
```

def grad[\[source\]](#)

The vector calculus operator `grad`.

```
noncomputable def grad (f : Space d → ℝ) :
  Space d → EuclideanSpace ℝ (Fin d)
```

def curl[\[source\]](#)

The vector calculus operator `curl`.

```
noncomputable def curl (f : Space → EuclideanSpace ℝ (Fin 3)) :
  Space → EuclideanSpace ℝ (Fin 3)
```

def div[\[source\]](#)*The vector calculus operator div .*

```
noncomputable def div (f : Space d → EuclideanSpace ℝ (Fin d)) :
  Space d → ℝ
```

def laplacian[\[source\]](#)*The scalar laplacian operator.*

```
noncomputable def laplacian (f : Space d → ℝ) :
  Space d → ℝ
```

def laplacian_vec[\[source\]](#)*The vector laplacian_vec operator.*

```
noncomputable def laplacian_vec (f : Space d → EuclideanSpace ℝ (Fin d)) :
  Space d → EuclideanSpace ℝ (Fin d)
```

2.5 PhysLean.ClassicalMechanics.Space.VectorIdentities[PhysLean/ClassicalMechanics/Space/VectorIdentities.lean](#) on GitHub.**lemma fderiv_pi'**[\[source\]](#)

```
lemma fderiv_pi' (h : DifferentiableAt ℝ Φ x) :
  fderiv ℝ Φ x = ContinuousLinearMap.pi fun i => (fderiv ℝ fun x => Φ x i) x
```

lemma differentiable_fderiv[\[source\]](#)

```
lemma ContDiff.differentiable_fderiv (f : X → Y) (hf : ContDiff ℝ 2 f) :
  Differentiable ℝ (fun x => fderiv ℝ f x)
```

lemma fderiv_coord_eq_proj_comp[\[source\]](#)

```
lemma fderiv_coord_eq_proj_comp (h : DifferentiableAt ℝ Φ x) :
  (fderiv ℝ fun x => Φ x i) x = (ContinuousLinearMap.proj i).comp (fderiv ℝ Φ x)
```

lemma deriv_add[\[source\]](#)*Derivatives on space distribute over addition.*

```
lemma deriv_add [NormedAddCommGroup M] [NormedSpace ℝ M]
  (f1 f2 : Space d → M) (hf1 : Differentiable ℝ f1) (hf2 : Differentiable ℝ f2) :
  ∂[u] (f1 + f2) = ∂[u] f1 + ∂[u] f2
```

lemma deriv_coord_add[\[source\]](#)*Derivatives on space distribute coordinate-wise over addition.*

```

lemma deriv_coord_add (f1 f2 : Space d → EuclideanSpace ℝ (Fin d))
  (hf1 : Differentiable ℝ f1) (hf2 : Differentiable ℝ f2) :
  (∂[u] (fun x => f1 x i + f2 x i)) =
    (∂[u] (fun x => f1 x i)) + (∂[u] (fun x => f2 x i))

```

lemma deriv_smul

[\[source\]](#)

Scalar multiplication on space derivatives.

```

lemma deriv_smul [NormedAddCommGroup M] [NormedSpace ℝ M]
  (f : Space d → M) (k : ℝ) (hf : Differentiable ℝ f) :
  ∂[u] (k • f) = (k • ∂[u] f)

```

lemma deriv_coord_smul

[\[source\]](#)

Coordinate-wise scalar multiplication on space derivatives.

```

lemma deriv_coord_smul (f : Space d → EuclideanSpace ℝ (Fin d)) (k : ℝ)
  (hf : Differentiable ℝ f) :
  ∂[u] (fun x => k * f x i) x = (k • ∂[u] (fun x => f x i)) x

```

lemma deriv_commute

[\[source\]](#)

Derivatives on space commute with one another.

```

lemma deriv_commute [NormedAddCommGroup M] [NormedSpace ℝ M]
  (f : Space d → M) (hf : ContDiff ℝ 2 f) : ∂[u] (∂[v] f) = ∂[v] (∂[u] f)

```

lemma differentiable_fderiv_coord

[\[source\]](#)

Coordinate functions of fderiv is differentiable.

```

lemma differentiable_fderiv_coord (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : ContDiff ℝ 2 f)
  :
  Differentiable ℝ (fderiv ℝ fun x => f x i)

```

lemma deriv_coord_2nd_add

[\[source\]](#)

Second derivatives distribute coordinate-wise over addition (all three components for div).

```

lemma deriv_coord_2nd_add (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : ContDiff ℝ 2 f) :
  ∂[i] (fun x => ∂[u] (fun x => f x u) x + (∂[v] (fun x => f x v) x + ∂[w] (fun x => f x w) x)) =
    (∂[i] (∂[u] (fun x => f x u))) + (∂[i] (∂[v] (fun x => f x v))) +
    (∂[i] (∂[w] (fun x => f x w)))

```

lemma deriv_coord_2nd_sub

[\[source\]](#)

Second derivatives distribute coordinate-wise over subtraction (two components for curl).

```

lemma deriv_coord_2nd_sub (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : ContDiff ℝ 2 f) :
  ∂[u] (fun x => ∂[v] (fun x => f x w) x - ∂[w] (fun x => f x v) x) =

```

$$(\partial[u] (\partial[v] (\text{fun } x \Rightarrow f \times w))) - (\partial[u] (\partial[w] (\text{fun } x \Rightarrow f \times v)))$$
lemma laplacian_eq_div_of_grad[\[source\]](#)

```
lemma laplacian_eq_div_of_grad (f : Space → ℝ) :
  Δ f = ∇ · ∇ f
```

lemma div_of_curl_eq_zero[\[source\]](#)

```
lemma div_of_curl_eq_zero (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : ContDiff ℝ 2 f) :
  ∇ · (∇ × f) = 0
```

lemma curl_of_curl[\[source\]](#)

```
lemma curl_of_curl (f : Space → EuclideanSpace ℝ (Fin 3)) (hf : ContDiff ℝ 2 f) :
  ∇ × (∇ × f) = ∇ (∇ · f) - Δ f
```

lemma grad_add[\[source\]](#)

```
lemma grad_add (f1 f2 : Space d → ℝ)
  (hf1 : Differentiable ℝ f1) (hf2 : Differentiable ℝ f2) :
  ∇ (f1 + f2) = ∇ f1 + ∇ f2
```

lemma div_add[\[source\]](#)

```
lemma div_add (f1 f2 : Space → EuclideanSpace ℝ (Fin 3))
  (hf1 : Differentiable ℝ f1) (hf2 : Differentiable ℝ f2) :
  ∇ · (f1 + f2) = ∇ · f1 + ∇ · f2
```

lemma curl_add[\[source\]](#)

```
lemma curl_add (f1 f2 : Space → EuclideanSpace ℝ (Fin 3))
  (hf1 : Differentiable ℝ f1) (hf2 : Differentiable ℝ f2) :
  ∇ × (f1 + f2) = ∇ × f1 + ∇ × f2
```

lemma grad_smul[\[source\]](#)

```
lemma grad_smul (f : Space d → ℝ) (k : ℝ)
  (hf : Differentiable ℝ f) :
  ∇ (k • f) = k • ∇ f
```

lemma div_smul[\[source\]](#)

```
lemma div_smul (f : Space → EuclideanSpace ℝ (Fin 3)) (k : ℝ)
  (hf : Differentiable ℝ f) :
  ∇ · (k • f) = k • ∇ · f
```

lemma curl_smul[\[source\]](#)

```
lemma curl_smul (f : Space → EuclideanSpace ℝ (Fin 3)) (k : ℝ)
  (hf : Differentiable ℝ f) :
```

$$\nabla \times (k \cdot f) = k \cdot \nabla \times f$$

lemma div_linear_map[\[source\]](#)

```
lemma div_linear_map (f : W → Space 3 → EuclideanSpace ℝ (Fin 3))
  (hf : ∀ w, Differentiable ℝ (f w))
  (hf' : IsLinearMap ℝ f) :
  IsLinearMap ℝ (fun w => ∇ · (f w))
```

lemma curl_linear_map[\[source\]](#)

```
lemma curl_linear_map (f : W → Space 3 → EuclideanSpace ℝ (Fin 3))
  (hf : ∀ w, Differentiable ℝ (f w))
  (hf' : IsLinearMap ℝ f) :
  IsLinearMap ℝ (fun w => ∇ × (f w))
```

2.6 PhysLean.ClassicalMechanics.Time.Basic

[PhysLean/ClassicalMechanics/Time/Basic.lean](#) on GitHub.

abbrev Time[\[source\]](#)

The type Time represents the time manifold.

```
abbrev Time
```

def deriv[\[source\]](#)

Given a function $f : \text{Time} \rightarrow M$ the derivative of f .

```
noncomputable def deriv [AddCommGroup M] [Module ℝ M] [TopologicalSpace M]
  (f : Time → M) : Time → M
```

2.7 PhysLean.ClassicalMechanics.VectorFields

[PhysLean/ClassicalMechanics/VectorFields.lean](#) on GitHub.

lemma dt_distrib[\[source\]](#)

```
lemma dt_distrib (f : Time → Space → EuclideanSpace ℝ (Fin 3)) :
  (fderiv ℝ (fun t => (fderiv ℝ (fun x => f t x u) x) dx1) t -
   fderiv ℝ (fun t => (fderiv ℝ (fun x => f t x v) x) dx2) t) 1
  =
  (fderiv ℝ (fun t => (fderiv ℝ (fun x => f t x u) x) dx1) t) 1 -
  (fderiv ℝ (fun t => (fderiv ℝ (fun x => f t x v) x) dx2) t) 1
```

lemma fderiv_coord_dt[\[source\]](#)

```
lemma fderiv_coord_dt (f : Time → Space → EuclideanSpace ℝ (Fin 3)) (t dt : Time)
  (hf : Differentiable ℝ (f)) :
  (fun x => (fderiv ℝ (fun t => f t x i) t) dt)
  =
  (fun x => (fderiv ℝ (fun t => f t x) t) dt i)
```

lemma fderiv_swap_time_space_coord[\[source\]](#)*Derivatives along space coordinates and time commute.*

```

lemma fderiv_swap_time_space_coord
  (f : Time → Space → EuclideanSpace ℝ (Fin 3)) (t dt : Time) (x dx : Space)
  (hf : ContDiff ℝ 2 ↑f) :
  fderiv ℝ (fun t' => fderiv ℝ (fun x' => f t' x' i) x dx) t dt
  =
  fderiv ℝ (fun x' => fderiv ℝ (fun t' => f t' x' i) t dt) x dx

```

lemma differentiableAt_fderiv_coord_single[\[source\]](#)

```

lemma differentiableAt_fderiv_coord_single
  (ft : Time → Space → EuclideanSpace ℝ (Fin 3)) (hf : ContDiff ℝ 2 ↑ft) :
  DifferentiableAt ℝ (fun t =>
    (fderiv ℝ (fun x => ft t x u) x) (EuclideanSpace.single v 1)) t

```

lemma time_deriv_curl_commute[\[source\]](#)*Curl and time derivative commute.*

```

lemma time_deriv_curl_commute (ft : Time → Space → EuclideanSpace ℝ (Fin 3))
  (t : Time) (x : Space) (hf : ContDiff ℝ 2 ↑ft) :
  ∂t (fun t => (∇ × ft t) x) t = (∇ × fun x => (∂t (fun t => ft t x) t)) x

```

2.8 PhysLean.Cosmology.FLRW.Basic

PhysLean/Cosmology/FLRW/Basic.lean on GitHub.

semiformal_result SpatialGeometry[\[source\]](#)*The inductive type with three constructors: - Spherical ($k : \mathbb{R}$) - Flat - Saddle ($k : \mathbb{R}$)*

```

semiformal_result "6ZZLH" SpatialGeometry : Type

```

semiformal_result S[\[source\]](#)*For s corresponding to - Spherical k , $S\ s\ r = k * \sin(r / k)$ - Flat, $S\ s\ r = r$, - Saddle k , $S\ s\ r = k * \sin(r / k)$.**Semiformal implementation note: There is likely a better name for this function.*

```

semiformal_result "6ZZTV" S (s : SpatialGeometry) : ℝ → ℝ

```

informal_lemma limit_S_saddle[\[source\]](#)*The limit of S (Saddle k) r as $k \rightarrow \infty$ is equal to S (Flat) r .*

```

informal_lemma limit_S_saddle where
  deps := []
  tag := "6ZZ3D"

```

informal_lemma limit_S_sphere[\[source\]](#)

The limit of S (Sphere k) r as $k \rightarrow \infty$ is equal to S (Flat) r .

```
informal_lemma limit_S_sphere where
```

```
  deps := []
```

```
  tag := "62A4R"
```

semiformal_result FLRW[\[source\]](#)

The structure FLRW is defined to contain the physical parameters of the Friedmann-Lemaître-Robertson-Walker metric. That is, it contains - The scale factor $a(t)$ - An element of SpatialGeometry.

Semiformal implementation note: It is possible that we should restrict $a(t)$ to be smooth or at least twice differentiable.

```
semiformal_result "6Z2AV" FLRW : Type
```

informal_definition hubbleConstant[\[source\]](#)

The hubble constant defined in terms of the scale factor as $(d_t a) / a$.

The notation H is used for the hubbleConstant.

Semiformal implementation note: Implement also scoped notation.

```
informal_definition hubbleConstant where
```

```
  deps := []
```

```
  tag := "6Z2NB"
```

informal_definition decelerationParameter[\[source\]](#)

The deceleration parameter defined in terms of the scale factor as $-(d_t d_t a) / a / (d_t a)^2$.

The notation q is used for the decelerationParameter.

Semiformal implementation note: Implement also scoped notation.

```
informal_definition decelerationParameter where
```

```
  deps := []
```

```
  tag := "6Z2UE"
```

informal_lemma decelerationParameter_eq_one_plus_hubbleConstant[\[source\]](#)

The deceleration parameter is equal to $-(1 + (d_t H)/H^2)$.

```
informal_lemma decelerationParameter_eq_one_plus_hubbleConstant where
```

```
  deps := []
```

```
  tag := "6Z23H"
```

informal_lemma time_evolution_hubbleConstant[\[source\]](#)

The time evolution of the hubble parameter is equal to $d_t H = -H^2 (1 + q)$.

```
informal_lemma time_evolution_hubbleConstant where
  deps := []
  tag := "6Z3BS"
```

informal_lemma hubbleConstant_decrease_iff

[\[source\]](#)

There exists a time at which the hubble constant decreases if and only if there exists a time where the deceleration parameter is less than -1.

```
informal_lemma hubbleConstant_decrease_iff where
  deps := []
  tag := "6Z3FS"
```

2.9 PhysLean.Electromagnetism.Basic

[PhysLean/Electromagnetism/Basic.lean](#) on GitHub.

def c

[\[source\]](#)

The speed of light.

```
noncomputable def c : ℝ
```

2.10 PhysLean.Electromagnetism.FieldStrength.Basic

[PhysLean/Electromagnetism/FieldStrength/Basic.lean](#) on GitHub.

lemma timeSliceLinearEquiv_ofElectricFieldAux

[\[source\]](#)

```
lemma timeSliceLinearEquiv_ofElectricFieldAux {d : ℕ} (E : ElectricField d) (t : Time)
  (x : Space d) :
  timeSliceLinearEquiv (ofElectricFieldAux E) t x =
  ((Tensor.basis _).repr.symm <|
    Finsupp.equivFunOnFinite.symm <| fun b =>
      match b 0, b 1 with
      | 0, (j + 1, hj) => - E t x ⟨j, by
        change j + 1 < 1 + d at hj
        omega)
      | ⟨j + 1, hj⟩, 0 => E t x ⟨j, by
        change j + 1 < 1 + d at hj
        omega)
      | _, _ => 0)
```

lemma ofElectricField_coe

[\[source\]](#)

```
lemma ofElectricField_coe {d : ℕ} (E : ElectricField d) :
  (ofElectricField E).1 = ofElectricFieldAux E
```

lemma timeSliceLinearEquiv_ofMagneticFieldAux

[\[source\]](#)

```
lemma timeSliceLinearEquiv_ofMagneticFieldAux (B : MagneticField) (t : Time)
  (x : Space 3) :
  timeSliceLinearEquiv (ofMagneticFieldAux B) t x =
  ((Tensor.basis _).repr.symm <|
```

```

Finsupp.equivFunOnFinite.symm <| fun b =>
match b 0, b 1 with
| 1, 2 => - B t x 2
| 1, 3 => B t x 1
| 2, 3 => - B t x 0
| 2, 1 => B t x 2
| 3, 1 => - B t x 1
| 3, 2 => B t x 0
| _, _ => 0)

```

lemma ofMagneticField_coe[\[source\]](#)

```

lemma ofMagneticField_coe (B : MagneticField) :
  (ofMagneticField B).1 = ofMagneticFieldAux B

```

lemma electricField_apply[\[source\]](#)

```

lemma electricField_apply {d : ℕ} (F : FieldStrength d) (t : Time) (x : Space d) (j : Fin d) :
  electricField F t x j = (Tensor.basis _).repr (timeSliceLinearEquiv F.1 t x)
  (fun | 0 => ⟨j + 1, by simp; omega⟩ | 1 => 0)

```

lemma timeSliceLinearEquiv_symm_electricField[\[source\]](#)

```

lemma timeSliceLinearEquiv_symm_electricField {d : ℕ} (F : FieldStrength d) :
  timeSliceLinearEquiv.symm (electricField F) = fun x (j : Fin d) =>
  (Tensor.basis _).repr (F.1 x) (fun | 0 => ⟨j + 1, by simp; omega⟩ | 1 => 0)

```

lemma magneticField_apply[\[source\]](#)

```

lemma magneticField_apply (F : FieldStrength) (t : Time) (x : Space) (j : Fin 3) :
  magneticField F t x j = match j with
  | 0 => (Tensor.basis _).repr (timeSliceLinearEquiv F.1 t x) (fun | 0 => 3 | 1 => 2)
  | 1 => (Tensor.basis _).repr (timeSliceLinearEquiv F.1 t x) (fun | 0 => 1 | 1 => 3)
  | 2 => (Tensor.basis _).repr (timeSliceLinearEquiv F.1 t x) (fun | 0 => 2 | 1 => 1)

```

lemma timeSliceLinearEquiv_symm_magneticField[\[source\]](#)

```

lemma timeSliceLinearEquiv_symm_magneticField (F : FieldStrength) :
  timeSliceLinearEquiv.symm (magneticField F) = fun x j =>
  match j with
  | 0 =>
    (Tensor.basis _).repr (F.1 x) (fun | 0 => 3 | 1 => 2)
  | 1 =>
    (Tensor.basis _).repr (F.1 x) (fun | 0 => 1 | 1 => 3)
  | 2 =>
    (Tensor.basis _).repr (F.1 x) (fun | 0 => 2 | 1 => 1)

```

2.11 PhysLean.Electromagnetism.Homogeneous

[PhysLean/Electromagnetism/Homogeneous.lean](#) on GitHub.

structure ChargedMedium[\[source\]](#)

Charged Medium is defined as Optical Medium with charge and current.

```
structure ChargedMedium extends OpticalMedium where
  ρ : ChargeDensity
  J : CurrentDensity
```

abbrev GaussLawElectric

[\[source\]](#)

Gauss's law for the Electric field in homogeneous medium.

```
abbrev ChargedMedium.GaussLawElectric (E : ElectricField) : Prop
```

abbrev GaussLawMagnetic

[\[source\]](#)

Gauss's law for the Magnetic field in homogeneous medium.

```
abbrev ChargedMedium.GaussLawMagnetic
  (B : MagneticField) : Prop
```

abbrev AmpereLaw

[\[source\]](#)

Ampère's law in homogeneous medium.

```
abbrev ChargedMedium.AmpereLaw (CM : ChargedMedium)
  (E : ElectricField) (B : MagneticField) : Prop
```

abbrev FaradayLaw

[\[source\]](#)

Faraday's law in homogeneous medium.

```
abbrev ChargedMedium.FaradayLaw
  (E : ElectricField) (B : MagneticField) : Prop
```

def free

[\[source\]](#)

Optical medium defined as charge and current free charged medium.

```
def OpticalMedium.free (OM : OpticalMedium) : ChargedMedium where
```

def FreeMaxwellEquations

[\[source\]](#)

The Maxwell equations for charge and current free medium.

```
def OpticalMedium.FreeMaxwellEquations (OM : OpticalMedium)
  (E : ElectricField) (B : MagneticField) : Prop
```

theorem gaussLawElectric_of_free

[\[source\]](#)

```
theorem OpticalMedium.gaussLawElectric_of_free (E : ElectricField) (B : MagneticField)
  (h : OM.FreeMaxwellEquations E B) :
  ε * (∇ · E t) x = 0
```

theorem gaussLawMagnetic_of_free[\[source\]](#)

```

theorem OpticalMedium.gaussLawMagnetic_of_free (E : ElectricField) (B : MagneticField)
  (h : OM.FreeMaxwellEquations E B) :
  ( $\nabla \cdot B$  t) x = 0

```

theorem ampereLaw_of_free[\[source\]](#)

```

theorem OpticalMedium.ampereLaw_of_free (E : ElectricField) (B : MagneticField)
  (h : OM.FreeMaxwellEquations E B) :
  ( $\nabla \times B$  t) x =  $\mu \cdot \varepsilon \cdot \partial_t$  (fun t => E t x) t

```

theorem faradayLaw_of_free[\[source\]](#)

```

theorem OpticalMedium.faradayLaw_of_free (E : ElectricField) (B : MagneticField)
  (h : OM.FreeMaxwellEquations E B) :
  ( $\nabla \times E$  t) x = -  $\partial_t$  (fun t => B t x) t

```

2.12 PhysLean.Electromagnetism.LorentzAction

[PhysLean/Electromagnetism/LorentzAction.lean](#) on GitHub.

semiformal_result lorentzAction[\[source\]](#)

The Lorentz action on the electric and magnetic fields.

```

semiformal_result "6WNUS" lorentzAction :
  MulAction (LorentzGroup 3) (ElectricField × MagneticField)

```

semiformal_result toElectricMagneticField_equivariant[\[source\]](#)

The equivalence toElectricMagneticField is equivariant with respect to the group action.

(In this semiformal result lorentzActionTemp should be replaced with lorentzAction.)

```

semiformal_result "6V204" toElectricMagneticField_equivariant (d : ℕ)
  (g : LorentzGroup 3) (E : ElectricField) (B : MagneticField)
  (lorentzActionTemp : (LorentzGroup 3) → (ElectricField × MagneticField)
    → (ElectricField × MagneticField)) (x : SpaceTime) :
  (toElectricMagneticField.symm (lorentzActionTemp g (E, B))).1 x =
  (realLorentzTensor.F.obj _).ρ g ((toElectricMagneticField.symm (E, B)).1 x)

```

2.13 PhysLean.Electromagnetism.MaxwellEquations

[PhysLean/Electromagnetism/MaxwellEquations.lean](#) on GitHub.

structure OpticalMedium[\[source\]](#)

An optcial medium refers to an isotropic medium (which may or may not be free space) which consists of the electric permittivity and the magnetic permeability.

```

structure OpticalMedium where
  ε : ℝ
  μ : ℝ

```

2.14 PhysLean.Mathematics.FDerivCurry

[PhysLean/Mathematics/FDerivCurry.lean](#) on GitHub.

lemma fderiv_uncurry

[\[source\]](#)

```
lemma fderiv_uncurry (f : X → Y → Z) (xy dxy : X × Y)
  (hf : DifferentiableAt ℚ (λf) xy) :
  fderiv ℚ λf xy dxy
  =
  fderiv ℚ (f · xy.2) xy.1 dxy.1 + fderiv ℚ (f xy.1 ·) xy.2 dxy.2
```

lemma fderiv_curry_fst

[\[source\]](#)

```
lemma fderiv_curry_fst (f : X × Y → Z) (x : X) (y : Y)
  (h : DifferentiableAt ℚ f (x,y)) (dx : X) :
  fderiv ℚ (fun x' => Function.curry f x' y) x dx = fderiv ℚ f (x,y) (dx, 0)
```

lemma fderiv_curry_snd

[\[source\]](#)

```
lemma fderiv_curry_snd (f : X × Y → Z) (x : X) (y : Y)
  (h : DifferentiableAt ℚ f (x,y)) (dy : Y) :
  fderiv ℚ (Function.curry f x) y dy = fderiv ℚ (f) (x,y) (0, dy)
```

lemma fderiv_uncurry_clm_comp

[\[source\]](#)

```
lemma fderiv_uncurry_clm_comp (f : X → Y → Z) (hf : Differentiable ℚ (λf)) :
  fderiv ℚ λf
  =
  fun xy =>
    (fderiv ℚ (f · xy.2) xy.1).comp (ContinuousLinearMap.fst ℚ X Y)
    +
    (fderiv ℚ (f xy.1 ·) xy.2).comp (ContinuousLinearMap.snd ℚ X Y)
```

lemma fderiv_wrt_prod_clm_comp

[\[source\]](#)

```
lemma fderiv_wrt_prod_clm_comp (f : X × Y → Z) (hf : Differentiable ℚ f) :
  fderiv ℚ f
  =
  fun xy =>
    (fderiv ℚ (fun x' => f (x',xy.2)) xy.1).comp (ContinuousLinearMap.fst ℚ X Y)
    +
    (fderiv ℚ (fun y' => f (xy.1,y')) xy.2).comp (ContinuousLinearMap.snd ℚ X Y)
```

lemma fderiv_curry_clm_apply

[\[source\]](#)

```
lemma fderiv_curry_clm_apply (f : X → Y →L[ℚ] Z) (y : Y) (x dx : X) (h : Differentiable ℚ f) :
  fderiv ℚ f x dx y
  =
  fderiv ℚ (f · y) x dx
```

lemma two_differentiable

[\[source\]](#)

Helper lemmas showing differentiability from ContDiff ℚ 2 λf.

```
lemma ContDiff.two_differentiable (f : X → Y → Z) (hf : ContDiff  $\mathbb{k}$  2  $\upharpoonright$ f) :
  Differentiable  $\mathbb{k}$  ( $\upharpoonright$ f)
```

lemma two_fderiv_differentiable

[\[source\]](#)

```
lemma ContDiff.two_fderiv_differentiable (f : X → Y → Z) (hf : ContDiff  $\mathbb{k}$  2  $\upharpoonright$ f) :
  Differentiable  $\mathbb{k}$  (fderiv  $\mathbb{k}$  ( $\upharpoonright$ f))
```

lemma fderiv_uncurry_comp_fst

[\[source\]](#)

```
lemma fderiv_uncurry_comp_fst (f : X → Y → Z) (y : Y) (hf : Differentiable  $\mathbb{k}$  ( $\upharpoonright$ f)) :
  fderiv  $\mathbb{k}$  (fun x' => ( $\upharpoonright$ f) (x', y))
=
  fun x => (fderiv  $\mathbb{k}$  ( $\upharpoonright$ f) (( $\cdot$ , y) x)).comp (fderiv  $\mathbb{k}$  ( $\cdot$ , y) x)
```

lemma fderiv_uncurry_comp_snd

[\[source\]](#)

```
lemma fderiv_uncurry_comp_snd (f : X → Y → Z) (x : X) (hf : Differentiable  $\mathbb{k}$  ( $\upharpoonright$ f)) :
  fderiv  $\mathbb{k}$  (fun y' => ( $\upharpoonright$ f) (x, y'))
=
  fun y => (fderiv  $\mathbb{k}$  ( $\upharpoonright$ f) ((x,  $\cdot$ ) y)).comp (fderiv  $\mathbb{k}$  (x,  $\cdot$ ) y)
```

lemma fderiv_curry_comp_fst

[\[source\]](#)

```
lemma fderiv_curry_comp_fst (f : X → Y → Z) (x dx : X) (y : Y)
  (hf : Differentiable  $\mathbb{k}$  ( $\upharpoonright$ f)) :
  (fderiv  $\mathbb{k}$  (fun x' => f x' y) x) dx
=
  (fderiv  $\mathbb{k}$  ( $\upharpoonright$ f) (( $\cdot$ , y) x)) ((fderiv  $\mathbb{k}$  ( $\cdot$ , y) x) dx)
```

lemma fderiv_curry_comp_snd

[\[source\]](#)

```
lemma fderiv_curry_comp_snd (f : X → Y → Z) (x : X) (y dy : Y)
  (hf : Differentiable  $\mathbb{k}$  ( $\upharpoonright$ f)) :
  (fderiv  $\mathbb{k}$  (fun y' => f x y') y) dy
=
  (fderiv  $\mathbb{k}$  ( $\upharpoonright$ f) ((x,  $\cdot$ ) y)) ((fderiv  $\mathbb{k}$  (x,  $\cdot$ ) y) dy)
```

lemma fderiv_inr_fst_clm

[\[source\]](#)

```
lemma fderiv_inr_fst_clm (x : X) (y : Y) :
  (fderiv  $\mathbb{k}$  (x,  $\cdot$ ) y) = ContinuousLinearMap.inr  $\mathbb{k}$  X Y
```

lemma fderiv_inl_snd_clm

[\[source\]](#)

```
lemma fderiv_inl_snd_clm (x : X) (y : Y) :
  (fderiv  $\mathbb{k}$  ( $\cdot$ , y) x) = ContinuousLinearMap.inl  $\mathbb{k}$  X Y
```

lemma function_differentiableAt_fst

[\[source\]](#)

```
lemma function_differentiableAt_fst (f : X → Y → Z) (x : X) (y : Y) (hf : Differentiable ℚ (λf)
) :
  DifferentiableAt ℚ (fun x' => f x' y) x
```

lemma function_differentiableAt_snd [\[source\]](#)

```
lemma function_differentiableAt_snd (f : X → Y → Z) (x : X) (y : Y) (hf : Differentiable ℚ (λf)
) :
  DifferentiableAt ℚ (fun y' => f x y') y
```

lemma fderiv_uncurry_differentiable_fst [\[source\]](#)

```
lemma fderiv_uncurry_differentiable_fst (f : X → Y → Z) (y : Y) (hf : ContDiff ℚ 2 λf) :
  Differentiable ℚ (fderiv ℚ fun x' => (λf) (x', y))
```

lemma fderiv_uncurry_differentiable_snd [\[source\]](#)

```
lemma fderiv_uncurry_differentiable_snd (f : X → Y → Z) (x : X) (hf : ContDiff ℚ 2 λf) :
  Differentiable ℚ (fderiv ℚ fun y' => (λf) (x, y'))
```

lemma fderiv_uncurry_differentiable_fst_comp_snd [\[source\]](#)

```
lemma fderiv_uncurry_differentiable_fst_comp_snd (f : X → Y → Z) (x : X) (hf : ContDiff ℚ 2 λf)
:
  Differentiable ℚ (fun y' => fderiv ℚ (fun x' => (λf) (x', y')) x)
```

lemma fderiv_uncurry_differentiable_snd_comp_fst [\[source\]](#)

```
lemma fderiv_uncurry_differentiable_snd_comp_fst (f : X → Y → Z) (y : Y) (hf : ContDiff ℚ 2 λf)
:
  Differentiable ℚ (fun x' => fderiv ℚ (fun y' => (λf) (x', y')) y)
```

lemma fderiv_curry_differentiableAt_fst_comp_snd [\[source\]](#)

```
lemma fderiv_curry_differentiableAt_fst_comp_snd (f : X → Y → Z) (x dx : X) (y : Y)
(hf : ContDiff ℚ 2 λf) :
  DifferentiableAt ℚ (fun y' => (fderiv ℚ (fun x' => f x' y') x) dx) y
```

lemma fderiv_curry_differentiableAt_snd_comp_fst [\[source\]](#)

```
lemma fderiv_curry_differentiableAt_snd_comp_fst (f : X → Y → Z) (x : X) (y dy : Y)
(hf : ContDiff ℚ 2 λf) :
  DifferentiableAt ℚ (fun x' => (fderiv ℚ (fun y' => f x' y') y) dy) x
```

lemma fderiv_swap [\[source\]](#)

```
lemma fderiv_swap [IsRCLikeNormedField ℚ] (f : X → Y → Z) (x dx : X) (y dy : Y)
(hf : ContDiff ℚ 2 λf) :
  fderiv ℚ (fun x' => fderiv ℚ (fun y' => f x' y') y dy) x dx
=
  fderiv ℚ (fun y' => fderiv ℚ (fun x' => f x' y') x dx) y dy
```

2.15 PhysLean.Meta.Informal.Post

[PhysLean/Meta/Informal/Post.lean](#) on GitHub.

def getTag [source]

Gets the tag associated with an informal definition or lemma.

```
unsafe def getTag (c : ConstantInfo) : CoreM String
```

2.16 PhysLean.Meta.Informal.SemiFormal

[PhysLean/Meta/Informal/SemiFormal.lean](#) on GitHub.

structure WantedInfo [source]

The information from a TODO ... command.

```
structure WantedInfo where
  content : String
  name : Name
  fileName : Name
  line : Nat
  tag : String
```

def «semiformal_result» [source]

A semiformal result is either a - definition in which the type is given but not the definition. - proof in which the proposition is given but not the proof. Semiformal results cannot be used in further code. They are essentially forgot about after made.

With minor modification they act in a similar way to `proof_wanted`, however they appear in PhysLean's TODO list and must be tagged accordingly. They must also always have a doc-string.

```
@[command_parser]
def «semiformal_result»
```

def elabLemmaWanted [source]

The elaborator for semiformal results.

```
@[command_elab «semiformal_result»]
def elabLemmaWanted : CommandElab
```

2.17 PhysLean.QFT.PerturbationTheory.FieldOpFreeAlgebra.Basic

[PhysLean/QFT/PerturbationTheory/FieldOpFreeAlgebra/Basic.lean](#) on GitHub.

lemma ofCrAnListF_injective [source]

```
lemma ofCrAnListF_injective : Function.Injective (ofCrAnListF (F := F))
```

2.18 PhysLean.QFT.PerturbationTheory.WickAlgebra.Basic

[PhysLean/QFT/PerturbationTheory/WickAlgebra/Basic.lean](#) on GitHub.

abbrev WickAlgebra

[\[source\]](#)

For a field specification $\mathcal{F}$, the algebra $\mathcal{F}.WickAlgebra$ is defined as the quotient of the free algebra $\mathcal{F}.FieldOpFreeAlgebra$ by the ideal generated by - $[ofCrAnOpF \phi c, ofCrAnOpF \phi c']_s \mathcal{F}$ for ϕc and $\phi c'$ field creation operators. This corresponds to the condition that two creation operators always super-commute. - $[ofCrAnOpF \phi a, ofCrAnOpF \phi a']_s \mathcal{F}$ for ϕa and $\phi a'$ field annihilation operators. This corresponds to the condition that two annihilation operators always super-commute. - $[ofCrAnOpF \phi, ofCrAnOpF \phi']_s \mathcal{F}$ for ϕ and ϕ' operators with different statistics. This corresponds to the condition that two operators with different statistics always super-commute. In other words, fermions and bosons always super-commute. - $[ofCrAnOpF \phi 1, [ofCrAnOpF \phi 2, ofCrAnOpF \phi 3]_s \mathcal{F}]_s \mathcal{F}$. This corresponds to the condition, when combined with the conditions above, that the super-commutator is in the center of the algebra.

abbrev WickAlgebra : Type

instance Setoid (FieldOpFreeAlgebra $\mathcal{F}$)

[\[source\]](#)

The instance of a setoid on $FieldOpFreeAlgebra$ from the ideal $TwoSidedIdeal$.

instance : Setoid (FieldOpFreeAlgebra $\mathcal{F}$)

lemma equiv_iff_sub_mem_ideal

[\[source\]](#)

lemma equiv_iff_sub_mem_ideal (x y : FieldOpFreeAlgebra $\mathcal{F}$) :
 $x \approx y \leftrightarrow x - y \in TwoSidedIdeal.span \mathcal{F}.fieldOpIdealSet$

lemma equiv_iff_exists_add

[\[source\]](#)

lemma equiv_iff_exists_add (x y : FieldOpFreeAlgebra $\mathcal{F}$) :
 $x \approx y \leftrightarrow \exists a, x = y + a \wedge a \in TwoSidedIdeal.span \mathcal{F}.fieldOpIdealSet$

def ι

[\[source\]](#)

For a field specification $\mathcal{F}$, ι is defined as the projection

$\mathcal{F}.FieldOpFreeAlgebra \rightarrow_a [\mathbb{C}] \mathcal{F}.WickAlgebra$

taking each element of $\mathcal{F}.FieldOpFreeAlgebra$ to its equivalence class in $WickAlgebra \mathcal{F}$.

def ι : FieldOpFreeAlgebra $\mathcal{F} \rightarrow_a [\mathbb{C}] WickAlgebra \mathcal{F}$ **where**

lemma ι _surjective

[\[source\]](#)

lemma ι _surjective : Function.Surjective (@ ι $\mathcal{F}$)

lemma ι_{apply} [\[source\]](#)

```
lemma  $\iota_{\text{apply}}$  (x : FieldOpFreeAlgebra  $\mathcal{F}$ ) :  $\iota$  x = Quotient.mk _ x
```

lemma $\iota_{\text{of_mem_fieldOpIdealSet}}$ [\[source\]](#)

```
lemma  $\iota_{\text{of\_mem\_fieldOpIdealSet}}$  (x : FieldOpFreeAlgebra  $\mathcal{F}$ ) (hx : x  $\in$   $\mathcal{F}$ .fieldOpIdealSet) :  
   $\iota$  x = 0
```

lemma $\iota_{\text{superCommuteF_of_create_create}}$ [\[source\]](#)

```
lemma  $\iota_{\text{superCommuteF\_of\_create\_create}}$  ( $\varphi$   $\varphi'$  :  $\mathcal{F}$ .CrAnFieldOp) ( $h\varphi$  :  $\mathcal{F} \mid^{>c} \varphi = \text{.create}$ )  
  ( $h\varphi'$  :  $\mathcal{F} \mid^{>c} \varphi' = \text{.create}$ ) :  $\iota$  [ofCrAnOpF  $\varphi$ , ofCrAnOpF  $\varphi'$ ] $_s$ F = 0
```

lemma $\iota_{\text{superCommuteF_of_annihilate_annihilate}}$ [\[source\]](#)

```
lemma  $\iota_{\text{superCommuteF\_of\_annihilate\_annihilate}}$  ( $\varphi$   $\varphi'$  :  $\mathcal{F}$ .CrAnFieldOp)  
  ( $h\varphi$  :  $\mathcal{F} \mid^{>c} \varphi = \text{.annihilate}$ ) ( $h\varphi'$  :  $\mathcal{F} \mid^{>c} \varphi' = \text{.annihilate}$ ) :  
   $\iota$  [ofCrAnOpF  $\varphi$ , ofCrAnOpF  $\varphi'$ ] $_s$ F = 0
```

lemma $\iota_{\text{superCommuteF_of_diff_statistic}}$ [\[source\]](#)

```
lemma  $\iota_{\text{superCommuteF\_of\_diff\_statistic}}$  { $\varphi$   $\psi$  :  $\mathcal{F}$ .CrAnFieldOp}  
  (h : ( $\mathcal{F} \mid^{>s} \varphi$ )  $\neq$  ( $\mathcal{F} \mid^{>s} \psi$ )) :  $\iota$  [ofCrAnOpF  $\varphi$ , ofCrAnOpF  $\psi$ ] $_s$ F = 0
```

lemma $\iota_{\text{superCommuteF_zero_of_fermionic}}$ [\[source\]](#)

```
lemma  $\iota_{\text{superCommuteF\_zero\_of\_fermionic}}$  ( $\varphi$   $\psi$  :  $\mathcal{F}$ .CrAnFieldOp)  
  (h : [ofCrAnOpF  $\varphi$ , ofCrAnOpF  $\psi$ ] $_s$ F  $\in$  statisticSubmodule fermionic) :  
   $\iota$  [ofCrAnOpF  $\varphi$ , ofCrAnOpF  $\psi$ ] $_s$ F = 0
```

lemma $\iota_{\text{superCommuteF_ofCrAnOpF_ofCrAnOpF_bosonic_or_zero}}$ [\[source\]](#)

```
lemma  $\iota_{\text{superCommuteF\_ofCrAnOpF\_ofCrAnOpF\_bosonic\_or\_zero}}$  ( $\varphi$   $\psi$  :  $\mathcal{F}$ .CrAnFieldOp) :  
  [ofCrAnOpF  $\varphi$ , ofCrAnOpF  $\psi$ ] $_s$ F  $\in$  statisticSubmodule bosonic  $\vee$   
   $\iota$  [ofCrAnOpF  $\varphi$ , ofCrAnOpF  $\psi$ ] $_s$ F = 0
```

lemma $\iota_{\text{superCommuteF_ofCrAnOpF_superCommuteF_ofCrAnOpF_ofCrAnOpF}}$ [\[source\]](#)

```
@[simp]  
lemma  $\iota_{\text{superCommuteF\_ofCrAnOpF\_superCommuteF\_ofCrAnOpF\_ofCrAnOpF}}$  ( $\varphi_1$   $\varphi_2$   $\varphi_3$  :  $\mathcal{F}$ .CrAnFieldOp) :  
   $\iota$  [ofCrAnOpF  $\varphi_1$ , [ofCrAnOpF  $\varphi_2$ , ofCrAnOpF  $\varphi_3$ ] $_s$ F] $_s$ F = 0
```

lemma $\iota_{\text{superCommuteF_superCommuteF_ofCrAnOpF_ofCrAnOpF_ofCrAnOpF}}$ [\[source\]](#)

```
lemma  $\iota_{\text{superCommuteF\_superCommuteF\_ofCrAnOpF\_ofCrAnOpF\_ofCrAnOpF}}$  ( $\varphi_1$   $\varphi_2$   $\varphi_3$  :  $\mathcal{F}$ .CrAnFieldOp) :  
   $\iota$  [[ofCrAnOpF  $\varphi_1$ , ofCrAnOpF  $\varphi_2$ ] $_s$ F, ofCrAnOpF  $\varphi_3$ ] $_s$ F = 0
```

lemma $\iota_{\text{superCommuteF_superCommuteF_ofCrAnOpF_ofCrAnOpF_ofCrAnListF}}$ [\[source\]](#)

```
lemma ι_superCommuteF_superCommuteF_ofCrAnOpF_ofCrAnOpF_ofCrAnListF (φ1 φ2 :  $\mathcal{F}$ .CrAnFieldOp)
  (φs : List  $\mathcal{F}$ .CrAnFieldOp) :
  ι [[ofCrAnOpF φ1, ofCrAnOpF φ2]sF, ofCrAnListF φs]sF = 0
```

lemma ι_superCommuteF_superCommuteF_ofCrAnOpF_ofCrAnOpF_fieldOpFreeAlgebra [\[source\]](#)

```
@[simp]
lemma ι_superCommuteF_superCommuteF_ofCrAnOpF_ofCrAnOpF_fieldOpFreeAlgebra (φ1 φ2 :  $\mathcal{F}$ .
  CrAnFieldOp)
  (a :  $\mathcal{F}$ .FieldOpFreeAlgebra) : ι [[ofCrAnOpF φ1, ofCrAnOpF φ2]sF, a]sF = 0
```

lemma ι_commute_fieldOpFreeAlgebra_superCommuteF_ofCrAnOpF_ofCrAnOpF [\[source\]](#)

```
lemma ι_commute_fieldOpFreeAlgebra_superCommuteF_ofCrAnOpF_ofCrAnOpF (φ1 φ2 :  $\mathcal{F}$ .CrAnFieldOp)
  (a :  $\mathcal{F}$ .FieldOpFreeAlgebra) : ι a * ι [ofCrAnOpF φ1, ofCrAnOpF φ2]sF -
  ι [ofCrAnOpF φ1, ofCrAnOpF φ2]sF * ι a = 0
```

lemma ι_superCommuteF_ofCrAnOpF_ofCrAnOpF_mem_center [\[source\]](#)

```
lemma ι_superCommuteF_ofCrAnOpF_ofCrAnOpF_mem_center (φ ψ :  $\mathcal{F}$ .CrAnFieldOp) :
  ι [ofCrAnOpF φ, ofCrAnOpF ψ]sF ∈ Subalgebra.center  $\subset$   $\mathcal{F}$ .WickAlgebra
```

lemma ι_eq_zero_iff_mem_ideal [\[source\]](#)

```
lemma ι_eq_zero_iff_mem_ideal (x : FieldOpFreeAlgebra  $\mathcal{F}$ ) :
  ι x = 0  $\leftrightarrow$  x ∈ TwoSidedIdeal.span  $\mathcal{F}$ .fieldOpIdealSet
```

lemma bosonicProjF_mem_fieldOpIdealSet_or_zero [\[source\]](#)

```
lemma bosonicProjF_mem_fieldOpIdealSet_or_zero (x : FieldOpFreeAlgebra  $\mathcal{F}$ )
  (hx : x ∈  $\mathcal{F}$ .fieldOpIdealSet) :
  x.bosonicProjF.1 ∈  $\mathcal{F}$ .fieldOpIdealSet  $\vee$  x.bosonicProjF = 0
```

lemma fermionicProjF_mem_fieldOpIdealSet_or_zero [\[source\]](#)

```
lemma fermionicProjF_mem_fieldOpIdealSet_or_zero (x : FieldOpFreeAlgebra  $\mathcal{F}$ )
  (hx : x ∈  $\mathcal{F}$ .fieldOpIdealSet) :
  x.fermionicProjF.1 ∈  $\mathcal{F}$ .fieldOpIdealSet  $\vee$  x.fermionicProjF = 0
```

lemma bosonicProjF_mem_ideal [\[source\]](#)

```
lemma bosonicProjF_mem_ideal (x : FieldOpFreeAlgebra  $\mathcal{F}$ )
  (hx : x ∈ TwoSidedIdeal.span  $\mathcal{F}$ .fieldOpIdealSet) :
  x.bosonicProjF.1 ∈ TwoSidedIdeal.span  $\mathcal{F}$ .fieldOpIdealSet
```

lemma fermionicProjF_mem_ideal [\[source\]](#)

```
lemma fermionicProjF_mem_ideal (x : FieldOpFreeAlgebra  $\mathcal{F}$ )
  (hx : x ∈ TwoSidedIdeal.span  $\mathcal{F}$ .fieldOpIdealSet) :
  x.fermionicProjF.1 ∈ TwoSidedIdeal.span  $\mathcal{F}$ .fieldOpIdealSet
```

lemma ι_eq_zero_iff_ι_bosonicProjF_fermonicProj_zero[\[source\]](#)

lemma ι_eq_zero_iff_ι_bosonicProjF_fermonicProj_zero (x : FieldOpFreeAlgebra $\mathcal{F}$) :
 ι x = 0 ↔ ι x.bosonicProjF.1 = 0 ∧ ι x.fermionicProjF.1 = 0

def ofFieldOp[\[source\]](#)

For a field specification $\mathcal{F}$ and an element ϕ of $\mathcal{F}.FieldOp$, ofFieldOp ϕ is defined as the element of $\mathcal{F}.WickAlgebra$ given by ι (ofFieldOpF ϕ).

def ofFieldOp (ϕ : $\mathcal{F}.FieldOp$) : $\mathcal{F}.WickAlgebra$

lemma ofFieldOp_eq_ι_ofFieldOpF[\[source\]](#)

lemma ofFieldOp_eq_ι_ofFieldOpF (ϕ : $\mathcal{F}.FieldOp$) : ofFieldOp ϕ = ι (ofFieldOpF ϕ)

def ofFieldOpList[\[source\]](#)

For a field specification $\mathcal{F}$ and a list ϕs of $\mathcal{F}.FieldOp$, ofFieldOpList ϕs is defined as the element of $\mathcal{F}.WickAlgebra$ given by ι (ofFieldOpListF ϕ).

def ofFieldOpList (ϕs : List $\mathcal{F}.FieldOp$) : $\mathcal{F}.WickAlgebra$

lemma ofFieldOpList_eq_ι_ofFieldOpListF[\[source\]](#)

lemma ofFieldOpList_eq_ι_ofFieldOpListF (ϕs : List $\mathcal{F}.FieldOp$) :
 ofFieldOpList ϕs = ι (ofFieldOpListF ϕs)

lemma ofFieldOpList_append[\[source\]](#)

lemma ofFieldOpList_append (ϕs ψs : List $\mathcal{F}.FieldOp$) :
 ofFieldOpList (ϕs ++ ψs) = ofFieldOpList ϕs * ofFieldOpList ψs

lemma ofFieldOpList_cons[\[source\]](#)

lemma ofFieldOpList_cons (ϕ : $\mathcal{F}.FieldOp$) (ϕs : List $\mathcal{F}.FieldOp$) :
 ofFieldOpList (ϕ :: ϕs) = ofFieldOp ϕ * ofFieldOpList ϕs

lemma ofFieldOpList_singleton[\[source\]](#)

lemma ofFieldOpList_singleton (ϕ : $\mathcal{F}.FieldOp$) :
 ofFieldOpList [ϕ] = ofFieldOp ϕ

def ofCrAnOp[\[source\]](#)

For a field specification $\mathcal{F}$ and an element ϕ of $\mathcal{F}.CrAnFieldOp$, ofCrAnOp ϕ is defined as the element of $\mathcal{F}.WickAlgebra$ given by ι (ofCrAnOpF ϕ).

def ofCrAnOp (ϕ : $\mathcal{F}.CrAnFieldOp$) : $\mathcal{F}.WickAlgebra$

lemma ofCrAnOp_eq_ι_ofCrAnOpF[\[source\]](#)

```
lemma ofCrAnOp_eq_ι_ofCrAnOpF (φ :  $\mathcal{F}$ .CrAnFieldOp) :
  ofCrAnOp φ = ι (ofCrAnOpF φ)
```

lemma ofFieldOp_eq_sum[\[source\]](#)

```
lemma ofFieldOp_eq_sum (φ :  $\mathcal{F}$ .FieldOp) :
  ofFieldOp φ = ( $\sum$  i :  $\mathcal{F}$ .fieldOpToCrAnType φ, ofCrAnOp (φ, i))
```

def ofCrAnList[\[source\]](#)

For a field specification $\mathcal{F}$ and a list φs of $\mathcal{F}$.CrAnFieldOp, *ofCrAnList* φs is defined as the element of $\mathcal{F}$.WickAlgebra given by ι (*ofCrAnListF* φ).

```
def ofCrAnList (φs : List  $\mathcal{F}$ .CrAnFieldOp) :  $\mathcal{F}$ .WickAlgebra
```

lemma ofCrAnList_eq_ι_ofCrAnListF[\[source\]](#)

```
lemma ofCrAnList_eq_ι_ofCrAnListF (φs : List  $\mathcal{F}$ .CrAnFieldOp) :
  ofCrAnList φs = ι (ofCrAnListF φs)
```

lemma ofCrAnList_append[\[source\]](#)

```
lemma ofCrAnList_append (φs ψs : List  $\mathcal{F}$ .CrAnFieldOp) :
  ofCrAnList (φs ++ ψs) = ofCrAnList φs * ofCrAnList ψs
```

lemma ofCrAnList_singleton[\[source\]](#)

```
lemma ofCrAnList_singleton (φ :  $\mathcal{F}$ .CrAnFieldOp) :
  ofCrAnList [φ] = ofCrAnOp φ
```

lemma ofFieldOpList_eq_sum[\[source\]](#)

```
lemma ofFieldOpList_eq_sum (φs : List  $\mathcal{F}$ .FieldOp) :
  ofFieldOpList φs =  $\sum$  s : CrAnSection φs, ofCrAnList s.1
```

def anPart[\[source\]](#)

For a field specification $\mathcal{F}$, and an element φ of $\mathcal{F}$.FieldOp, the annihilation part of $\mathcal{F}$.FieldOp as an element of $\mathcal{F}$.WickAlgebra. Thus for φ - an incoming asymptotic state this is $\emptyset$. - a position based state this is *ofCrAnOp* (φ , *.create*). - an outgoing asymptotic state this is *ofCrAnOp* (φ , *()*).

```
def anPart (φ :  $\mathcal{F}$ .FieldOp) :  $\mathcal{F}$ .WickAlgebra
```

lemma anPart_eq_ι_anPartF[\[source\]](#)

```
lemma anPart_eq_ι_anPartF (φ :  $\mathcal{F}$ .FieldOp) : anPart φ = ι (anPartF φ)
```

lemma anPart_inAsymp[\[source\]](#)

```
@[simp]
lemma anPart_inAsymp (φ : (Σ f,  $\mathcal{F}$ .AsymptoticLabel f) × Momentum) :
  anPart (FieldOp.inAsymp φ) = 0
```

lemma anPart_position[\[source\]](#)

```
@[simp]
lemma anPart_position (φ : (Σ f,  $\mathcal{F}$ .PositionLabel f) × SpaceTime) :
  anPart (FieldOp.position φ) =
    ofCrAnOp (FieldOp.position φ, CreateAnnihilate.annihilate)
```

lemma anPart_outAsymp[\[source\]](#)

```
@[simp]
lemma anPart_outAsymp (φ : (Σ f,  $\mathcal{F}$ .AsymptoticLabel f) × Momentum) :
  anPart (FieldOp.outAsymp φ) = ofCrAnOp (FieldOp.outAsymp φ, ())
```

def crPart[\[source\]](#)

For a field specification $\mathcal{F}$, and an element ϕ of $\mathcal{F}$.FieldOp, the creation part of $\mathcal{F}$.FieldOp as an element of $\mathcal{F}$.WickAlgebra. Thus for ϕ - an incoming asymptotic state this is ofCrAnOp $\langle \phi, () \rangle$. - a position based state this is ofCrAnOp $\langle \phi, .create \rangle$. - an outgoing asymptotic state this is 0.

```
def crPart (φ :  $\mathcal{F}$ .FieldOp) :  $\mathcal{F}$ .WickAlgebra
```

lemma crPart_eq_ι_crPartF[\[source\]](#)

```
lemma crPart_eq_ι_crPartF (φ :  $\mathcal{F}$ .FieldOp) : crPart φ = ι (crPartF φ)
```

lemma crPart_inAsymp[\[source\]](#)

```
@[simp]
lemma crPart_inAsymp (φ : (Σ f,  $\mathcal{F}$ .AsymptoticLabel f) × Momentum) :
  crPart (FieldOp.inAsymp φ) = ofCrAnOp (FieldOp.inAsymp φ, ())
```

lemma crPart_position[\[source\]](#)

```
@[simp]
lemma crPart_position (φ : (Σ f,  $\mathcal{F}$ .PositionLabel f) × SpaceTime) :
  crPart (FieldOp.position φ) =
    ofCrAnOp (FieldOp.position φ, CreateAnnihilate.create)
```

lemma crPart_outAsymp[\[source\]](#)

```
@[simp]
lemma crPart_outAsymp (φ : (Σ f,  $\mathcal{F}$ .AsymptoticLabel f) × Momentum) :
  crPart (FieldOp.outAsymp φ) = 0
```

lemma ofFieldOp_eq_crPart_add_anPart[\[source\]](#)

For field specification $\mathcal{F}$, and an element ϕ of $\mathcal{F}.FieldOp$ the following relation holds:

$$ofFieldOp \ \phi = crPart \ \phi + anPart \ \phi$$

That is, every field operator splits into its creation part plus its annihilation part.

```
lemma ofFieldOp_eq_crPart_add_anPart (ϕ : F.FieldOp) :
  ofFieldOp ϕ = crPart ϕ + anPart ϕ
```

lemma anPart_outAsymp_eq_ofFieldOp[\[source\]](#)

```
lemma anPart_outAsymp_eq_ofFieldOp (ϕ : (Σ f, F.AsymptoticLabel f) × Momentum) :
  anPart (FieldOp.outAsymp ϕ) = ofFieldOp (FieldOp.outAsymp ϕ)
```

lemma crPart_inAsymp_eq_ofFieldOp[\[source\]](#)

```
lemma crPart_inAsymp_eq_ofFieldOp (ϕ : (Σ f, F.AsymptoticLabel f) × Momentum) :
  crPart (FieldOp.inAsymp ϕ) = ofFieldOp (FieldOp.inAsymp ϕ)
```

2.19 PhysLean.QFT.PerturbationTheory.WickAlgebra.Grading

[PhysLean/QFT/PerturbationTheory/WickAlgebra/Grading.lean](#) on GitHub.

def statSubmodule[\[source\]](#)

The submodule of $\mathcal{F}.WickAlgebra$ spanned by lists of field statistic f .

```
def statSubmodule (f : FieldStatistic) : Submodule C F.WickAlgebra
```

lemma ofCrAnList_mem_statSubmodule_of_eq[\[source\]](#)

```
lemma ofCrAnList_mem_statSubmodule_of_eq (ϕs : List F.CrAnFieldOp) (f : FieldStatistic)
  (h : (F |>_s ϕs) = f) : ofCrAnList ϕs ∈ statSubmodule f
```

lemma ofCrAnList_mem_statSubmodule[\[source\]](#)

```
lemma ofCrAnList_mem_statSubmodule (ϕs : List F.CrAnFieldOp) :
  ofCrAnList ϕs ∈ statSubmodule (F |>_s ϕs)
```

lemma mem_bosonic_of_mem_free_bosonic[\[source\]](#)

```
lemma mem_bosonic_of_mem_free_bosonic (a : F.FieldOpFreeAlgebra)
  (h : a ∈ statisticSubmodule bosonic) : ∃ a ∈ statSubmodule .bosonic
```

lemma mem_fermionic_of_mem_free_fermionic[\[source\]](#)

```
lemma mem_fermionic_of_mem_free_fermionic (a : F.FieldOpFreeAlgebra)
  (h : a ∈ statisticSubmodule fermionic) : ∃ a ∈ statSubmodule .fermionic
```

lemma mem_statSubmodule_of_mem_statisticSubmodule[\[source\]](#)

```
lemma mem_statSubmodule_of_mem_statisticSubmodule (f : FieldStatistic) (a :  $\mathcal{F}$ .
  FieldOpFreeAlgebra)
  (h : a ∈ statisticSubmodule f) :  $\iota$  a ∈ statSubmodule f
```

def ι StateSubmodule[\[source\]](#)

The projection of statisticSubmodule ($\mathcal{F} := \mathcal{F}$) f defined in the free algebra to statSubmodule ($\mathcal{F} := \mathcal{F}$) f .

```
def  $\iota$ StateSubmodule (f : FieldStatistic) :
  statisticSubmodule ( $\mathcal{F} := \mathcal{F}$ )  $f \rightarrow_l[\mathbb{C}]$  statSubmodule ( $\mathcal{F} := \mathcal{F}$ )  $f$  where
```

def bosonicProjFree[\[source\]](#)

The projection of $\mathcal{F}$.FieldOpFreeAlgebra to statSubmodule ($\mathcal{F} := \mathcal{F}$) bosonic.

```
def bosonicProjFree :  $\mathcal{F}$ .FieldOpFreeAlgebra  $\rightarrow_l[\mathbb{C}]$  statSubmodule ( $\mathcal{F} := \mathcal{F}$ ) bosonic
```

lemma bosonicProjFree_eq_ ι _bosonicProjF[\[source\]](#)

```
lemma bosonicProjFree_eq_ $\iota$ _bosonicProjF (a :  $\mathcal{F}$ .FieldOpFreeAlgebra) :
  (bosonicProjFree a).1 =  $\iota$  (bosonicProjF a)
```

lemma bosonicProjFree_zero_of_ ι _zero[\[source\]](#)

```
lemma bosonicProjFree_zero_of_ $\iota$ _zero (a :  $\mathcal{F}$ .FieldOpFreeAlgebra) (h :  $\iota$  a = 0) :
  bosonicProjFree a = 0
```

lemma bosonicProjFree_eq_of_equiv[\[source\]](#)

```
lemma bosonicProjFree_eq_of_equiv (a b :  $\mathcal{F}$ .FieldOpFreeAlgebra) (h : a  $\approx$  b) :
  bosonicProjFree a = bosonicProjFree b
```

def bosonicProj[\[source\]](#)

The projection of $\mathcal{F}$.WickAlgebra to statSubmodule ($\mathcal{F} := \mathcal{F}$) bosonic.

```
def bosonicProj :  $\mathcal{F}$ .WickAlgebra  $\rightarrow_l[\mathbb{C}]$  statSubmodule ( $\mathcal{F} := \mathcal{F}$ ) bosonic where
```

lemma bosonicProj_eq_bosonicProjFree[\[source\]](#)

```
lemma bosonicProj_eq_bosonicProjFree (a :  $\mathcal{F}$ .FieldOpFreeAlgebra) :
  bosonicProj ( $\iota$  a) = bosonicProjFree a
```

def fermionicProjFree[\[source\]](#)

The projection of $\mathcal{F}$.FieldOpFreeAlgebra to statSubmodule ($\mathcal{F} := \mathcal{F}$) fermionic.

```
def fermionicProjFree :  $\mathcal{F}$ .FieldOpFreeAlgebra  $\rightarrow_l[\mathbb{C}]$  statSubmodule ( $\mathcal{F} := \mathcal{F}$ ) fermionic
```

lemma fermionicProjFree_eq_ι_fermionicProjF[\[source\]](#)

```
lemma fermionicProjFree_eq_ι_fermionicProjF (a :  $\mathcal{F}$ .FieldOpFreeAlgebra) :
  (fermionicProjFree a).1 = ι (fermionicProjF a)
```

lemma fermionicProjFree_zero_of_ι_zero[\[source\]](#)

```
lemma fermionicProjFree_zero_of_ι_zero (a :  $\mathcal{F}$ .FieldOpFreeAlgebra) (h : ι a = 0) :
  fermionicProjFree a = 0
```

lemma fermionicProjFree_eq_of_equiv[\[source\]](#)

```
lemma fermionicProjFree_eq_of_equiv (a b :  $\mathcal{F}$ .FieldOpFreeAlgebra) (h : a ≈ b) :
  fermionicProjFree a = fermionicProjFree b
```

def fermionicProj[\[source\]](#)

The projection of $\mathcal{F}$.WickAlgebra to statSubmodule ($\mathcal{F} := \mathcal{F}$) fermionic.

```
def fermionicProj :  $\mathcal{F}$ .WickAlgebra  $\rightarrow_l$  [C] statSubmodule ( $\mathcal{F} := \mathcal{F}$ ) fermionic where
```

lemma fermionicProj_eq_fermionicProjFree[\[source\]](#)

```
lemma fermionicProj_eq_fermionicProjFree (a :  $\mathcal{F}$ .FieldOpFreeAlgebra) :
  fermionicProj (ι a) = fermionicProjFree a
```

lemma bosonicProj_add_fermionicProj[\[source\]](#)

```
lemma bosonicProj_add_fermionicProj (a :  $\mathcal{F}$ .WickAlgebra) :
  bosonicProj a + (fermionicProj a).1 = a
```

lemma bosonicProj_mem_bosonic[\[source\]](#)

```
lemma bosonicProj_mem_bosonic (a :  $\mathcal{F}$ .WickAlgebra) (ha : a ∈ statSubmodule .bosonic) :
  bosonicProj a = ⟨a, ha⟩
```

lemma fermionicProj_mem_fermionic[\[source\]](#)

```
lemma fermionicProj_mem_fermionic (a :  $\mathcal{F}$ .WickAlgebra) (ha : a ∈ statSubmodule .fermionic) :
  fermionicProj a = ⟨a, ha⟩
```

lemma bosonicProj_mem_fermionic[\[source\]](#)

```
lemma bosonicProj_mem_fermionic (a :  $\mathcal{F}$ .WickAlgebra) (ha : a ∈ statSubmodule .fermionic) :
  bosonicProj a = 0
```

lemma fermionicProj_mem_bosonic[\[source\]](#)

```
lemma fermionicProj_mem_bosonic (a :  $\mathcal{F}$ .WickAlgebra) (ha : a ∈ statSubmodule .bosonic) :
  fermionicProj a = 0
```

lemma mem_bosonic_iff_fermionicProj_eq_zero[\[source\]](#)

```
lemma mem_bosonic_iff_fermionicProj_eq_zero (a :  $\mathcal{F}$ .WickAlgebra) :
  a ∈ statSubmodule bosonic ↔ fermionicProj a = 0
```

lemma mem_fermionic_iff_bosonicProj_eq_zero[\[source\]](#)

```
lemma mem_fermionic_iff_bosonicProj_eq_zero (a :  $\mathcal{F}$ .WickAlgebra) :
  a ∈ statSubmodule fermionic ↔ bosonicProj a = 0
```

lemma eq_zero_of_bosonic_and_fermionic[\[source\]](#)

```
lemma eq_zero_of_bosonic_and_fermionic {a :  $\mathcal{F}$ .WickAlgebra}
  (hb : a ∈ statSubmodule bosonic) (hf : a ∈ statSubmodule fermionic) : a = 0
```

lemma bosonicProj_fermionicProj_eq_zero[\[source\]](#)

```
@[simp]
lemma bosonicProj_fermionicProj_eq_zero (a :  $\mathcal{F}$ .WickAlgebra) :
  bosonicProj (fermionicProj a).1 = 0
```

lemma fermionicProj_bosonicProj_eq_zero[\[source\]](#)

```
@[simp]
lemma fermionicProj_bosonicProj_eq_zero (a :  $\mathcal{F}$ .WickAlgebra) :
  fermionicProj (bosonicProj a).1 = 0
```

lemma bosonicProj_bosonicProj_eq_bosonicProj[\[source\]](#)

```
@[simp]
lemma bosonicProj_bosonicProj_eq_bosonicProj (a :  $\mathcal{F}$ .WickAlgebra) :
  bosonicProj (bosonicProj a).1 = bosonicProj a
```

lemma fermionicProj_fermionicProj_eq_fermionicProj[\[source\]](#)

```
@[simp]
lemma fermionicProj_fermionicProj_eq_fermionicProj (a :  $\mathcal{F}$ .WickAlgebra) :
  fermionicProj (fermionicProj a).1 = fermionicProj a
```

lemma bosonicProj_of_bosonic_part[\[source\]](#)

```
@[simp]
lemma bosonicProj_of_bosonic_part
  (a : DirectSum FieldStatistic (fun i => (statSubmodule ( $\mathcal{F}$  :=  $\mathcal{F}$ ) i))) :
  bosonicProj (a bosonic).1 = (a bosonic)
```

lemma bosonicProj_of_fermionic_part[\[source\]](#)

```
@[simp]
lemma bosonicProj_of_fermionic_part
  (a : DirectSum FieldStatistic (fun i => (statSubmodule ( $\mathcal{F}$  :=  $\mathcal{F}$ ) i))) :
  bosonicProj (a fermionic).1 = 0
```

lemma fermionicProj_of_bosonic_part[\[source\]](#)

```
@[simp]
lemma fermionicProj_of_bosonic_part
  (a : DirectSum FieldStatistic (fun i => (statSubmodule (F := F) i))) :
  fermionicProj (a bosonic).1 = 0
```

lemma fermionicProj_of_fermionic_part[\[source\]](#)

```
@[simp]
lemma fermionicProj_of_fermionic_part
  (a : DirectSum FieldStatistic (fun i => (statSubmodule (F := F) i))) :
  fermionicProj (a fermionic).1 = (a fermionic)
```

lemma coeAddMonoidHom_apply_eq_bosonic_plus_fermionic[\[source\]](#)

```
lemma coeAddMonoidHom_apply_eq_bosonic_plus_fermionic
  (a : DirectSum FieldStatistic (fun i => (statSubmodule (F := F) i))) :
  DirectSum.coeAddMonoidHom statSubmodule a = a.1 bosonic + a.1 fermionic
```

lemma directSum_eq_bosonic_plus_fermionic[\[source\]](#)

```
lemma directSum_eq_bosonic_plus_fermionic
  (a : DirectSum FieldStatistic (fun i => (statSubmodule (F := F) i))) :
  a = (DirectSum.of (fun i => 1(statSubmodule (F := F) i)) bosonic) (a bosonic) +
  (DirectSum.of (fun i => 1(statSubmodule (F := F) i)) fermionic) (a fermionic)
```

instance WickAlgebraGrade[\[source\]](#)

For a field statistic $\mathcal{F}$, the algebra $\mathcal{F}.WickAlgebra$ is graded by $FieldStatistic$. Those of $CrAnList$ ϕs for which ϕs has an overall bosonic statistic (i.e. $\mathcal{F} \mid_{>s} \phi s = \text{bosonic}$) span bosonic submodule, whilst those of $CrAnList$ ϕs for which ϕs has an overall fermionic statistic (i.e. $\mathcal{F} \mid_{>s} \phi s = \text{fermionic}$) span the fermionic submodule.

```
instance WickAlgebraGrade : GradedAlgebra (A := F.WickAlgebra) statSubmodule where
```

2.20 PhysLean.QFT.PerturbationTheory.WickAlgebra.NormalOrder.Basic

[PhysLean/QFT/PerturbationTheory/WickAlgebra/NormalOrder/Basic.lean](#) on GitHub.

lemma 1_normalOrderF_superCommuteF_ofCrAnListF_ofCrAnListF_eq_zero[\[source\]](#)

```
lemma 1_normalOrderF_superCommuteF_ofCrAnListF_ofCrAnListF_eq_zero
  (φa φa' : F.CrAnFieldOp) (φs φs' : List F.CrAnFieldOp) :
  1 Nf(ofCrAnListF φs * [ofCrAnOpF φa, ofCrAnOpF φa']sF * ofCrAnListF φs') = 0
```

lemma 1_normalOrderF_superCommuteF_ofCrAnListF_eq_zero[\[source\]](#)

```
lemma 1_normalOrderF_superCommuteF_ofCrAnListF_eq_zero
  (φa φa' : F.CrAnFieldOp) (φs : List F.CrAnFieldOp)
  (a : F.FieldOpFreeAlgebra) :
  1 Nf(ofCrAnListF φs * [ofCrAnOpF φa, ofCrAnOpF φa']sF * a) = 0
```

lemma $\mathfrak{I}_{\text{normalOrderF_superCommuteF_ofCrAnOpF_eq_zero_mul}}$ [\[source\]](#)

```

lemma  $\mathfrak{I}_{\text{normalOrderF\_superCommuteF\_ofCrAnOpF\_eq\_zero\_mul}}$  ( $\phi a \ \phi a' : \mathcal{F}.\text{CrAnFieldOp}$ )
  ( $a \ b : \mathcal{F}.\text{FieldOpFreeAlgebra}$ ) :
   $\mathfrak{I} \mathcal{N}^f(a * [\text{ofCrAnOpF } \phi a, \text{ofCrAnOpF } \phi a']_sF * b) = 0$ 

```

lemma $\mathfrak{I}_{\text{normalOrderF_superCommuteF_ofCrAnOpF_ofCrAnListF_eq_zero_mul}}$ [\[source\]](#)

```

lemma  $\mathfrak{I}_{\text{normalOrderF\_superCommuteF\_ofCrAnOpF\_ofCrAnListF\_eq\_zero\_mul}}$  ( $\phi a : \mathcal{F}.\text{CrAnFieldOp}$ )
  ( $\phi s : \text{List } \mathcal{F}.\text{CrAnFieldOp}$ ) ( $a \ b : \mathcal{F}.\text{FieldOpFreeAlgebra}$ ) :
   $\mathfrak{I} \mathcal{N}^f(a * [\text{ofCrAnOpF } \phi a, \text{ofCrAnListF } \phi s]_sF * b) = 0$ 

```

lemma $\mathfrak{I}_{\text{normalOrderF_superCommuteF_ofCrAnListF_ofCrAnOpF_eq_zero_mul}}$ [\[source\]](#)

```

lemma  $\mathfrak{I}_{\text{normalOrderF\_superCommuteF\_ofCrAnListF\_ofCrAnOpF\_eq\_zero\_mul}}$  ( $\phi a : \mathcal{F}.\text{CrAnFieldOp}$ )
  ( $\phi s : \text{List } \mathcal{F}.\text{CrAnFieldOp}$ ) ( $a \ b : \mathcal{F}.\text{FieldOpFreeAlgebra}$ ) :
   $\mathfrak{I} \mathcal{N}^f(a * [\text{ofCrAnListF } \phi s, \text{ofCrAnOpF } \phi a]_sF * b) = 0$ 

```

lemma $\mathfrak{I}_{\text{normalOrderF_superCommuteF_ofCrAnListF_ofCrAnListF_eq_zero_mul}}$ [\[source\]](#)

```

lemma  $\mathfrak{I}_{\text{normalOrderF\_superCommuteF\_ofCrAnListF\_ofCrAnListF\_eq\_zero\_mul}}$ 
  ( $\phi s \ \phi s' : \text{List } \mathcal{F}.\text{CrAnFieldOp}$ ) ( $a \ b : \mathcal{F}.\text{FieldOpFreeAlgebra}$ ) :
   $\mathfrak{I} \mathcal{N}^f(a * [\text{ofCrAnListF } \phi s, \text{ofCrAnListF } \phi s']_sF * b) = 0$ 

```

lemma $\mathfrak{I}_{\text{normalOrderF_superCommuteF_ofCrAnListF_eq_zero_mul}}$ [\[source\]](#)

```

lemma  $\mathfrak{I}_{\text{normalOrderF\_superCommuteF\_ofCrAnListF\_eq\_zero\_mul}}$ 
  ( $\phi s : \text{List } \mathcal{F}.\text{CrAnFieldOp}$ )
  ( $a \ b \ c : \mathcal{F}.\text{FieldOpFreeAlgebra}$ ) :
   $\mathfrak{I} \mathcal{N}^f(a * [\text{ofCrAnListF } \phi s, c]_sF * b) = 0$ 

```

lemma $\mathfrak{I}_{\text{normalOrderF_superCommuteF_eq_zero_mul}}$ [\[source\]](#)

```

@[simp]
lemma  $\mathfrak{I}_{\text{normalOrderF\_superCommuteF\_eq\_zero\_mul}}$ 
  ( $a \ b \ c \ d : \mathcal{F}.\text{FieldOpFreeAlgebra}$ ) :  $\mathfrak{I} \mathcal{N}^f(a * [d, c]_sF * b) = 0$ 

```

lemma $\mathfrak{I}_{\text{normalOrder_superCommuteF_eq_zero_mul_right}}$ [\[source\]](#)

```

@[simp]
lemma  $\mathfrak{I}_{\text{normalOrder\_superCommuteF\_eq\_zero\_mul\_right}}$  ( $b \ c \ d : \mathcal{F}.\text{FieldOpFreeAlgebra}$ ) :
   $\mathfrak{I} \mathcal{N}^f([d, c]_sF * b) = 0$ 

```

lemma $\mathfrak{I}_{\text{normalOrderF_superCommuteF_eq_zero_mul_left}}$ [\[source\]](#)

```

@[simp]
lemma  $\mathfrak{I}_{\text{normalOrderF\_superCommuteF\_eq\_zero\_mul\_left}}$  ( $a \ c \ d : \mathcal{F}.\text{FieldOpFreeAlgebra}$ ) :
   $\mathfrak{I} \mathcal{N}^f(a * [d, c]_sF) = 0$ 

```

lemma $\mathfrak{I}_{\text{normalOrderF_superCommuteF_eq_zero_mul_mul_right}}$ [\[source\]](#)

@[simp]

lemma $\imath_{\text{normalOrderF_superCommuteF_eq_zero_mul_mul_right}}$ (a b1 b2 c d : $\mathcal{F}.\text{FieldOpFreeAlgebra}$) :
 $\imath \mathcal{N}^f(a * [d, c]_s F * b1 * b2) = 0$

lemma $\imath_{\text{normalOrderF_superCommuteF_eq_zero}}$

[\[source\]](#)

@[simp]

lemma $\imath_{\text{normalOrderF_superCommuteF_eq_zero}}$ (c d : $\mathcal{F}.\text{FieldOpFreeAlgebra}$) : $\imath \mathcal{N}^f([d, c]_s F) = 0$

lemma $\imath_{\text{normalOrderF_zero_of_mem_ideal}}$

[\[source\]](#)

lemma $\imath_{\text{normalOrderF_zero_of_mem_ideal}}$ (a : $\mathcal{F}.\text{FieldOpFreeAlgebra}$)
 (h : a $\in$ $\text{TwoSidedIdeal.span } \mathcal{F}.\text{fieldOpIdealSet}$) : $\imath \mathcal{N}^f(a) = 0$

lemma $\imath_{\text{normalOrderF_eq_of_equiv}}$

[\[source\]](#)

lemma $\imath_{\text{normalOrderF_eq_of_equiv}}$ (a b : $\mathcal{F}.\text{FieldOpFreeAlgebra}$) (h : a $\approx$ b) :
 $\imath \mathcal{N}^f(a) = \imath \mathcal{N}^f(b)$

def normalOrder

[\[source\]](#)

For a field specification $\mathcal{F}$, normalOrder is the linear map

$\text{WickAlgebra } \mathcal{F} \rightarrow_{\mathbb{C}} \text{WickAlgebra } \mathcal{F}$

defined as the descent of $\imath \circ_{\mathbb{C}} \text{normalOrderF} : \text{FieldOpFreeAlgebra } \mathcal{F} \rightarrow_{\mathbb{C}} \text{WickAlgebra } \mathcal{F}$ from $\text{FieldOpFreeAlgebra } \mathcal{F}$ to $\text{WickAlgebra } \mathcal{F}$. This descent exists because $\imath \circ_{\mathbb{C}} \text{normalOrderF}$ is well-defined on equivalence classes.

The notation $\mathcal{N}(a)$ is used for $\text{normalOrder } a$ for a an element of $\text{WickAlgebra } \mathcal{F}$.

noncomputable def $\text{normalOrder} : \text{WickAlgebra } \mathcal{F} \rightarrow_{\mathbb{C}} \text{WickAlgebra } \mathcal{F}$ **where**

2.21 PhysLean.QFT.PerturbationTheory.WickAlgebra.NormalOrder.Lemmas

[PhysLean/QFT/PerturbationTheory/WickAlgebra/NormalOrder/Lemmas.lean](#) on GitHub.

lemma $\text{normalOrder_eq_}\imath_{\text{normalOrderF}}$

[\[source\]](#)

lemma $\text{normalOrder_eq_}\imath_{\text{normalOrderF}}$ (a : $\mathcal{F}.\text{FieldOpFreeAlgebra}$) :
 $\mathcal{N}(\imath a) = \imath \mathcal{N}^f(a)$

lemma $\text{normalOrder_ofCrAnList}$

[\[source\]](#)

lemma $\text{normalOrder_ofCrAnList}$ (ϕ s : List $\mathcal{F}.\text{CrAnFieldOp}$) :
 $\mathcal{N}(\text{ofCrAnList } \phi\text{s}) = \text{normalOrderSign } \phi\text{s} \cdot \text{ofCrAnList } (\text{normalOrderList } \phi\text{s})$

lemma $\text{normalOrder_one_eq_one}$

[\[source\]](#)

@[simp]

lemma $\text{normalOrder_one_eq_one} : \text{normalOrder } (\mathcal{F} := \mathcal{F}) 1 = 1$

lemma normalOrder_ofFieldOpList_nil[\[source\]](#)

@[simp]

lemma normalOrder_ofFieldOpList_nil : normalOrder ($\mathcal{F} := \mathcal{F}$) (ofFieldOpList []) = 1**lemma normalOrder_ofCrAnList_nil**[\[source\]](#)

@[simp]

lemma normalOrder_ofCrAnList_nil : normalOrder ($\mathcal{F} := \mathcal{F}$) (ofCrAnList []) = 1**lemma ofCrAnList_eq_normalOrder**[\[source\]](#)**lemma** ofCrAnList_eq_normalOrder (ϕs : List $\mathcal{F}$.CrAnFieldOp) :
ofCrAnList (normalOrderList ϕs) = normalOrderSign ϕs • $\mathcal{N}$ (ofCrAnList ϕs)**lemma normalOrder_normalOrder_mid**[\[source\]](#)**lemma** normalOrder_normalOrder_mid ($a\ b\ c$: $\mathcal{F}$.WickAlgebra) :
 $\mathcal{N}(a * b * c) = \mathcal{N}(a * \mathcal{N}(b) * c)$ **lemma normalOrder_normalOrder_left**[\[source\]](#)**lemma** normalOrder_normalOrder_left ($a\ b$: $\mathcal{F}$.WickAlgebra) :
 $\mathcal{N}(a * b) = \mathcal{N}(\mathcal{N}(a) * b)$ **lemma normalOrder_normalOrder_right**[\[source\]](#)**lemma** normalOrder_normalOrder_right ($a\ b$: $\mathcal{F}$.WickAlgebra) :
 $\mathcal{N}(a * b) = \mathcal{N}(a * \mathcal{N}(b))$ **lemma normalOrder_normalOrder**[\[source\]](#)**lemma** normalOrder_normalOrder (a : $\mathcal{F}$.WickAlgebra) : $\mathcal{N}(\mathcal{N}(a)) = \mathcal{N}(a)$ **lemma normalOrder_mul_anPart**[\[source\]](#)**lemma** normalOrder_mul_anPart (ϕ : $\mathcal{F}$.FieldOp) (a : $\mathcal{F}$.WickAlgebra) :
 $\mathcal{N}(a * \text{anPart } \phi) = \mathcal{N}(a) * \text{anPart } \phi$ **lemma crPart_mul_normalOrder**[\[source\]](#)**lemma** crPart_mul_normalOrder (ϕ : $\mathcal{F}$.FieldOp) (a : $\mathcal{F}$.WickAlgebra) :
 $\mathcal{N}(\text{crPart } \phi * a) = \text{crPart } \phi * \mathcal{N}(a)$ **lemma normalOrder_superCommute_eq_zero**[\[source\]](#)

For a field specification $\mathcal{F}$, and a and b in $\mathcal{F}$.WickAlgebra the normal ordering of the super commutator of a and b vanishes, i.e. $\mathcal{N}([a, b]_s) = 0$.

@[simp]

lemma normalOrder_superCommute_eq_zero ($a\ b$: $\mathcal{F}$.WickAlgebra) :
 $\mathcal{N}([a, b]_s) = 0$

lemma normalOrder_superCommute_left_eq_zero[\[source\]](#)

```
@[simp]
lemma normalOrder_superCommute_left_eq_zero (a b c:  $\mathcal{F}$ .WickAlgebra) :
   $\mathcal{N}([a, b]_s * c) = 0$ 
```

lemma normalOrder_superCommute_right_eq_zero[\[source\]](#)

```
@[simp]
lemma normalOrder_superCommute_right_eq_zero (a b c:  $\mathcal{F}$ .WickAlgebra) :
   $\mathcal{N}(c * [a, b]_s) = 0$ 
```

lemma normalOrder_superCommute_mid_eq_zero[\[source\]](#)

```
@[simp]
lemma normalOrder_superCommute_mid_eq_zero (a b c d :  $\mathcal{F}$ .WickAlgebra) :
   $\mathcal{N}(a * [c, d]_s * b) = 0$ 
```

lemma normalOrder_ofFieldOp_ofFieldOp_swap[\[source\]](#)

```
lemma normalOrder_ofFieldOp_ofFieldOp_swap ( $\phi \phi'$  :  $\mathcal{F}$ .FieldOp):
   $\mathcal{N}(\text{ofFieldOp } \phi * \text{ofFieldOp } \phi') = \mathcal{S}(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi') \cdot \mathcal{N}(\text{ofFieldOp } \phi' * \text{ofFieldOp } \phi)$ 
```

lemma normalOrder_ofCrAnOp_ofCrAnList[\[source\]](#)

```
lemma normalOrder_ofCrAnOp_ofCrAnList ( $\phi$  :  $\mathcal{F}$ .CrAnFieldOp)
  ( $\phi s$  : List  $\mathcal{F}$ .CrAnFieldOp) :  $\mathcal{N}(\text{ofCrAnOp } \phi * \text{ofCrAnList } \phi s) =$ 
   $\mathcal{S}(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi s) \cdot \mathcal{N}(\text{ofCrAnList } \phi s * \text{ofCrAnOp } \phi)$ 
```

lemma normalOrder_ofCrAnOp_ofFieldOpList_swap[\[source\]](#)

```
lemma normalOrder_ofCrAnOp_ofFieldOpList_swap ( $\phi$  :  $\mathcal{F}$ .CrAnFieldOp) ( $\phi'$  : List  $\mathcal{F}$ .FieldOp) :
   $\mathcal{N}(\text{ofCrAnOp } \phi * \text{ofFieldOpList } \phi') = \mathcal{S}(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi') \cdot$ 
   $\mathcal{N}(\text{ofFieldOpList } \phi' * \text{ofCrAnOp } \phi)$ 
```

lemma normalOrder_anPart_ofFieldOpList_swap[\[source\]](#)

```
lemma normalOrder_anPart_ofFieldOpList_swap ( $\phi$  :  $\mathcal{F}$ .FieldOp) ( $\phi'$  : List  $\mathcal{F}$ .FieldOp) :
   $\mathcal{N}(\text{anPart } \phi * \text{ofFieldOpList } \phi') = \mathcal{S}(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi') \cdot \mathcal{N}(\text{ofFieldOpList } \phi' * \text{anPart } \phi)$ 
```

lemma normalOrder_ofFieldOpList_anPart_swap[\[source\]](#)

```
lemma normalOrder_ofFieldOpList_anPart_swap ( $\phi$  :  $\mathcal{F}$ .FieldOp) ( $\phi'$  : List  $\mathcal{F}$ .FieldOp) :
   $\mathcal{N}(\text{ofFieldOpList } \phi' * \text{anPart } \phi) = \mathcal{S}(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi') \cdot \mathcal{N}(\text{anPart } \phi * \text{ofFieldOpList } \phi')$ 
```

lemma normalOrder_ofFieldOpList_mul_anPart_swap[\[source\]](#)

```
lemma normalOrder_ofFieldOpList_mul_anPart_swap ( $\phi$  :  $\mathcal{F}$ .FieldOp) ( $\phi s$  : List  $\mathcal{F}$ .FieldOp) :
   $\mathcal{N}(\text{ofFieldOpList } \phi s) * \text{anPart } \phi = \mathcal{S}(\mathcal{F} \mid>_s \phi, \mathcal{F} \mid>_s \phi s) \cdot \mathcal{N}(\text{anPart } \phi * \text{ofFieldOpList } \phi s)$ 
```

lemma anPart_mul_normalOrder_ofFieldOpList_eq_superCommute[\[source\]](#)

```

lemma anPart_mul_normalOrder_ofFieldOpList_eq_superCommute (φ :  $\mathcal{F}$ .FieldOp)
  (ψs' : List  $\mathcal{F}$ .FieldOp) : anPart φ *  $\mathcal{N}$ (ofFieldOpList ψs') =
  S( $\mathcal{F}$  |>s φ,  $\mathcal{F}$  |>s ψs') •  $\mathcal{N}$ (ofFieldOpList ψs' * anPart φ) +
  [anPart φ,  $\mathcal{N}$ (ofFieldOpList ψs')]_s

```

lemma ofCrAnOp_superCommute_normalOrder_ofCrAnList_sum[\[source\]](#)

For a field specification $\mathcal{F}$, an element ϕ of $\mathcal{F}$.CrAnFieldOp, a list ϕs of $\mathcal{F}$.CrAnFieldOp, the following relation holds

$$[\phi, \mathcal{N}(\phi_{\circ} \dots \phi_n)]_s = \sum i, S(\phi, \phi_{\circ} \dots \phi_{i-1}) \cdot [\phi, \phi_i]_s * \mathcal{N}(\phi_{\circ} \dots \phi_{i-1} \phi_{i+1} \dots \phi_n).$$

The proof of this result ultimately goes as follows - The definition of `normalOrder` is used to rewrite $\mathcal{N}(\phi_{\circ} \dots \phi_n)$ as a scalar multiple of a `ofCrAnList` ϕsn where ϕsn is the normal ordering of $\phi_{\circ} \dots \phi_n$. - `superCommuteF_ofCrAnListF_ofCrAnListF_eq_sum` is used to rewrite the super commutator of ϕ (considered as a list with one element) with `ofCrAnList` ϕsn as a sum of super commutators, one for each element of ϕsn . - The fact that super-commutators are in the center of $\mathcal{F}$.WickAlgebra is used to rearrange terms. - Properties of ordered lists, and `normalOrderSign_eraseIdx` are then used to complete the proof.

```

lemma ofCrAnOp_superCommute_normalOrder_ofCrAnList_sum (φ :  $\mathcal{F}$ .CrAnFieldOp)
  (ψs : List  $\mathcal{F}$ .CrAnFieldOp) : [ofCrAnOp φ,  $\mathcal{N}$ (ofCrAnList ψs)]_s =  $\sum n : \text{Fin } \psi s.\text{length},$ 
  S( $\mathcal{F}$  |>s φ,  $\mathcal{F}$  |>s (ψs.take n)) • [ofCrAnOp φ, ofCrAnOp ψs[n]]_s
  *  $\mathcal{N}$ (ofCrAnList (ψs.eraseIdx n))

```

lemma ofCrAnOp_superCommute_normalOrder_ofFieldOpList_sum[\[source\]](#)

```

lemma ofCrAnOp_superCommute_normalOrder_ofFieldOpList_sum (φ :  $\mathcal{F}$ .CrAnFieldOp)
  (ψs : List  $\mathcal{F}$ .FieldOp) :
  [ofCrAnOp φ,  $\mathcal{N}$ (ofFieldOpList ψs)]_s =  $\sum n : \text{Fin } \psi s.\text{length}, S(\mathcal{F} |>_s \phi, \mathcal{F} |>_s (\psi s.\text{take } n))$ 
  •
  [ofCrAnOp φ, ofFieldOp ψs[n]]_s *  $\mathcal{N}$ (ofFieldOpList (ψs.eraseIdx n))

```

lemma anPart_superCommute_normalOrder_ofFieldOpList_sum[\[source\]](#)

The commutator of the annihilation part of a field operator with a normal ordered list of field operators can be decomposed into the sum of the commutators of the annihilation part with each element of the list of field operators, i.e. $[anPart \phi, \mathcal{N}(\phi_{\circ} \dots \phi_n)]_s = \sum i, S(\phi, \phi_{\circ} \dots \phi_{i-1}) \cdot [anPart \phi, \phi_i]_s * \mathcal{N}(\phi_{\circ} \dots \phi_{i-1} \phi_{i+1} \dots \phi_n).$

```

lemma anPart_superCommute_normalOrder_ofFieldOpList_sum (φ :  $\mathcal{F}$ .FieldOp) (ψs : List  $\mathcal{F}$ .FieldOp)
  :
  [anPart φ,  $\mathcal{N}$ (ofFieldOpList ψs)]_s =  $\sum n : \text{Fin } \psi s.\text{length}, S(\mathcal{F} |>_s \phi, \mathcal{F} |>_s (\psi s.\text{take } n))$  •
  [anPart φ, ofFieldOpF ψs[n]]_s *  $\mathcal{N}$ (ofFieldOpList (ψs.eraseIdx n))

```

lemma anPart_mul_normalOrder_ofFieldOpList_eq_superCommute_reorder[\[source\]](#)

Within a proto-operator algebra we have that $anPartF \phi * \mathcal{N}(\phi_{\circ} \phi_1 \dots \phi_n) = \mathcal{N}((anPart \phi) \phi_{\circ} \phi_1 \dots \phi_n) + [anpart \phi, \mathcal{N}(\phi_{\circ} \phi_1 \dots \phi_n)]_s.$

```

lemma anPart_mul_normalOrder_ofFieldOpList_eq_superCommute_reorder (φ :  $\mathcal{F}$ .FieldOp)
  (ψs : List  $\mathcal{F}$ .FieldOp) : anPart φ *  $\mathcal{N}$ (ofFieldOpList ψs) =

```

$$\mathcal{N}(\text{anPart } \varphi * \text{ofFieldOpList } \varphi\text{s}) + [\text{anPart } \varphi, \mathcal{N}(\text{ofFieldOpList } \varphi\text{s})]_s$$

lemma ofFieldOp_mul_normalOrder_ofFieldOpList_eq_superCommute

[\[source\]](#)

*Within a proto-operator algebra we have that $\varphi * \mathcal{N}^f(\varphi_\emptyset \varphi_1 \dots \varphi_n) = \mathcal{N}^f(\varphi \varphi_\emptyset \varphi_1 \dots \varphi_n) + [\text{anPart } \varphi, \mathcal{N}^f(\varphi_\emptyset \varphi_1 \dots \varphi_n)]_s F$.*

lemma ofFieldOp_mul_normalOrder_ofFieldOpList_eq_superCommute ($\varphi : \mathcal{F}.\text{FieldOp}$)
 ($\varphi\text{s} : \text{List } \mathcal{F}.\text{FieldOp}$) : ofFieldOp $\varphi * \mathcal{N}(\text{ofFieldOpList } \varphi\text{s}) =$
 $\mathcal{N}(\text{ofFieldOp } \varphi * \text{ofFieldOpList } \varphi\text{s}) + [\text{anPart } \varphi, \mathcal{N}(\text{ofFieldOpList } \varphi\text{s})]_s$

def contractStateAtIndex

[\[source\]](#)

*In the expansion of ofFieldOpF $\varphi * \text{normalOrderF } (\text{ofFieldOpListF } \varphi\text{s})$ the element of $\mathcal{O}.A$ associated with contracting φ with the (optional) n th element of φs .*

noncomputable def contractStateAtIndex ($\varphi : \mathcal{F}.\text{FieldOp}$) ($\varphi\text{s} : \text{List } \mathcal{F}.\text{FieldOp}$)
 ($n : \text{Option } (\text{Fin } \varphi\text{s}.length)$) : $\mathcal{F}.\text{WickAlgebra}$

lemma ofFieldOp_mul_normalOrder_ofFieldOpList_eq_sum

[\[source\]](#)

*For a field specification $\mathcal{F}$, a φ in $\mathcal{F}.\text{FieldOp}$ and a list φs of $\mathcal{F}.\text{FieldOp}$ then $\varphi * \mathcal{N}(\varphi_\emptyset \varphi_1 \dots \varphi_n)$ is equal to*

$$\mathcal{N}(\varphi \varphi_\emptyset \varphi_1 \dots \varphi_n) + \sum i, (S(\varphi, \varphi_\emptyset \varphi_1 \dots \varphi_{i-1}) \cdot [\text{anPart } \varphi, \varphi_i]_s) * \mathcal{N}(\varphi_\emptyset \dots \varphi_{i-1} \varphi_{i+1} \dots \varphi_n).$$

The proof ultimately goes as follows: - ofFieldOp_eq_crPart_add_anPart is used to split φ into its creation and annihilation parts. - The following relation is then used

$$\text{crPart } \varphi * \mathcal{N}(\varphi_\emptyset \varphi_1 \dots \varphi_n) = \mathcal{N}(\text{crPart } \varphi * \varphi_\emptyset \varphi_1 \dots \varphi_n).$$

*- It used that $\text{anPart } \varphi * \mathcal{N}(\varphi_\emptyset \varphi_1 \dots \varphi_n)$ is equal to*

$$S(\varphi, \varphi_\emptyset \varphi_1 \dots \varphi_n) \mathcal{N}(\varphi_\emptyset \varphi_1 \dots \varphi_n) * \text{anPart } \varphi + [\text{anPart } \varphi, \mathcal{N}(\varphi_\emptyset \varphi_1 \dots \varphi_n)]$$

- Then it is used that

$$S(\varphi, \varphi_\emptyset \varphi_1 \dots \varphi_n) \mathcal{N}(\varphi_\emptyset \varphi_1 \dots \varphi_n) * \text{anPart } \varphi = \mathcal{N}(\text{anPart } \varphi * \varphi_\emptyset \varphi_1 \dots \varphi_n)$$

- The result ofCrAnOp_superCommute_normalOrder_ofCrAnList_sum is used to expand $[\text{anPart } \varphi, \mathcal{N}(\varphi_\emptyset \varphi_1 \dots \varphi_n)]$ as a sum.

lemma ofFieldOp_mul_normalOrder_ofFieldOpList_eq_sum ($\varphi : \mathcal{F}.\text{FieldOp}$) ($\varphi\text{s} : \text{List } \mathcal{F}.\text{FieldOp}$) :
 ofFieldOp $\varphi * \mathcal{N}(\text{ofFieldOpList } \varphi\text{s}) =$
 $\sum n : \text{Option } (\text{Fin } \varphi\text{s}.length), \text{contractStateAtIndex } \varphi \varphi\text{s } n * \mathcal{N}(\text{ofFieldOpList } (\text{optionEraseZ } \varphi\text{s } \varphi\text{s } n))$

lemma ofFieldOpList_normalOrder_insert

[\[source\]](#)

Within a proto-operator algebra, $N(\varphi \varphi_\emptyset \varphi_1 \dots \varphi_n) = s \cdot N(\varphi_\emptyset \dots \varphi_{k-1} \varphi \varphi_k \dots \varphi_n)$, where s is the exchange sign for φ and $\varphi_\emptyset \dots \varphi_{k-1}$.

lemma ofFieldOpList_normalOrder_insert ($\varphi : \mathcal{F}.\text{FieldOp}$) ($\varphi\text{s} : \text{List } \mathcal{F}.\text{FieldOp}$)
 ($k : \text{Fin } \varphi\text{s}.length.succ$) : $\mathcal{N}(\text{ofFieldOpList } (\varphi :: \varphi\text{s})) =$
 $S(\mathcal{F} \mid_{>s} \varphi, \mathcal{F} \mid_{>s} \varphi\text{s}.take\ k) \cdot \mathcal{N}(\text{ofFieldOpList } (\varphi\text{s}.insertIdx\ k\ \varphi))$

lemma normalOrder_crPart_mul_crPart[\[source\]](#)

```
@[simp]
lemma normalOrder_crPart_mul_crPart (φ φ' :  $\mathcal{F}$ .Field0p) :
   $\mathcal{N}(\text{crPart } \varphi * \text{crPart } \varphi') = \text{crPart } \varphi * \text{crPart } \varphi'$ 
```

lemma normalOrder_anPart_mul_anPart[\[source\]](#)

```
@[simp]
lemma normalOrder_anPart_mul_anPart (φ φ' :  $\mathcal{F}$ .Field0p) :
   $\mathcal{N}(\text{anPart } \varphi * \text{anPart } \varphi') = \text{anPart } \varphi * \text{anPart } \varphi'$ 
```

lemma normalOrder_crPart_mul_anPart[\[source\]](#)

```
@[simp]
lemma normalOrder_crPart_mul_anPart (φ φ' :  $\mathcal{F}$ .Field0p) :
   $\mathcal{N}(\text{crPart } \varphi * \text{anPart } \varphi') = \text{crPart } \varphi * \text{anPart } \varphi'$ 
```

lemma normalOrder_anPart_mul_crPart[\[source\]](#)

```
@[simp]
lemma normalOrder_anPart_mul_crPart (φ φ' :  $\mathcal{F}$ .Field0p) :
   $\mathcal{N}(\text{anPart } \varphi * \text{crPart } \varphi') = \mathcal{S}(\mathcal{F} \mid_{>s} \varphi, \mathcal{F} \mid_{>s} \varphi') \bullet \text{crPart } \varphi' * \text{anPart } \varphi$ 
```

lemma normalOrder_ofField0p_mul_ofField0p[\[source\]](#)

```
lemma normalOrder_ofField0p_mul_ofField0p (φ φ' :  $\mathcal{F}$ .Field0p) :  $\mathcal{N}(\text{ofField0p } \varphi * \text{ofField0p } \varphi')$ 
=
   $\text{crPart } \varphi * \text{crPart } \varphi' + \mathcal{S}(\mathcal{F} \mid_{>s} \varphi, \mathcal{F} \mid_{>s} \varphi') \bullet (\text{crPart } \varphi' * \text{anPart } \varphi) +$ 
   $\text{crPart } \varphi * \text{anPart } \varphi' + \text{anPart } \varphi * \text{anPart } \varphi'$ 
```

2.22 PhysLean.QFT.PerturbationTheory.WickAlgebra.NormalOrder.WickContractions

[PhysLean/QFT/PerturbationTheory/WickAlgebra/NormalOrder/WickContractions.lean](#) on GitHub.

lemma normalOrder_uncontracted_none[\[source\]](#)

For a list $\varphi s = \varphi_0 \dots \varphi_n$ of $\mathcal{F}$.Field0p, a Wick contraction $\varphi s \Lambda$ of φs , an element φ of $\mathcal{F}$.Field0p, and a $i \leq \varphi s.\text{length}$, then the following relation holds:

$$\mathcal{N}([\varphi s \Lambda \leftarrow \varphi \text{ i none}]^{uc}) = s \bullet \mathcal{N}(\varphi :: [\varphi s \Lambda]^{uc})$$

where s is the exchange sign for φ and the uncontracted fields in $\varphi_0 \dots \varphi_{i-1}$.

The proof of this result ultimately is a consequence of `normalOrder_superCommute_eq_zero`.

```
lemma normalOrder_uncontracted_none (φ :  $\mathcal{F}$ .Field0p) (φs : List  $\mathcal{F}$ .Field0p)
  (i : Fin φs.length.succ) (φsΛ : WickContraction φs.length) :
   $\mathcal{N}(\text{ofField0pList } [\varphi s \Lambda \leftarrow \varphi \text{ i none}]^{uc})$ 
=  $\mathcal{S}(\mathcal{F} \mid_{>s} \varphi, \mathcal{F} \mid_{>s} \{\varphi s.\text{get } i.\text{succAbove } x < i\}) \bullet$ 
   $\mathcal{N}(\text{ofField0pList } (\varphi :: [\varphi s \Lambda]^{uc}))$ 
```

lemma normalOrder_uncontracted_some[\[source\]](#)

For a list $\phi s = \phi_0 \dots \phi_n$ of $\mathcal{F}.FieldOp$, a Wick contraction $\phi s \Lambda$ of ϕs , an element ϕ of $\mathcal{F}.FieldOp$, a $i \leq \phi s.length$ and a k in $\phi s \Lambda.uncontracted$, then $\mathcal{N}([\phi s \Lambda \leftarrow \phi \ i \ (some \ k)]^{uc})$ is equal to the normal ordering of $[\phi s \Lambda]^{uc}$ with the $\mathcal{F}.FieldOp$ corresponding to k removed.

The proof of this result ultimately is a consequence of definitions.

```
lemma normalOrder_uncontracted_some (ϕ : ℱ.FieldOp) (ϕs : List ℱ.FieldOp)
  (i : Fin ϕs.length.succ) (ϕsΛ : WickContraction ϕs.length) (k : ϕsΛ.uncontracted) :
  N(ofFieldOpList [ϕsΛ ← ϕ i (some k)]uc)
  = N(ofFieldOpList (optionEraseZ [ϕsΛ]uc ϕ ((uncontractedFieldOpEquiv ϕs ϕsΛ) k)))
```

2.23 PhysLean.QFT.PerturbationTheory.WickAlgebra.StaticWickTheorem

[PhysLean/QFT/PerturbationTheory/WickAlgebra/StaticWickTheorem.lean](#) on GitHub.

theorem static_wick_theorem[\[source\]](#)

For a list ϕs of $\mathcal{F}.FieldOp$, the static version of Wick's theorem states that

$$\phi s = \sum \phi s \Lambda, \phi s \Lambda.staticWickTerm$$

where the sum is over all Wick contraction $\phi s \Lambda$.

The proof is via induction on ϕs . - The base case $\phi s = []$ is handled by `staticWickTerm_empty_nil`.

The inductive step works as follows:

For the LHS: 1. The proof considers $\phi_0 \dots \phi_n$ as $\phi_0(\phi_1 \dots \phi_n)$ and uses the induction hypothesis on $\phi_1 \dots \phi_n$. 2. This gives terms of the form $\phi * \phi s \Lambda.staticWickTerm$ on which `mul_staticWickTerm_eq_sum` is used where $\phi s \Lambda$ is a Wick contraction of $\phi_1 \dots \phi_n$, to rewrite terms as a sum over optional uncontracted elements of $\phi s \Lambda$

On the LHS we now have a sum over Wick contractions $\phi s \Lambda$ of $\phi_1 \dots \phi_n$ (from 1) and optional uncontracted elements of $\phi s \Lambda$ (from 2)

For the RHS: 1. The sum over Wick contractions of $\phi_0 \dots \phi_n$ on the RHS is split via `insertLift_sum` into a sum over Wick contractions $\phi s \Lambda$ of $\phi_1 \dots \phi_n$ and sum over optional uncontracted elements of $\phi s \Lambda$.

Both sides are now sums over the same thing and their terms equate by the nature of the lemmas used.

```
theorem static_wick_theorem : (ϕs : List ℱ.FieldOp) →
  ofFieldOpList ϕs = ∑ (ϕsΛ : WickContraction ϕs.length), ϕsΛ.staticWickTerm
| [] => by
  simp only [ofFieldOpList, ofFieldOpListF_nil, map_one, List.length_nil]
  rw [sum_WickContraction_nil]
  rw [staticWickTerm_empty_nil]
| ϕ :: ϕs => by
  rw [ofFieldOpList_cons, static_wick_theorem ϕs]
  rw [show (ϕ :: ϕs) = ϕs.insertIdx ((0, Nat.zero_lt_succ ϕs.length) : Fin ϕs.length.succ) ϕ
    from rfl]
  conv_rhs => rw [insertLift_sum]
  rw [Finset.mul_sum]
  apply Finset.sum_congr rfl
  intro c _
```

```
rw [mul_staticWickTerm_eq_sum]
rfl
```

2.24 PhysLean.QFT.PerturbationTheory.WickAlgebra.SuperCommute

[PhysLean/QFT/PerturbationTheory/WickAlgebra/SuperCommute.lean](#) on GitHub.

lemma `ι_superCommuteF_eq_zero_of_ι_right_zero` [\[source\]](#)

lemma `ι_superCommuteF_eq_zero_of_ι_right_zero` (a b : $\mathcal{F}$.FieldOpFreeAlgebra) (h : $\iota b = 0$) :
 $\iota [a, b]_s F = 0$

lemma `ι_superCommuteF_eq_zero_of_ι_left_zero` [\[source\]](#)

lemma `ι_superCommuteF_eq_zero_of_ι_left_zero` (a b : $\mathcal{F}$.FieldOpFreeAlgebra) (h : $\iota a = 0$) :
 $\iota [a, b]_s F = 0$

lemma `ι_superCommuteF_right_zero_of_mem_ideal` [\[source\]](#)

lemma `ι_superCommuteF_right_zero_of_mem_ideal` (a b : $\mathcal{F}$.FieldOpFreeAlgebra)
(h : b ∈ TwoSidedIdeal.span $\mathcal{F}$.fieldOpIdealSet) : $\iota [a, b]_s F = 0$

lemma `ι_superCommuteF_eq_of_equiv_right` [\[source\]](#)

lemma `ι_superCommuteF_eq_of_equiv_right` (a b1 b2 : $\mathcal{F}$.FieldOpFreeAlgebra) (h : b1 ≈ b2) :
 $\iota [a, b1]_s F = \iota [a, b2]_s F$

def `superCommuteRight` [\[source\]](#)

The super commutator on the WickAlgebra defined as a linear map $[a, _]_s$.

noncomputable def `superCommuteRight` (a : $\mathcal{F}$.FieldOpFreeAlgebra) :
WickAlgebra $\mathcal{F} \rightarrow_l [\mathbb{C}]$ WickAlgebra $\mathcal{F}$ **where**

lemma `superCommuteRight_apply_ι` [\[source\]](#)

lemma `superCommuteRight_apply_ι` (a b : $\mathcal{F}$.FieldOpFreeAlgebra) :
`superCommuteRight` a (ιb) = $\iota [a, b]_s F$

lemma `superCommuteRight_apply_quot` [\[source\]](#)

lemma `superCommuteRight_apply_quot` (a b : $\mathcal{F}$.FieldOpFreeAlgebra) :
`superCommuteRight` a $\llbracket b \rrbracket = \iota [a, b]_s F$

lemma `superCommuteRight_eq_of_equiv` [\[source\]](#)

lemma `superCommuteRight_eq_of_equiv` (a1 a2 : $\mathcal{F}$.FieldOpFreeAlgebra) (h : a1 ≈ a2) :
`superCommuteRight` a1 = `superCommuteRight` a2

def superCommute[\[source\]](#)

For a field specification $\mathcal{F}$, *superCommute* is the linear map

$$\text{WickAlgebra } \mathcal{F} \rightarrow_l[\mathbb{C}] \text{ WickAlgebra } \mathcal{F} \rightarrow_l[\mathbb{C}] \text{ WickAlgebra } \mathcal{F}$$

defined as the descent of $\iota \circ \text{superCommuteF}$ in both arguments. In particular for φs and $\varphi s'$ lists of $\mathcal{F}.\text{CrAnFieldOp}$ in $\text{WickAlgebra } \mathcal{F}$ the following relation holds:

$$\text{superCommute } \varphi s \ \varphi s' = \varphi s * \varphi s' - S(\varphi s, \varphi s') \bullet \varphi s' * \varphi s$$

The notation $[a, b]_s$ is used for *superCommute* $a \ b$.

```
noncomputable def superCommute : WickAlgebra  $\mathcal{F}$   $\rightarrow_l[\mathbb{C}]$ 
WickAlgebra  $\mathcal{F} \rightarrow_l[\mathbb{C}]$  WickAlgebra  $\mathcal{F}$  where
```

lemma superCommute_eq_ι_superCommuteF[\[source\]](#)

```
lemma superCommute_eq_ι_superCommuteF (a b :  $\mathcal{F}.\text{FieldOpFreeAlgebra}$ ) :
[ι a, ι b]s = ι [a, b]sF
```

lemma superCommute_create_create[\[source\]](#)

```
lemma superCommute_create_create { $\varphi \ \varphi'$  :  $\mathcal{F}.\text{CrAnFieldOp}$ }
(h :  $\mathcal{F} \mid^{>c} \varphi = .\text{create}$ ) (h' :  $\mathcal{F} \mid^{>c} \varphi' = .\text{create}$ ) :
[ofCrAnOp  $\varphi$ , ofCrAnOp  $\varphi'$ ]s = 0
```

lemma superCommute_annihilate_annihilate[\[source\]](#)

```
lemma superCommute_annihilate_annihilate { $\varphi \ \varphi'$  :  $\mathcal{F}.\text{CrAnFieldOp}$ }
(h :  $\mathcal{F} \mid^{>c} \varphi = .\text{annihilate}$ ) (h' :  $\mathcal{F} \mid^{>c} \varphi' = .\text{annihilate}$ ) :
[ofCrAnOp  $\varphi$ , ofCrAnOp  $\varphi'$ ]s = 0
```

lemma superCommute_diff_statistic[\[source\]](#)

```
lemma superCommute_diff_statistic { $\varphi \ \varphi'$  :  $\mathcal{F}.\text{CrAnFieldOp}$ } (h : ( $\mathcal{F} \mid^{>s} \varphi$ )  $\neq$  ( $\mathcal{F} \mid^{>s} \varphi'$ ) ) :
[ofCrAnOp  $\varphi$ , ofCrAnOp  $\varphi'$ ]s = 0
```

lemma superCommute_ofCrAnOp_ofFieldOp_diff_stat_zero[\[source\]](#)

```
lemma superCommute_ofCrAnOp_ofFieldOp_diff_stat_zero ( $\varphi$  :  $\mathcal{F}.\text{CrAnFieldOp}$ ) ( $\psi$  :  $\mathcal{F}.\text{FieldOp}$ )
(h : ( $\mathcal{F} \mid^{>s} \varphi$ )  $\neq$  ( $\mathcal{F} \mid^{>s} \psi$ )) : [ofCrAnOp  $\varphi$ , ofFieldOp  $\psi$ ]s = 0
```

lemma superCommute_anPart_ofFieldOpF_diff_grade_zero[\[source\]](#)

```
lemma superCommute_anPart_ofFieldOpF_diff_grade_zero ( $\varphi \ \psi$  :  $\mathcal{F}.\text{FieldOp}$ )
(h : ( $\mathcal{F} \mid^{>s} \varphi$ )  $\neq$  ( $\mathcal{F} \mid^{>s} \psi$ )) : [anPart  $\varphi$ , ofFieldOp  $\psi$ ]s = 0
```

lemma superCommute_ofCrAnOp_ofCrAnOp_mem_center[\[source\]](#)

```
lemma superCommute_ofCrAnOp_ofCrAnOp_mem_center ( $\varphi \ \varphi'$  :  $\mathcal{F}.\text{CrAnFieldOp}$ ) :
[ofCrAnOp  $\varphi$ , ofCrAnOp  $\varphi'$ ]s  $\in$  Subalgebra.center  $\mathbb{C}$  (WickAlgebra  $\mathcal{F}$ )
```

lemma superCommute_ofCrAnOp_ofCrAnOp_commute[\[source\]](#)

```
lemma superCommute_ofCrAnOp_ofCrAnOp_commute (φ φ' :  $\mathcal{F}$ .CrAnFieldOp)
  (a : WickAlgebra  $\mathcal{F}$ ) :
  a * [ofCrAnOp φ, ofCrAnOp φ']s = [ofCrAnOp φ, ofCrAnOp φ']s * a
```

lemma superCommute_ofCrAnOp_ofFieldOp_mem_center[\[source\]](#)

```
lemma superCommute_ofCrAnOp_ofFieldOp_mem_center (φ :  $\mathcal{F}$ .CrAnFieldOp) (φ' :  $\mathcal{F}$ .FieldOp) :
  [ofCrAnOp φ, ofFieldOp φ']s ∈ Subalgebra.center  $\mathbb{C}$  (WickAlgebra  $\mathcal{F}$ )
```

lemma superCommute_ofCrAnOp_ofFieldOp_commute[\[source\]](#)

```
lemma superCommute_ofCrAnOp_ofFieldOp_commute (φ :  $\mathcal{F}$ .CrAnFieldOp) (φ' :  $\mathcal{F}$ .FieldOp)
  (a : WickAlgebra  $\mathcal{F}$ ) : a * [ofCrAnOp φ, ofFieldOp φ']s =
  [ofCrAnOp φ, ofFieldOp φ']s * a
```

lemma superCommute_anPart_ofFieldOp_mem_center[\[source\]](#)

```
lemma superCommute_anPart_ofFieldOp_mem_center (φ φ' :  $\mathcal{F}$ .FieldOp) :
  [anPart φ, ofFieldOp φ']s ∈ Subalgebra.center  $\mathbb{C}$  (WickAlgebra  $\mathcal{F}$ )
```

lemma superCommute_ofCrAnList_ofCrAnList[\[source\]](#)

```
lemma superCommute_ofCrAnList_ofCrAnList (φs φs' : List  $\mathcal{F}$ .CrAnFieldOp) :
  [ofCrAnList φs, ofCrAnList φs']s =
  ofCrAnList (φs ++ φs') -  $S(\mathcal{F} \mid_{>s} \phi_s, \mathcal{F} \mid_{>s} \phi_{s'}) \cdot \text{ofCrAnList } (\phi_{s'} ++ \phi_s)$ 
```

lemma superCommute_ofCrAnOp_ofCrAnOp[\[source\]](#)

```
lemma superCommute_ofCrAnOp_ofCrAnOp (φ φ' :  $\mathcal{F}$ .CrAnFieldOp) :
  [ofCrAnOp φ, ofCrAnOp φ']s = ofCrAnOp φ * ofCrAnOp φ' -
   $S(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi') \cdot \text{ofCrAnOp } \phi' * \text{ofCrAnOp } \phi$ 
```

lemma superCommute_ofCrAnList_ofFieldOpList[\[source\]](#)

```
lemma superCommute_ofCrAnList_ofFieldOpList (φcas : List  $\mathcal{F}$ .CrAnFieldOp)
  (φs : List  $\mathcal{F}$ .FieldOp) :
  [ofCrAnList φcas, ofFieldOpList φs]s = ofCrAnList φcas * ofFieldOpList φs -
   $S(\mathcal{F} \mid_{>s} \phi_{cas}, \mathcal{F} \mid_{>s} \phi_s) \cdot \text{ofFieldOpList } \phi_s * \text{ofCrAnList } \phi_{cas}$ 
```

lemma superCommute_ofFieldOpList_ofFieldOpList[\[source\]](#)

```
lemma superCommute_ofFieldOpList_ofFieldOpList (φs φs' : List  $\mathcal{F}$ .FieldOp) :
  [ofFieldOpList φs, ofFieldOpList φs']s = ofFieldOpList φs * ofFieldOpList φs' -
   $S(\mathcal{F} \mid_{>s} \phi_s, \mathcal{F} \mid_{>s} \phi_{s'}) \cdot \text{ofFieldOpList } \phi_{s'} * \text{ofFieldOpList } \phi_s$ 
```

lemma superCommute_ofFieldOp_ofFieldOpList[\[source\]](#)

```
lemma superCommute_ofFieldOp_ofFieldOpList (φ :  $\mathcal{F}$ .FieldOp) (φs : List  $\mathcal{F}$ .FieldOp) :
  [ofFieldOp φ, ofFieldOpList φs]s = ofFieldOp φ * ofFieldOpList φs -
   $S(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi_s) \cdot \text{ofFieldOpList } \phi_s * \text{ofFieldOp } \phi$ 
```

lemma superCommute_ofFieldOpList_ofFieldOp[\[source\]](#)

```

lemma superCommute_ofFieldOpList_ofFieldOp (φs : List ℱ.FieldOp) (φ : ℱ.FieldOp) :
  [ofFieldOpList φs, ofFieldOp φ]s = ofFieldOpList φs * ofFieldOp φ -
  S(ℱ |>s φs, ℱ |>s φ) • ofFieldOp φ * ofFieldOpList φs

```

lemma superCommute_anPart_crPart[\[source\]](#)

```

lemma superCommute_anPart_crPart (φ φ' : ℱ.FieldOp) :
  [anPart φ, crPart φ']s = anPart φ * crPart φ' -
  S(ℱ |>s φ, ℱ |>s φ') • crPart φ' * anPart φ

```

lemma superCommute_crPart_anPart[\[source\]](#)

```

lemma superCommute_crPart_anPart (φ φ' : ℱ.FieldOp) :
  [crPart φ, anPart φ']s = crPart φ * anPart φ' -
  S(ℱ |>s φ, ℱ |>s φ') • anPart φ' * crPart φ

```

lemma superCommute_crPart_crPart[\[source\]](#)

```

@[simp]
lemma superCommute_crPart_crPart (φ φ' : ℱ.FieldOp) : [crPart φ, crPart φ']s = 0

```

lemma superCommute_anPart_anPart[\[source\]](#)

```

@[simp]
lemma superCommute_anPart_anPart (φ φ' : ℱ.FieldOp) : [anPart φ, anPart φ']s = 0

```

lemma superCommute_crPart_ofFieldOpList[\[source\]](#)

```

lemma superCommute_crPart_ofFieldOpList (φ : ℱ.FieldOp) (φs : List ℱ.FieldOp) :
  [crPart φ, ofFieldOpList φs]s = crPart φ * ofFieldOpList φs -
  S(ℱ |>s φ, ℱ |>s φs) • ofFieldOpList φs * crPart φ

```

lemma superCommute_anPart_ofFieldOpList[\[source\]](#)

```

lemma superCommute_anPart_ofFieldOpList (φ : ℱ.FieldOp) (φs : List ℱ.FieldOp) :
  [anPart φ, ofFieldOpList φs]s = anPart φ * ofFieldOpList φs -
  S(ℱ |>s φ, ℱ |>s φs) • ofFieldOpList φs * anPart φ

```

lemma superCommute_crPart_ofFieldOp[\[source\]](#)

```

lemma superCommute_crPart_ofFieldOp (φ φ' : ℱ.FieldOp) :
  [crPart φ, ofFieldOp φ']s = crPart φ * ofFieldOp φ' -
  S(ℱ |>s φ, ℱ |>s φ') • ofFieldOp φ' * crPart φ

```

lemma superCommute_anPart_ofFieldOp[\[source\]](#)

```

lemma superCommute_anPart_ofFieldOp (φ φ' : ℱ.FieldOp) :
  [anPart φ, ofFieldOp φ']s = anPart φ * ofFieldOp φ' -
  S(ℱ |>s φ, ℱ |>s φ') • ofFieldOp φ' * anPart φ

```

lemma ofCrAnList_mul_ofCrAnList_eq_superCommute[\[source\]](#)

```

lemma ofCrAnList_mul_ofCrAnList_eq_superCommute (φs φs' : List  $\mathcal{F}$ .CrAnFieldOp) :
  ofCrAnList φs * ofCrAnList φs' =
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi_s, \mathcal{F} \mid_{>s} \phi_{s'}) \cdot \text{ofCrAnList } \phi_{s'} * \text{ofCrAnList } \phi_s$ 
  + [ofCrAnList φs, ofCrAnList φs']s

```

lemma ofCrAnOp_mul_ofCrAnList_eq_superCommute[\[source\]](#)

```

lemma ofCrAnOp_mul_ofCrAnList_eq_superCommute (φ :  $\mathcal{F}$ .CrAnFieldOp)
  (φs' : List  $\mathcal{F}$ .CrAnFieldOp) : ofCrAnOp φ * ofCrAnList φs' =
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi_{s'}) \cdot \text{ofCrAnList } \phi_{s'} * \text{ofCrAnOp } \phi$ 
  + [ofCrAnOp φ, ofCrAnList φs']s

```

lemma ofFieldOpList_mul_ofFieldOpList_eq_superCommute[\[source\]](#)

```

lemma ofFieldOpList_mul_ofFieldOpList_eq_superCommute (φs φs' : List  $\mathcal{F}$ .FieldOp) :
  ofFieldOpList φs * ofFieldOpList φs' =
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi_s, \mathcal{F} \mid_{>s} \phi_{s'}) \cdot \text{ofFieldOpList } \phi_{s'} * \text{ofFieldOpList } \phi_s$ 
  + [ofFieldOpList φs, ofFieldOpList φs']s

```

lemma ofFieldOp_mul_ofFieldOpList_eq_superCommute[\[source\]](#)

```

lemma ofFieldOp_mul_ofFieldOpList_eq_superCommute (φ :  $\mathcal{F}$ .FieldOp) (φs' : List  $\mathcal{F}$ .FieldOp) :
  ofFieldOp φ * ofFieldOpList φs' =  $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi_{s'}) \cdot \text{ofFieldOpList } \phi_{s'} * \text{ofFieldOp } \phi$ 
  + [ofFieldOp φ, ofFieldOpList φs']s

```

lemma ofFieldOp_mul_ofFieldOp_eq_superCommute[\[source\]](#)

```

lemma ofFieldOp_mul_ofFieldOp_eq_superCommute (φ φ' :  $\mathcal{F}$ .FieldOp) :
  ofFieldOp φ * ofFieldOp φ' =  $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi')$  · ofFieldOp φ' * ofFieldOp φ
  + [ofFieldOp φ, ofFieldOp φ']s

```

lemma ofFieldOpList_mul_ofFieldOp_eq_superCommute[\[source\]](#)

```

lemma ofFieldOpList_mul_ofFieldOp_eq_superCommute (φs : List  $\mathcal{F}$ .FieldOp) (φ :  $\mathcal{F}$ .FieldOp) :
  ofFieldOpList φs * ofFieldOp φ =  $\mathcal{S}(\mathcal{F} \mid_{>s} \phi_s, \mathcal{F} \mid_{>s} \phi)$  · ofFieldOp φ * ofFieldOpList φs
  + [ofFieldOpList φs, ofFieldOp φ]s

```

lemma ofCrAnList_mul_ofFieldOpList_eq_superCommute[\[source\]](#)

```

lemma ofCrAnList_mul_ofFieldOpList_eq_superCommute (φs : List  $\mathcal{F}$ .CrAnFieldOp)
  (φs' : List  $\mathcal{F}$ .FieldOp) : ofCrAnList φs * ofFieldOpList φs' =
   $\mathcal{S}(\mathcal{F} \mid_{>s} \phi_s, \mathcal{F} \mid_{>s} \phi_{s'}) \cdot \text{ofFieldOpList } \phi_{s'} * \text{ofCrAnList } \phi_s$ 
  + [ofCrAnList φs, ofFieldOpList φs']s

```

lemma crPart_mul_anPart_eq_superCommute[\[source\]](#)

```

lemma crPart_mul_anPart_eq_superCommute (φ φ' :  $\mathcal{F}$ .FieldOp) :
  crPart φ * anPart φ' =  $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi')$  · anPart φ' * crPart φ
  + [crPart φ, anPart φ']s

```

lemma anPart_mul_crPart_eq_superCommute[\[source\]](#)

```

lemma anPart_mul_crPart_eq_superCommute (φ φ' :  $\mathcal{F}$ .FieldOp) :
  anPart φ * crPart φ' =  $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi')$  • crPart φ' * anPart φ
  + [anPart φ, crPart φ']s

```

lemma crPart_mul_crPart_swap[\[source\]](#)

```

lemma crPart_mul_crPart_swap (φ φ' :  $\mathcal{F}$ .FieldOp) :
  crPart φ * crPart φ' =  $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi')$  • crPart φ' * crPart φ

```

lemma anPart_mul_anPart_swap[\[source\]](#)

```

lemma anPart_mul_anPart_swap (φ φ' :  $\mathcal{F}$ .FieldOp) :
  anPart φ * anPart φ' =  $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi')$  • anPart φ' * anPart φ

```

lemma superCommute_ofCrAnList_ofCrAnList_symm[\[source\]](#)

```

lemma superCommute_ofCrAnList_ofCrAnList_symm (φs φs' : List  $\mathcal{F}$ .CrAnFieldOp) :
  [ofCrAnList φs, ofCrAnList φs']s =
  (-  $\mathcal{S}(\mathcal{F} \mid_{>s} \phi_s, \mathcal{F} \mid_{>s} \phi_{s'})$ ) • [ofCrAnList φs', ofCrAnList φs]s

```

lemma superCommute_ofCrAnOp_ofCrAnOp_symm[\[source\]](#)

```

lemma superCommute_ofCrAnOp_ofCrAnOp_symm (φ φ' :  $\mathcal{F}$ .CrAnFieldOp) :
  [ofCrAnOp φ, ofCrAnOp φ']s =
  (-  $\mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi')$ ) • [ofCrAnOp φ', ofCrAnOp φ]s

```

lemma superCommute_ofCrAnList_ofCrAnList_eq_sum[\[source\]](#)

```

lemma superCommute_ofCrAnList_ofCrAnList_eq_sum (φs φs' : List  $\mathcal{F}$ .CrAnFieldOp) :
  [ofCrAnList φs, ofCrAnList φs']s =
   $\sum (n : \text{Fin } \phi_{s'}.length), \mathcal{S}(\mathcal{F} \mid_{>s} \phi_s, \mathcal{F} \mid_{>s} \phi_{s'}.take\ n) \cdot$ 
  ofCrAnList (φs'.take n) * [ofCrAnList φs, ofCrAnOp (φs'.get n)]s *
  ofCrAnList (φs'.drop (n + 1))

```

lemma superCommute_ofCrAnOp_ofCrAnList_eq_sum[\[source\]](#)

```

lemma superCommute_ofCrAnOp_ofCrAnList_eq_sum (φ :  $\mathcal{F}$ .CrAnFieldOp)
  (φs' : List  $\mathcal{F}$ .CrAnFieldOp) : [ofCrAnOp φ, ofCrAnList φs']s =
   $\sum (n : \text{Fin } \phi_{s'}.length), \mathcal{S}(\mathcal{F} \mid_{>s} \phi, \mathcal{F} \mid_{>s} \phi_{s'}.take\ n) \cdot$ 
  [ofCrAnOp φ, ofCrAnOp (φs'.get n)]s * ofCrAnList (φs'.eraseIdx n)

```

lemma superCommute_ofCrAnList_ofFieldOpList_eq_sum[\[source\]](#)

```

lemma superCommute_ofCrAnList_ofFieldOpList_eq_sum (φs : List  $\mathcal{F}$ .CrAnFieldOp)
  (φs' : List  $\mathcal{F}$ .FieldOp) : [ofCrAnList φs, ofFieldOpList φs']s =
   $\sum (n : \text{Fin } \phi_{s'}.length), \mathcal{S}(\mathcal{F} \mid_{>s} \phi_s, \mathcal{F} \mid_{>s} \phi_{s'}.take\ n) \cdot$ 
  ofFieldOpList (φs'.take n) * [ofCrAnList φs, ofFieldOp (φs'.get n)]s *
  ofFieldOpList (φs'.drop (n + 1))

```

lemma superCommute_ofCrAnOp_ofFieldOpList_eq_sum[\[source\]](#)

```

lemma superCommute_ofCrAnOp_ofFieldOpList_eq_sum (φ :  $\mathcal{F}$ .CrAnFieldOp) (ψs' : List  $\mathcal{F}$ .FieldOp) :
  [ofCrAnOp φ, ofFieldOpList ψs']s =
  ∑ (n : Fin ψs'.length),  $\mathcal{S}(\mathcal{F} \mid>_s \varphi, \mathcal{F} \mid>_s \psi_{s'}.take\ n) \cdot$ 
  [ofCrAnOp φ, ofFieldOp (ψs'.get n)]s * ofFieldOpList (ψs'.eraseIdx n)

```

2.25 PhysLean.QFT.PerturbationTheory.WickAlgebra.TimeContraction

[PhysLean/QFT/PerturbationTheory/WickAlgebra/TimeContraction.lean](#) on GitHub.

def timeContract[\[source\]](#)

For a field specification $\mathcal{F}$, and φ and ψ elements of $\mathcal{F}$.FieldOp, the element of $\mathcal{F}$.WickAlgebra, `timeContract φ ψ` is defined to be $\mathcal{T}(\varphi\psi) - \mathcal{N}(\varphi\psi)$.

```

def timeContract (φ ψ :  $\mathcal{F}$ .FieldOp) :  $\mathcal{F}$ .WickAlgebra

```

lemma timeContract_eq_smul[\[source\]](#)

```

lemma timeContract_eq_smul (φ ψ :  $\mathcal{F}$ .FieldOp) : timeContract φ ψ =
   $\mathcal{T}(\text{ofFieldOp } \varphi * \text{ofFieldOp } \psi) + (-1 : \mathbb{C}) \cdot \mathcal{N}(\text{ofFieldOp } \varphi * \text{ofFieldOp } \psi)$ 

```

lemma timeContract_of_timeOrderRel[\[source\]](#)

For a field specification $\mathcal{F}$, and φ and ψ elements of $\mathcal{F}$.FieldOp, if φ and ψ are time-ordered then

$\text{timeContract } \varphi \psi = [\text{anPart } \varphi, \text{ofFieldOp } \psi]_s$.

```

lemma timeContract_of_timeOrderRel (φ ψ :  $\mathcal{F}$ .FieldOp) (h : timeOrderRel φ ψ) :
  timeContract φ ψ = [anPart φ, ofFieldOp ψ]s

```

lemma timeContract_of_not_timeOrderRel[\[source\]](#)

```

lemma timeContract_of_not_timeOrderRel (φ ψ :  $\mathcal{F}$ .FieldOp) (h : ¬ timeOrderRel φ ψ) :
  timeContract φ ψ =  $\mathcal{S}(\mathcal{F} \mid>_s \varphi, \mathcal{F} \mid>_s \psi) \cdot \text{timeContract } \psi \varphi$ 

```

lemma timeContract_of_not_timeOrderRel_expand[\[source\]](#)

For a field specification $\mathcal{F}$, and φ and ψ elements of $\mathcal{F}$.FieldOp, if φ and ψ are not time-ordered then

$\text{timeContract } \varphi \psi = \mathcal{S}(\mathcal{F} \mid>_s \varphi, \mathcal{F} \mid>_s \psi) \cdot [\text{anPart } \psi, \text{ofFieldOp } \varphi]_s$.

```

lemma timeContract_of_not_timeOrderRel_expand (φ ψ :  $\mathcal{F}$ .FieldOp) (h : ¬ timeOrderRel φ ψ) :
  timeContract φ ψ =  $\mathcal{S}(\mathcal{F} \mid>_s \varphi, \mathcal{F} \mid>_s \psi) \cdot [\text{anPart } \psi, \text{ofFieldOp } \varphi]_s$ 

```

lemma timeContract_eq_superCommute[\[source\]](#)

```

lemma timeContract_eq_superCommute (φ ψ :  $\mathcal{F}$ .FieldOp) :
  timeContract φ ψ = if timeOrderRel φ ψ then [anPart φ, ofFieldOp ψ]s
  else  $\mathcal{S}(\mathcal{F} \mid>_s \varphi, \mathcal{F} \mid>_s \psi) \cdot [\text{anPart } \psi, \text{ofFieldOp } \varphi]_s$ 

```

lemma timeContract_mem_center[\[source\]](#)

For a field specification $\mathcal{F}$, and ϕ and ψ elements of $\mathcal{F}.FieldOp$, then $timeContract \phi \psi$ is in the center of $\mathcal{F}.WickAlgebra$.

```
lemma timeContract_mem_center ( $\phi \psi : \mathcal{F}.FieldOp$ ) :  
  timeContract  $\phi \psi \in Subalgebra.center \subset \mathcal{F}.WickAlgebra$ 
```

lemma timeContract_zero_of_diff_grade[\[source\]](#)

```
lemma timeContract_zero_of_diff_grade ( $\phi \psi : \mathcal{F}.FieldOp$ ) ( $h : (\mathcal{F} \mid_{>s} \phi) \neq (\mathcal{F} \mid_{>s} \psi)$ ) :  
  timeContract  $\phi \psi = 0$ 
```

lemma normalOrder_timeContract[\[source\]](#)

```
lemma normalOrder_timeContract ( $\phi \psi : \mathcal{F}.FieldOp$ ) :  
   $\mathcal{N}(timeContract \phi \psi) = 0$ 
```

lemma timeOrder_timeContract_eq_time_mid[\[source\]](#)

```
lemma timeOrder_timeContract_eq_time_mid { $\phi \psi : \mathcal{F}.FieldOp$ }  
  ( $h1 : timeOrderRel \phi \psi$ ) ( $h2 : timeOrderRel \psi \phi$ ) ( $a b : \mathcal{F}.WickAlgebra$ ) :  
   $\mathcal{T}(a * timeContract \phi \psi * b) = timeContract \phi \psi * \mathcal{T}(a * b)$ 
```

lemma timeOrder_timeContract_eq_time_left[\[source\]](#)

```
lemma timeOrder_timeContract_eq_time_left { $\phi \psi : \mathcal{F}.FieldOp$ }  
  ( $h1 : timeOrderRel \phi \psi$ ) ( $h2 : timeOrderRel \psi \phi$ ) ( $b : \mathcal{F}.WickAlgebra$ ) :  
   $\mathcal{T}(timeContract \phi \psi * b) = timeContract \phi \psi * \mathcal{T}(b)$ 
```

lemma timeOrder_timeContract_neq_time[\[source\]](#)

```
lemma timeOrder_timeContract_neq_time { $\phi \psi : \mathcal{F}.FieldOp$ }  
  ( $h1 : \neg (timeOrderRel \phi \psi \wedge timeOrderRel \psi \phi)$ ) :  
   $\mathcal{T}(timeContract \phi \psi) = 0$ 
```

lemma timeContract_inAsymp_inAsymp[\[source\]](#)

The time contraction of an incoming asymptotic field with another incoming asymptotic field is zero.

This prevents Feynman diagrams where incoming vertices are connected to incoming vertices.

```
lemma timeContract_inAsymp_inAsymp ( $\phi \psi : ((f : Field \mathcal{F}) \times AsymptoticLabel \mathcal{F} f) \times Momentum$ ) :  
  timeContract ( $.inAsymp \phi$ ) ( $.inAsymp \psi$ ) = 0
```

lemma timeContract_outAsymp_outAsymp[\[source\]](#)

The time contraction of an outgoing asymptotic field with another outgoing asymptotic field is zero.

This prevents Feynman diagrams where outgoing vertices are connected to outgoing vertices.

```

lemma timeContract_outAsymp_outAsymp (φ ψ : ((f : Field  $\mathcal{F}$ ) × AsymptoticLabel  $\mathcal{F}$  f) × Momentum)
:
timeContract (.outAsymp φ) (.outAsymp ψ) = 0

```

2.26 PhysLean.QFT.PerturbationTheory.WickAlgebra.TimeOrder

[PhysLean/QFT/PerturbationTheory/WickAlgebra/TimeOrder.lean](#) on GitHub.

lemma `ι_timeOrderF_superCommuteF_superCommuteF_eq_time_ofCrAnListF` [\[source\]](#)

```

lemma ι_timeOrderF_superCommuteF_superCommuteF_eq_time_ofCrAnListF {φ1 φ2 φ3 :  $\mathcal{F}$ .CrAnFieldOp}
(φs1 φs2 : List  $\mathcal{F}$ .CrAnFieldOp) (h :
  crAnTimeOrderRel φ1 φ2 ∧ crAnTimeOrderRel φ1 φ3 ∧
  crAnTimeOrderRel φ2 φ1 ∧ crAnTimeOrderRel φ2 φ3 ∧
  crAnTimeOrderRel φ3 φ1 ∧ crAnTimeOrderRel φ3 φ2) :
ι  $\mathcal{T}^f$ (ofCrAnListF φs1 * [ofCrAnOpF φ1, [ofCrAnOpF φ2, ofCrAnOpF φ3]sF]sF *
ofCrAnListF φs2) = 0

```

lemma `ι_timeOrderF_superCommuteF_superCommuteF_ofCrAnListF` [\[source\]](#)

```

lemma ι_timeOrderF_superCommuteF_superCommuteF_ofCrAnListF {φ1 φ2 φ3 :  $\mathcal{F}$ .CrAnFieldOp}
(φs1 φs2 : List  $\mathcal{F}$ .CrAnFieldOp) :
ι  $\mathcal{T}^f$ (ofCrAnListF φs1 * [ofCrAnOpF φ1, [ofCrAnOpF φ2, ofCrAnOpF φ3]sF]sF * ofCrAnListF φs2)
= 0

```

lemma `ι_timeOrderF_superCommuteF_superCommuteF` [\[source\]](#)

```

@[simp]
lemma ι_timeOrderF_superCommuteF_superCommuteF {φ1 φ2 φ3 :  $\mathcal{F}$ .CrAnFieldOp}
(a b :  $\mathcal{F}$ .FieldOpFreeAlgebra) :
ι  $\mathcal{T}^f$ (a * [ofCrAnOpF φ1, [ofCrAnOpF φ2, ofCrAnOpF φ3]sF]sF * b) = 0

```

lemma `ι_timeOrderF_superCommuteF_eq_time` [\[source\]](#)

```

lemma ι_timeOrderF_superCommuteF_eq_time {φ ψ :  $\mathcal{F}$ .CrAnFieldOp}
(hφψ : crAnTimeOrderRel φ ψ) (hψφ : crAnTimeOrderRel ψ φ) (a b :  $\mathcal{F}$ .FieldOpFreeAlgebra) :
ι  $\mathcal{T}^f$ (a * [ofCrAnOpF φ, ofCrAnOpF ψ]sF * b) =
ι ([ofCrAnOpF φ, ofCrAnOpF ψ]sF *  $\mathcal{T}^f$ (a * b))

```

lemma `ι_timeOrderF_superCommuteF_neq_time` [\[source\]](#)

```

lemma ι_timeOrderF_superCommuteF_neq_time {φ ψ :  $\mathcal{F}$ .CrAnFieldOp}
(hφψ : ¬ (crAnTimeOrderRel φ ψ ∧ crAnTimeOrderRel ψ φ)) (a b :  $\mathcal{F}$ .FieldOpFreeAlgebra) :
ι  $\mathcal{T}^f$ (a * [ofCrAnOpF φ, ofCrAnOpF ψ]sF * b) = 0

```

lemma `ι_timeOrderF_zero_of_mem_ideal` [\[source\]](#)

```

lemma ι_timeOrderF_zero_of_mem_ideal (a :  $\mathcal{F}$ .FieldOpFreeAlgebra)
(h : a ∈ TwoSidedIdeal.span  $\mathcal{F}$ .fieldOpIdealSet) : ι  $\mathcal{T}^f$ (a) = 0

```

lemma ι_timeOrderF_eq_of_equiv

[\[source\]](#)

lemma ι_timeOrderF_eq_of_equiv (a b : $\mathcal{F}$.FieldOpFreeAlgebra) (h : a ≈ b) :
 ι $\mathcal{T}^f$ (a) = ι $\mathcal{T}^f$ (b)

def timeOrder

[\[source\]](#)

For a field specification $\mathcal{F}$, timeOrder is the linear map

WickAlgebra $\mathcal{F} \rightarrow_l[\mathbb{C}]$ WickAlgebra $\mathcal{F}$

defined as the descent of ι ∘ ι_timeOrderF : FieldOpFreeAlgebra $\mathcal{F} \rightarrow_l[\mathbb{C}]$ WickAlgebra $\mathcal{F}$ from FieldOpFreeAlgebra $\mathcal{F}$ to WickAlgebra $\mathcal{F}$. This descent exists because ι ∘ ι_timeOrderF is well-defined on equivalence classes.

The notation $\mathcal{T}(a)$ is used for timeOrder a.

noncomputable def timeOrder : WickAlgebra $\mathcal{F} \rightarrow_l[\mathbb{C}]$ WickAlgebra $\mathcal{F}$ **where**

lemma timeOrder_eq_ι_timeOrderF

[\[source\]](#)

lemma timeOrder_eq_ι_timeOrderF (a : $\mathcal{F}$.FieldOpFreeAlgebra) :
 $\mathcal{T}(\iota a) = \iota \mathcal{T}^f(a)$

lemma timeOrder_ofFieldOp_ofFieldOp_ordered

[\[source\]](#)

lemma timeOrder_ofFieldOp_ofFieldOp_ordered {φ ψ : $\mathcal{F}$.FieldOp} (h : timeOrderRel φ ψ) :
 $\mathcal{T}(\text{ofFieldOp } \varphi * \text{ofFieldOp } \psi) = \text{ofFieldOp } \varphi * \text{ofFieldOp } \psi$

lemma timeOrder_ofFieldOp_ofFieldOp_not_ordered

[\[source\]](#)

lemma timeOrder_ofFieldOp_ofFieldOp_not_ordered {φ ψ : $\mathcal{F}$.FieldOp} (h : ¬ timeOrderRel φ ψ) :
 $\mathcal{T}(\text{ofFieldOp } \varphi * \text{ofFieldOp } \psi) = S(\mathcal{F} \mid_{>s} \varphi, \mathcal{F} \mid_{>s} \psi) \bullet \text{ofFieldOp } \psi * \text{ofFieldOp } \varphi$

lemma timeOrder_ofFieldOp_ofFieldOp_not_ordered_eq_timeOrder

[\[source\]](#)

lemma timeOrder_ofFieldOp_ofFieldOp_not_ordered_eq_timeOrder {φ ψ : $\mathcal{F}$.FieldOp}
 (h : ¬ timeOrderRel φ ψ) :
 $\mathcal{T}(\text{ofFieldOp } \varphi * \text{ofFieldOp } \psi) = S(\mathcal{F} \mid_{>s} \varphi, \mathcal{F} \mid_{>s} \psi) \bullet \mathcal{T}(\text{ofFieldOp } \psi * \text{ofFieldOp } \varphi)$

lemma timeOrder_ofFieldOpList_nil

[\[source\]](#)

lemma timeOrder_ofFieldOpList_nil : $\mathcal{T}(\text{ofFieldOpList } (\mathcal{F} := \mathcal{F}) []) = 1$

lemma timeOrder_ofFieldOpList_singleton

[\[source\]](#)

@[simp]

lemma timeOrder_ofFieldOpList_singleton (φ : $\mathcal{F}$.FieldOp) :
 $\mathcal{T}(\text{ofFieldOpList } [\varphi]) = \text{ofFieldOpList } [\varphi]$

lemma timeOrder_eq_maxTimeField_mul_finset[\[source\]](#)

For a field specification $\mathcal{F}$, the time order operator acting on a list of $\mathcal{F}.FieldOp$, $\mathcal{T}(\varphi_0 \dots \varphi_n)$, is equal to $\mathcal{S}(\varphi_i, \varphi_0 \dots \varphi_{i-1}) \cdot \varphi_i \cdot \mathcal{T}(\varphi_0 \dots \varphi_{i-1} \varphi_{i+1} \dots \varphi_n)$ where φ_i is the maximal time field operator in $\varphi_0 \dots \varphi_n$.

The proof of this result ultimately relies on basic properties of ordering and signs.

```
lemma timeOrder_eq_maxTimeField_mul_finset (φ : F.FieldOp) (φs : List F.FieldOp) :
  T(ofFieldOpList (φ :: φs)) = S(F |>_s maxTimeField φ φs, F |>_s ((eraseMaxTimeField φ φs).
  get,
    (Finset.univ.filter (fun x =>
      (maxTimeFieldPosFin φ φs).succAbove x < maxTimeFieldPosFin φ φs))) •
    ofFieldOp (maxTimeField φ φs) * T(ofFieldOpList (eraseMaxTimeField φ φs))
```

lemma timeOrder_superCommute_eq_time_mid[\[source\]](#)

```
lemma timeOrder_superCommute_eq_time_mid {φ ψ : F.CrAnFieldOp}
  (hφψ : crAnTimeOrderRel φ ψ) (hψφ : crAnTimeOrderRel ψ φ) (a b : F.WickAlgebra) :
  T(a * [ofCrAnOp φ, ofCrAnOp ψ]_s * b) =
  [ofCrAnOp φ, ofCrAnOp ψ]_s * T(a * b)
```

lemma timeOrder_superCommute_eq_time_left[\[source\]](#)

```
lemma timeOrder_superCommute_eq_time_left {φ ψ : F.CrAnFieldOp}
  (hφψ : crAnTimeOrderRel φ ψ) (hψφ : crAnTimeOrderRel ψ φ) (b : F.WickAlgebra) :
  T([ofCrAnOp φ, ofCrAnOp ψ]_s * b) =
  [ofCrAnOp φ, ofCrAnOp ψ]_s * T(b)
```

lemma timeOrder_superCommute_neq_time[\[source\]](#)

```
lemma timeOrder_superCommute_neq_time {φ ψ : F.CrAnFieldOp}
  (hφψ : ¬ (crAnTimeOrderRel φ ψ ∧ crAnTimeOrderRel ψ φ)) :
  T([ofCrAnOp φ, ofCrAnOp ψ]_s) = 0
```

lemma timeOrder_superCommute_anPart_ofFieldOp_neq_time[\[source\]](#)

```
lemma timeOrder_superCommute_anPart_ofFieldOp_neq_time {φ ψ : F.FieldOp}
  (hφψ : ¬ (timeOrderRel φ ψ ∧ timeOrderRel ψ φ)) :
  T([anPart φ, ofFieldOp ψ]_s) = 0
```

lemma timeOrder_timeOrder_mid[\[source\]](#)

For a field specification $\mathcal{F}$, and a, b, c in $\mathcal{F}.WickAlgebra$, then $\mathcal{T}(a * b * c) = \mathcal{T}(a * \mathcal{T}(b) * c)$.

```
lemma timeOrder_timeOrder_mid (a b c : F.WickAlgebra) :
  T(a * b * c) = T(a * T(b) * c)
```

lemma timeOrder_timeOrder_left[\[source\]](#)

```
lemma timeOrder_timeOrder_left (b c : F.WickAlgebra) :
  T(b * c) = T(T(b) * c)
```

lemma timeOrder_timeOrder_right[\[source\]](#)

```
lemma timeOrder_timeOrder_right (a b :  $\mathcal{F}$ .WickAlgebra) :
   $\mathcal{T}(a * b) = \mathcal{T}(a * \mathcal{T}(b))$ 
```

lemma timeOrder_timeOrder[\[source\]](#)

Time ordering is a projection.

```
lemma timeOrder_timeOrder (a :  $\mathcal{F}$ .WickAlgebra) :
   $\mathcal{T}(\mathcal{T}(a)) = \mathcal{T}(a)$ 
```

2.27 PhysLean.QFT.PerturbationTheory.WickAlgebra.Universality

[PhysLean/QFT/PerturbationTheory/WickAlgebra/Universality.lean](#) on GitHub.

def universalLiftMap[\[source\]](#)

For a field specification, $\mathcal{F}$, given an algebra A and a function $f : \mathcal{F}.CrAnFieldOp \rightarrow A$ such that the lift of f to $FreeAlgebra.lift \mathbb{C} f : FreeAlgebra \mathbb{C} \mathcal{F}.CrAnFieldOp \rightarrow A$ is zero on the ideal defining $\mathcal{F}.WickAlgebra$, the corresponding map $\mathcal{F}.WickAlgebra \rightarrow A$.

```
def universalLiftMap {A : Type} [Semiring A] [Algebra  $\mathbb{C}$  A] (f :  $\mathcal{F}.CrAnFieldOp \rightarrow A$ )
  (h1 :  $\forall a \in TwoSidedIdeal.span \mathcal{F}.fieldOpIdealSet, FreeAlgebra.lift \mathbb{C} f a = 0$ ) :
  WickAlgebra  $\mathcal{F} \rightarrow A$ 
```

lemma universalLiftMap_1[\[source\]](#)

```
@[simp]
lemma universalLiftMap_1 {A : Type} [Semiring A] [Algebra  $\mathbb{C}$  A] (f :  $\mathcal{F}.CrAnFieldOp \rightarrow A$ )
  (h1 :  $\forall a \in TwoSidedIdeal.span \mathcal{F}.fieldOpIdealSet, FreeAlgebra.lift \mathbb{C} f a = 0$ ) :
  universalLiftMap f h1 (1 a) = FreeAlgebra.lift  $\mathbb{C}$  f a
```

def universalLift[\[source\]](#)

For a field specification, $\mathcal{F}$, given an algebra A and a function $f : \mathcal{F}.CrAnFieldOp \rightarrow A$ such that the lift of f to $FreeAlgebra.lift \mathbb{C} f : FreeAlgebra \mathbb{C} \mathcal{F}.CrAnFieldOp \rightarrow A$ is zero on the ideal defining $\mathcal{F}.WickAlgebra$, the corresponding algebra map $\mathcal{F}.WickAlgebra \rightarrow A$.

```
def universalLift {A : Type} [Semiring A] [Algebra  $\mathbb{C}$  A] (f :  $\mathcal{F}.CrAnFieldOp \rightarrow A$ )
  (h1 :  $\forall a \in TwoSidedIdeal.span \mathcal{F}.fieldOpIdealSet, FreeAlgebra.lift \mathbb{C} f a = 0$ ) :
  WickAlgebra  $\mathcal{F} \rightarrow_a[\mathbb{C}] A$  where
```

lemma universalLift_1[\[source\]](#)

```
@[simp]
lemma universalLift_1 {A : Type} [Semiring A] [Algebra  $\mathbb{C}$  A] (f :  $\mathcal{F}.CrAnFieldOp \rightarrow A$ )
  (h1 :  $\forall a \in TwoSidedIdeal.span \mathcal{F}.fieldOpIdealSet, FreeAlgebra.lift \mathbb{C} f a = 0$ ) :
  universalLift f h1 (1 a) = FreeAlgebra.lift  $\mathbb{C}$  f a
```

lemma universality[\[source\]](#)

For a field specification, $\mathcal{F}$, the algebra $\mathcal{F}.\text{WickAlgebra}$ satisfies the following universal property. Let $f : \mathcal{F}.\text{CrAnFieldOp} \rightarrow A$ be a function and $g : \mathcal{F}.\text{FieldOpFreeAlgebra} \rightarrow_a[\mathbb{C}] A$ the universal lift of that function associated with the free algebra $\mathcal{F}.\text{FieldOpFreeAlgebra}$. If g is zero on the ideal defining $\mathcal{F}.\text{WickAlgebra}$, then there exists algebra map $g' : \text{WickAlgebra } \mathcal{F} \rightarrow_a[\mathbb{C}] A$ such that $g' \circ \iota = g$, and furthermore this algebra map is unique.

```
lemma universality {A : Type} [Semiring A] [Algebra ℂ A] (f : ℱ.CrAnFieldOp → A)
(h1 : ∀ a ∈ TwoSidedIdeal.span ℱ.fieldOpIdealSet, FreeAlgebra.lift ℂ f a = 0) :
∃! g : WickAlgebra ℱ →_a[ℂ] A, g ∘ ι = FreeAlgebra.lift ℂ f
```

2.28 PhysLean.QFT.PerturbationTheory.WickAlgebra.WicksTheoremNormal

[PhysLean/QFT/PerturbationTheory/WickAlgebra/WicksTheoremNormal.lean](#) on GitHub.

lemma timeOrder_ofFieldOpList_eqTimeOnly[\[source\]](#)

For a list ϕs of $\mathcal{F}.\text{FieldOp}$, then

$$\mathcal{T}(\phi s) = \sum \phi s \Lambda, \phi s \Lambda.\text{sign} \cdot \phi s \Lambda.\text{timeContract} * \mathcal{T}(\mathcal{N}([\phi s \Lambda]^{uc}))$$

where the sum is over all Wick contraction $\phi s \Lambda$ which only have equal time contractions.

This result follows from - `static_wick_theorem` to rewrite $\mathcal{T}(\phi s)$ on the left hand side as a sum of $\mathcal{T}(\phi s \Lambda.\text{staticWickTerm})$. - `EqTimeOnly.timeOrder_staticContract_of_not_mem` and `timeOrder_timeOrder_mid` to set to those $\mathcal{T}(\phi s \Lambda.\text{staticWickTerm})$ for which $\phi s \Lambda$ has a contracted pair which are not equal time to zero. - `staticContract_eq_timeContract_of_eqTimeOnly` to rewrite the static contract in the remaining $\mathcal{T}(\phi s \Lambda.\text{staticWickTerm})$ as a time contract. - `timeOrder_timeContract_mul_of_eqTimeOnly_left` to move the time contracts out of the time ordering.

```
lemma timeOrder_ofFieldOpList_eqTimeOnly (ϕs : List ℱ.FieldOp) :
  ℱ(ofFieldOpList ϕs) = ∑ (ϕsΛ : {ϕsΛ // ϕsΛ.EqTimeOnly (ϕs := ϕs)}),
  ϕsΛ.1.sign • ϕsΛ.1.timeContract.1 * ℱ(ℱ(ofFieldOpList [ϕsΛ.1]uc))
```

lemma timeOrder_ofFieldOpList_eq_eqTimeOnly_empty[\[source\]](#)

```
lemma timeOrder_ofFieldOpList_eq_eqTimeOnly_empty (ϕs : List ℱ.FieldOp) :
  ℱ(ofFieldOpList ϕs) = ℱ(ℱ(ofFieldOpList ϕs)) +
  ∑ (ϕsΛ : {ϕsΛ // ϕsΛ.EqTimeOnly (ϕs := ϕs) ∧ ϕsΛ ≠ empty}),
  ϕsΛ.1.sign • ϕsΛ.1.timeContract.1 * ℱ(ℱ(ofFieldOpList [ϕsΛ.1]uc))
```

lemma normalOrder_timeOrder_ofFieldOpList_eq_eqTimeOnly_empty[\[source\]](#)

For a list ϕs of $\mathcal{F}.\text{FieldOp}$, then

$$\mathcal{T}(\mathcal{N}(\phi s)) = \mathcal{T}(\phi s) - \sum \phi s \Lambda, \phi s \Lambda.\text{sign} \cdot \phi s \Lambda.\text{timeContract}.1 * \mathcal{T}(\mathcal{N}([\phi s \Lambda]^{uc}))$$

where the sum is over all *non-empty* Wick contraction $\phi s \Lambda$ which only have equal time contractions.

This result follows directly from - `timeOrder_ofFieldOpList_eqTimeOnly`

```

lemma normalOrder_timeOrder_ofFieldOpList_eq_eqTimeOnly_empty (φs : List  $\mathcal{F}$ .FieldOp) :
   $\mathcal{T}(\mathcal{N}(\text{ofFieldOpList } \phi s)) = \mathcal{T}(\text{ofFieldOpList } \phi s) -$ 
   $\sum (\phi s \Lambda : \{\phi s \Lambda // \phi s \Lambda.\text{EqTimeOnly } (\phi s := \phi s) \wedge \phi s \Lambda \neq \text{empty}\}),$ 
   $\phi s \Lambda.1.\text{sign} \cdot \phi s \Lambda.1.\text{timeContract}.1 * \mathcal{T}(\mathcal{N}(\text{ofFieldOpList } [\phi s \Lambda.1]^{uc}))$ 

```

lemma timeOrder_haveEqTime_split

[\[source\]](#)

For a list ϕs of $\mathcal{F}$.FieldOp, then $\mathcal{T}(\phi s)$ is equal to the sum of

- $\sum \phi s \Lambda$, $\phi s \Lambda.\text{wickTerm}$ where the sum is over all Wick contraction $\phi s \Lambda$ which have no contractions of equal time. - $\sum \phi s \Lambda$, $\phi s \Lambda.\text{sign} \cdot \phi s \Lambda.\text{timeContract} * (\sum \phi s \text{succ} \Lambda$, $\phi s \text{succ} \Lambda.\text{wickTerm})$, where the first sum is over all Wick contraction $\phi s \Lambda$ which only have equal time contractions and the second sum is over all Wick contraction $\phi s \text{succ} \Lambda$ of the uncontracted elements of $\phi s \Lambda$ which do not have any equal time contractions.

The proof proceeds as follows - wicks_theorem is used to rewrite $\mathcal{T}(\phi s)$ as a sum over all Wick contractions. - The sum over all Wick contractions is then split additively into two parts based on having or not having an equal time contractions. - Using join, the sum $\sum \phi s \Lambda$, $_$ over Wick contractions which do have equal time contractions is split into two sums $\sum \phi s \Lambda$, $\sum \phi s \text{succ} \Lambda$, $_$, the first over non-zero elements which only have equal time contractions and the second over Wick contractions $\phi s \text{succ} \Lambda$ of $[\phi s \Lambda]^{uc}$ which do not have equal time contractions. - join_sign_timeContract is then used to equate terms.

```

lemma timeOrder_haveEqTime_split (φs : List  $\mathcal{F}$ .FieldOp) :
   $\mathcal{T}(\text{ofFieldOpList } \phi s) = (\sum (\phi s \Lambda : \{\phi s \Lambda : \text{WickContraction } \phi s.\text{length} // \neg \text{HaveEqTime } \phi s \Lambda\}),$ 
   $\phi s \Lambda.1.\text{sign} \cdot \phi s \Lambda.1.\text{timeContract}.1 * \mathcal{N}(\text{ofFieldOpList } [\phi s \Lambda.1]^{uc}))$ 
  +  $\sum (\phi s \Lambda : \{\phi s \Lambda // \phi s \Lambda.\text{EqTimeOnly } (\phi s := \phi s) \wedge \phi s \Lambda \neq \text{empty}\}), \phi s \Lambda.1.\text{sign} \cdot \phi s \Lambda.1.$ 
   $\text{timeContract} *$ 
   $(\sum \phi s \text{succ} \Lambda : \{\phi s \text{succ} \Lambda : \text{WickContraction } [\phi s \Lambda.1]^{uc}.\text{length} // \neg \phi s \text{succ} \Lambda.\text{HaveEqTime } \},$ 
   $\phi s \text{succ} \Lambda.1.\text{wickTerm})$ 

```

lemma normalOrder_timeOrder_ofFieldOpList_eq_not_haveEqTime_sub_inductive

[\[source\]](#)

```

lemma normalOrder_timeOrder_ofFieldOpList_eq_not_haveEqTime_sub_inductive (φs : List  $\mathcal{F}$ .FieldOp) :
   $\mathcal{T}(\mathcal{N}(\text{ofFieldOpList } \phi s)) =$ 
   $(\sum (\phi s \Lambda : \{\phi s \Lambda : \text{WickContraction } \phi s.\text{length} // \neg \text{HaveEqTime } \phi s \Lambda\}), \phi s \Lambda.1.\text{wickTerm})$ 
  +  $\sum (\phi s \Lambda : \{\phi s \Lambda // \phi s \Lambda.\text{EqTimeOnly } (\phi s := \phi s) \wedge \phi s \Lambda \neq \text{empty}\}),$ 
   $\text{sign } \phi s \uparrow \phi s \Lambda \cdot (\phi s \Lambda.1).\text{timeContract} *$ 
   $(\sum \phi s \text{succ} \Lambda : \{\phi s \text{succ} \Lambda : \text{WickContraction } [\phi s \Lambda.1]^{uc}.\text{length} // \neg \phi s \text{succ} \Lambda.\text{HaveEqTime } \},$ 
   $\phi s \text{succ} \Lambda.1.\text{wickTerm} - \mathcal{T}(\mathcal{N}(\text{ofFieldOpList } [\phi s \Lambda.1]^{uc})))$ 

```

lemma wicks_theorem_normal_order_empty

[\[source\]](#)

```

lemma wicks_theorem_normal_order_empty :  $\mathcal{T}(\mathcal{N}(\text{ofFieldOpList } [])) =$ 
   $\sum (\phi s \Lambda : \{\phi s \Lambda : \text{WickContraction } ([] : \text{List } \mathcal{F}.\text{FieldOp}).\text{length} // \neg \text{HaveEqTime } \phi s \Lambda\}),$ 
   $\phi s \Lambda.1.\text{wickTerm}$ 

```

theorem wicks_theorem_normal_order

[\[source\]](#)

For a list ϕs of $\mathcal{F}$.FieldOp, the normal-ordered version of Wick's theorem states that

$$\mathcal{T}(\mathcal{N}(\phi s)) = \sum \phi s \Lambda, \phi s \Lambda.\text{wickTerm}$$

where the sum is over all Wick contraction $\varphi s \Lambda$ in which no two contracted elements have the same time.

The proof proceeds by induction on φs , with the base case $[]$ holding by following through definitions. and the inductive case holding as a result of - `timeOrder_haveEqTime_split` - `normalOrder_timeOrder_ofFieldOpList_eq_eqTimeOnly_empty` - and the induction hypothesis on $\mathcal{T}(\mathcal{N}([\varphi s \Lambda.1]^{uc}))$ for contractions $\varphi s \Lambda$ of φs which only have equal time contractions and are non-empty.

```

theorem wicks_theorem_normal_order : ( $\varphi s$  : List  $\mathcal{F}$ .FieldOp) →
   $\mathcal{T}(\mathcal{N}(\text{ofFieldOpList } \varphi s)) =$ 
   $\sum (\varphi s \Lambda : \{\varphi s \Lambda : \text{WickContraction } \varphi s.\text{length} \text{ // } \neg \text{HaveEqTime } \varphi s \Lambda\}), \varphi s \Lambda.1.\text{wickTerm}$ 
| [] => wicks_theorem_normal_order_empty
|  $\varphi :: \varphi s$  => by
  rw [normalOrder_timeOrder_ofFieldOpList_eq_not_haveEqTime_sub_inductive]
  simp only [Algebra.smul_mul_assoc, ne_eq, add_eq_left]
  apply Finset.sum_eq_zero
  intro  $\varphi s \Lambda$  h $\varphi s \Lambda$ 
  simp only [smul_eq_zero]
  right
  have ih

```

2.29 PhysLean.QFT.PerturbationTheory.WickContraction.Perm

[PhysLean/QFT/PerturbationTheory/WickContraction/Perm.lean](#) on GitHub.

def Perm

[\[source\]](#)

For a list φs of $\mathcal{F}$.FieldOp, and two Wick contractions $\varphi s \Lambda_1$ and $\varphi s \Lambda_2$ of φs , we say that $\varphi s \Lambda_1$ and $\varphi s \Lambda_2$ are permutations of each other if they have the same Wick term.

```

def Perm { $\varphi s$  : List  $\mathcal{F}$ .FieldOp} ( $\varphi s \Lambda_1$   $\varphi s \Lambda_2$  : WickContraction  $\varphi s.\text{length}$ ) : Prop

```

lemma refl

[\[source\]](#)

The reflexivity of the Perm relation.

```

@[refl]
lemma refl : Perm  $\varphi s \Lambda$   $\varphi s \Lambda$ 

```

lemma symm

[\[source\]](#)

The symmetry of the Perm relation.

```

@[symm]
lemma symm (h : Perm  $\varphi s \Lambda_1$   $\varphi s \Lambda_2$ ) : Perm  $\varphi s \Lambda_2$   $\varphi s \Lambda_1$ 

```

lemma trans

[\[source\]](#)

The transitivity of the Perm relation.

```

@[trans]
lemma trans (h12 : Perm  $\varphi s \Lambda_1$   $\varphi s \Lambda_2$ ) (h23 : Perm  $\varphi s \Lambda_2$   $\varphi s \Lambda_3$ ) :
  Perm  $\varphi s \Lambda_1$   $\varphi s \Lambda_3$ 

```

semiformal_result isFull_of_isFull[\[source\]](#)

If $\text{Perm } \varphi\Lambda_1 \ \varphi\Lambda_2$ then if $\varphi\Lambda_1$ is a full Wick contraction then $\varphi\Lambda_2$ is a full Wick contraction..

Implementation note: Please contact JTS before attempting this.

```
semiformal_result "ABLD" isFull_of_isFull (h : Perm  $\varphi\Lambda_1 \ \varphi\Lambda_2$ ) (hf : IsFull  $\varphi\Lambda_1$ ) : IsFull  $\varphi\Lambda_2$ 
```

semiformal_result perm_uncontractedList[\[source\]](#)

If $\text{Perm } \varphi\Lambda_1 \ \varphi\Lambda_2$ then the uncontracted lists of $\varphi\Lambda_1$ and $\varphi\Lambda_2$ are permutations of each other.

Implementation note: Please contact JTS before attempting this.

```
semiformal_result "ABJD5" perm_uncontractedList (h : Perm  $\varphi\Lambda_1 \ \varphi\Lambda_2$ ) : [ $\varphi\Lambda_1$ ]uc.Perm [ $\varphi\Lambda_2$ ]uc
```

2.30 PhysLean.Relativity.Lorentz.Group.Basic

[PhysLean/Relativity/Lorentz/Group/Basic.lean](#) on GitHub.

lemma inv_eq_dual[\[source\]](#)

```
lemma inv_eq_dual ( $\Lambda$  : LorentzGroup d) :  
  ( $\Lambda^{-1}$  : LorentzGroup d) = minkowskiMatrix.dual  $\Lambda$ .1
```

2.31 PhysLean.Relativity.Lorentz.Group.Orthochronous

[PhysLean/Relativity/Lorentz/Group/Orthochronous.lean](#) on GitHub.

lemma iff_in_orthchroRep_ker[\[source\]](#)

The orthochronous Lorentz transformations form the kernel of the homomorphism from LorentzGroup to $\mathbb{Z}_2$.

```
lemma IsOrthochronous.iff_in_orthchroRep_ker : IsOrthochronous  $\Lambda \leftrightarrow \Lambda \in \text{orthchroRep.ker}$ 
```

lemma orthchroRep_inv_eq_self[\[source\]](#)

The homomorphism from LorentzGroup to $\mathbb{Z}_2$ assigns the same value to any Lorentz transformation and its inverse.

```
lemma orthchroRep_inv_eq_self ( $\Lambda$  : LorentzGroup d) : orthchroRep  $\Lambda$  = orthchroRep  $\Lambda^{-1}$ 
```

lemma iff_inv_isOrthochronous[\[source\]](#)

A Lorentz transformation is orthochronous iff its inverse is orthochronous.

```
lemma IsOrthochronous.iff_inv_isOrthochronous { $\Lambda$  : LorentzGroup d} :  
  IsOrthochronous  $\Lambda \leftrightarrow \text{IsOrthochronous } \Lambda^{-1}$ 
```

2.32 PhysLean.Relativity.Lorentz.Group.Proper

[PhysLean/Relativity/Lorentz/Group/Proper.lean](#) on GitHub.

lemma mul_isProper_of_isProper_isProper[\[source\]](#)

The product of two proper Lorentz transformations is proper.

```
lemma mul_isProper_of_isProper_isProper {Λ Λ' : LorentzGroup d}
  (h : IsProper Λ) (h' : IsProper Λ') : IsProper (Λ * Λ')
```

2.33 PhysLean.Relativity.Lorentz.Group.Restricted

[PhysLean/Relativity/Lorentz/Group/Restricted.lean](#) on GitHub.

def restricted[\[source\]](#)

The restricted Lorentz group comprises the proper and orthochronous elements of the Lorentz group.

```
def restricted (d : ℕ) : Subgroup (LorentzGroup d) where
```

lemma restricted_normal_subgroup[\[source\]](#)

The restricted Lorentz group is a normal subgroup of the Lorentz group.

```
lemma restricted_normal_subgroup {d : ℕ} : (restricted d).Normal
```

2.34 PhysLean.Relativity.PauliMatrices.Basic

[PhysLean/Relativity/PauliMatrices/Basic.lean](#) on GitHub.

lemma pauliContr_eq_fromConstTriple[\[source\]](#)

```
lemma pauliContr_eq_fromConstTriple : σ^^ = fromConstTriple PauliMatrix.asConstTensor
```

lemma pauliContr_eq_fromTripleT[\[source\]](#)

```
lemma pauliContr_eq_fromTripleT : σ^^ = fromTripleT PauliMatrix.asTensor
```

lemma pauliContr_eq_basis[\[source\]](#)

```
lemma pauliContr_eq_basis : pauliContr =
  Tensor.basis ![Color.up, Color.upL, Color.upR] (fun | 0 => 0 | 1 => 0 | 2 => 0)
+ Tensor.basis ![Color.up, Color.upL, Color.upR] (fun | 0 => 0 | 1 => 1 | 2 => 1)
+ Tensor.basis ![Color.up, Color.upL, Color.upR] (fun | 0 => 1 | 1 => 0 | 2 => 1)
+ Tensor.basis ![Color.up, Color.upL, Color.upR] (fun | 0 => 1 | 1 => 1 | 2 => 0)
- I • Tensor.basis ![Color.up, Color.upL, Color.upR] (fun | 0 => 2 | 1 => 0 | 2 => 1)
+ I • Tensor.basis ![Color.up, Color.upL, Color.upR] (fun | 0 => 2 | 1 => 1 | 2 => 0)
+ Tensor.basis ![Color.up, Color.upL, Color.upR] (fun | 0 => 3 | 1 => 0 | 2 => 0)
- Tensor.basis ![Color.up, Color.upL, Color.upR] (fun | 0 => 3 | 1 => 1 | 2 => 1)
```

lemma pauliContr_eq_ofRat[\[source\]](#)

```
lemma pauliContr_eq_ofRat : pauliContr = ofRat (fun b =>
  if b 0 = 0 ∧ b 1 = b 2 then ⟨1, 0⟩ else
  if b 0 = 1 ∧ b 1 ≠ b 2 then ⟨1, 0⟩ else
  if b 0 = 2 ∧ b 1 = 0 ∧ b 2 = 1 then ⟨0, -1⟩ else
```

```

if b 0 = 2 ∧ b 1 = 1 ∧ b 2 = 0 then (0, 1) else
if b 0 = 3 ∧ b 1 = 0 ∧ b 2 = 0 then (1, 0) else
if b 0 = 3 ∧ b 1 = 3 ∧ b 2 = 3 then (-1, 0) else 0)

```

lemma pauliCo_eq_ofRat[\[source\]](#)

```

lemma pauliCo_eq_ofRat : pauliCo = ofRat (fun b =>
  if b 0 = 0 ∧ b 1 = b 2 then (1, 0) else
  if b 0 = 1 ∧ b 1 ≠ b 2 then (-1, 0) else
  if b 0 = 2 ∧ b 1 = 0 ∧ b 2 = 1 then (0, 1) else
  if b 0 = 2 ∧ b 1 = 1 ∧ b 2 = 0 then (0, -1) else
  if b 0 = 3 ∧ b 1 = 0 ∧ b 2 = 0 then (-1, 0) else
  if b 0 = 3 ∧ b 1 = 1 ∧ b 2 = 1 then (1, 0) else (0, 0))

```

lemma pauliCoDown_eq_ofRat[\[source\]](#)

```

lemma pauliCoDown_eq_ofRat : pauliCoDown = ofRat (fun b =>
  if b 0 = 0 ∧ b 1 = b 2 then (1, 0) else
  if b 0 = 1 ∧ b 1 ≠ b 2 then (1, 0) else
  if b 0 = 2 ∧ b 1 = 0 ∧ b 2 = 1 then (0, -1) else
  if b 0 = 2 ∧ b 1 = 1 ∧ b 2 = 0 then (0, 1) else
  if b 0 = 3 ∧ b 1 = 1 ∧ b 2 = 1 then (-1, 0) else
  if b 0 = 3 ∧ b 1 = 0 ∧ b 2 = 0 then (1, 0) else (0, 0))

```

lemma actionT_pauliContr[\[source\]](#)

The tensor pauliContr is invariant under the action of $SL(2, \mathbb{C})$.

```
lemma actionT_pauliContr (g : SL(2, ℂ)) : g • pauliContr = pauliContr
```

lemma actionT_pauliCo[\[source\]](#)

The tensor pauliCo is invariant under the action of $SL(2, \mathbb{C})$.

```
lemma actionT_pauliCo (g : SL(2, ℂ)) : g • pauliCo = pauliCo
```

lemma actionT_pauliCoDown[\[source\]](#)

The tensor pauliCoDown is invariant under the action of $SL(2, \mathbb{C})$.

```
lemma actionT_pauliCoDown (g : SL(2, ℂ)) : g • pauliCoDown = pauliCoDown
```

lemma actionT_pauliContrDown[\[source\]](#)

The tensor pauliContrDown is invariant under the action of $SL(2, \mathbb{C})$.

```
lemma actionT_pauliContrDown (g : SL(2, ℂ)) : g • pauliContrDown = pauliContrDown
```

2.35 PhysLean.Relativity.SpaceTime.Basic

[PhysLean/Relativity/SpaceTime/Basic.lean](#) on GitHub.

lemma space_toCoord_symm[\[source\]](#)

```
@[simp]
lemma space_toCoord_symm {d : ℕ} (f : Fin 1 ⊕ Fin d → ℝ) :
  space (Lorentz.Vector.toCoord.symm f) = fun i => f (Sum.inr i)
```

informal_lemma space_equivariant[\[source\]](#)

The function space is equivariant with respect to rotations.

```
informal_lemma space_equivariant where
  deps := [``space]
  tag := "7MTYX"
```

def time[\[source\]](#)

The time part of spacetime.

```
def time {d : ℕ} : SpaceTime d →l [ℝ] Time where
```

lemma time_toCoord_symm[\[source\]](#)

```
@[simp]
lemma time_toCoord_symm {d : ℕ} (f : Fin 1 ⊕ Fin d → ℝ) :
  time (Lorentz.Vector.toCoord.symm f) = f (Sum.inl 0)
```

def toTimeAndSpace[\[source\]](#)

A continuous linear equivalence between SpaceTime d and Time × Space d.

```
def toTimeAndSpace {d : ℕ} : SpaceTime d ≈L [ℝ] Time × Space d
```

lemma toTimeAndSpace_basis_natAdd[\[source\]](#)

```
lemma toTimeAndSpace_basis_natAdd {d : ℕ} (i : Fin d) :
  toTimeAndSpace ((Tensor.basis (S := realLorentzTensor d) ![Color.up])
    fun x => Fin.cast (by simp) (Fin.natAdd 1 i))
  = (0, Space.basis i)
```

lemma toTimeAndSpace_basis_castAdd[\[source\]](#)

```
lemma toTimeAndSpace_basis_castAdd {d : ℕ} :
  toTimeAndSpace ((Tensor.basis (S := realLorentzTensor d) ![Color.up])
    fun x => Fin.cast (by simp) (Fin.castAdd d (0 : Fin 1))) = (1, 0)
```

lemma deriv_toTimeAndSpace[\[source\]](#)

```
@[simp]
lemma deriv_toTimeAndSpace {d : ℕ} (μ : Fin (1 + d)) (y : SpaceTime d) :
  SpaceTime.deriv μ (toTimeAndSpace) y = toTimeAndSpace
    ((Tensor.basis _ (fun x => Fin.cast (by simp) μ)))
```

lemma deriv_comp_toTimeAndSpace_natAdd[\[source\]](#)

```

lemma deriv_comp_toTimeAndSpace_natAdd {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M] {d :
  ℕ}
  (i : Fin (d)) (f : Time × Space d → M) (y : SpaceTime d) :
  SpaceTime.deriv (Fin.natAdd 1 i) (f ∘ toTimeAndSpace) y =
  fderiv ℝ f (toTimeAndSpace y) (0, Space.basis i)

```

lemma deriv_comp_toTimeAndSpace_castAdd[\[source\]](#)

```

lemma deriv_comp_toTimeAndSpace_castAdd {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M]
  {d : ℕ} (f : Time × Space d → M) (y : SpaceTime d) :
  SpaceTime.deriv (Fin.castAdd d 0) (f ∘ toTimeAndSpace) y =
  fderiv ℝ f (toTimeAndSpace y) (1, 0)

```

2.36 PhysLean.Relativity.SpaceTime.TimeSlice

[PhysLean/Relativity/SpaceTime/TimeSlice.lean](#) on GitHub.

def timeSlice[\[source\]](#)

The timeslice of a function $\text{SpaceTime } d \rightarrow M$ forming a function $\text{Time} \rightarrow \text{Space } d \rightarrow M$.

```

def timeSlice {d : ℕ} {M : Type} : (SpaceTime d → M) ≈ (Time → Space d → M) where

```

def timeSliceLinearEquiv[\[source\]](#)

The timeslice of a function $\text{SpaceTime } d \rightarrow M$ forming a function $\text{Time} \rightarrow \text{Space } d \rightarrow M$, as a linear equivalence.

```

def timeSliceLinearEquiv {d : ℕ} {M : Type} [AddCommGroup M] [Module ℝ M] :
  (SpaceTime d → M) ≈ℓ[ℝ] (Time → Space d → M) where

```

lemma timeSliceLinearEquiv_apply[\[source\]](#)

```

lemma timeSliceLinearEquiv_apply {d : ℕ} {M : Type} [AddCommGroup M] [Module ℝ M]
  (f : SpaceTime d → M) : timeSliceLinearEquiv f = timeSlice f

```

lemma timeSliceLinearEquiv_symm_apply[\[source\]](#)

```

lemma timeSliceLinearEquiv_symm_apply {d : ℕ} {M : Type} [AddCommGroup M] [Module ℝ M]
  (f : Time → Space d → M) : timeSliceLinearEquiv.symm f = timeSlice.symm f

```

lemma timeSlice_spatial_deriv[\[source\]](#)

The derivative on space commutes with time-slicing.

```

lemma timeSlice_spatial_deriv {M : Type}
  [NormedAddCommGroup M] [NormedSpace ℝ M] {d : ℕ} {f : SpaceTime d → M}
  {t : Time} {x : Space d}
  (hdiff : DifferentiableAt ℝ f (toTimeAndSpace.symm (t, x))) (i : Fin d) :
  timeSlice (∂_ (Fin.natAdd 1 i) f) t x = ∂[i] (timeSlice f t) x

```

lemma timeSlice_time_deriv[\[source\]](#)

The derivative on time commutes with time-slicing.

```
lemma timeSlice_time_deriv {M : Type}
  [NormedAddCommGroup M] [NormedSpace ℝ M] {d : ℕ} (f : SpaceTime d → M)
  {t : Time} {x : Space d}
  (hdiff : DifferentiableAt ℝ f (toTimeAndSpace.symm (t, x))) :
  timeSlice (∂_ (finSumFinEquiv (Sum.inl 0)) f) t x = ∂_ (fun t => timeSlice f t x) t
```

2.37 PhysLean.Relativity.Special.TwinParadox.Basic

[PhysLean/Relativity/Special/TwinParadox/Basic.lean](#) on GitHub.

informal_lemma ageGap_nonneg[\[source\]](#)

In the twin paradox with instantous acceleration, Twin A is always older then Twin B.

```
informal_lemma ageGap_nonneg where
  deps := [``ageGap]
  tag := "7ROVE"
```

2.38 PhysLean.Relativity.Tensors.Basic

[PhysLean/Relativity/Tensors/Basic.lean](#) on GitHub.

lemma map_mid_move_left[\[source\]](#)

```
lemma map_mid_move_left {n n1 : ℕ} {c : Fin n → S.C} {c1 : Fin n1 → S.C} (p : Pure S c)
  (p' : Pure S c1) {c' : S.C}
  (i : Fin n) (j : Fin n1) (hi : c i = c') (hj : c1 j = c') :
  S.FD.map (eqToHom (by rw [hi] : { as := c i } = { as := c' })) (p i) =
  S.FD.map (eqToHom (by rw [hj] : { as := c1 j } = { as := c' })) (p' j)
  ↔ S.FD.map (eqToHom (by rw [hi, hj] : { as := c i } = { as := c1 j })) (p i) =
  (p' j)
```

lemma basis_repr_pure[\[source\]](#)

```
@[simp]
lemma basis_repr_pure {n : ℕ} (c : Fin n → S.C)
  (p : Pure S c) :
  (basis c).repr p.toTensor = p.component
```

lemma finrank_tensor_eq[\[source\]](#)

```
lemma finrank_tensor_eq {n : ℕ} [StrongRankCondition k] (c : Fin n → S.C) :
  Module.finrank k (S.Tensor c) = [] x, S.repDim (c x)
```

instance SMul (G) (Pure S c)[\[source\]](#)

```
noncomputable instance : SMul (G) (Pure S c)
```

lemma auto[\[source\]](#)

```

lemma PermCond.auto {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  {σ : Fin m → Fin n} (h : PermCond c c1 σ := by {simp [PermCond]; try decide}) :
  PermCond c c1 σ

```

lemma on_id[\[source\]](#)

```

@[simp]
lemma PermCond.on_id {n : ℕ} {c c1 : Fin n → S.C} :
  PermCond c c1 (id : Fin n → Fin n) ↔ ∀ i, c i = c1 i

```

lemma on_id_symm[\[source\]](#)

```

lemma PermCond.on_id_symm {n : ℕ} {c c1 : Fin n → S.C} (h : PermCond c1 c id) :
  PermCond c c1 (id : Fin n → Fin n)

```

lemma apply_inv_apply[\[source\]](#)

```

lemma PermCond.apply_inv_apply {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  (σ : Fin m → Fin n) (h : PermCond c c1 σ) (x : Fin m) :
  h.inv σ (σ x) = x

```

lemma inv_apply_apply[\[source\]](#)

```

lemma PermCond.inv_apply_apply {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  (σ : Fin m → Fin n) (h : PermCond c c1 σ) (x : Fin n) :
  σ (h.inv σ x) = x

```

lemma inv_perserve_color[\[source\]](#)

```

lemma PermCond.inv_perserve_color {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  {σ : Fin m → Fin n} (h : PermCond c c1 σ) (x : Fin n) :
  c1 (h.inv σ x) = c x

```

lemma permP_id_self[\[source\]](#)

```

@[simp]
lemma Pure.permP_id_self {n : ℕ} {c : Fin n → S.C} (p : Pure S c) :
  Pure.permP (id : Fin n → Fin n) (by simp : PermCond c c id) p = p

```

lemma permT_id_self[\[source\]](#)

```

@[simp]
lemma permT_id_self {n : ℕ} {c : Fin n → S.C} (t : S.Tensor c) :
  permT (id : Fin n → Fin n) (by simp : PermCond c c id) t = t

```

lemma permT_congr_eq_id[\[source\]](#)

```

lemma permT_congr_eq_id {n : ℕ} {c : Fin n → S.C} (t : S.Tensor c)
  (σ : Fin n → Fin n) (hσ : PermCond c c σ) (h : σ = id) :
  permT σ (hσ) t = t

```

lemma permT_congr_eq_id'[\[source\]](#)

```
lemma permT_congr_eq_id' {n : ℕ} {c : Fin n → S.C} (t t1 : S.Tensor c)
  (σ : Fin n → Fin n) (hσ : PermCond c c σ) (h : σ = id) (ht : t = t1) :
  permT σ (hσ) t = t1
```

lemma permT_basis_repr_symm_apply[\[source\]](#)

```
lemma permT_basis_repr_symm_apply {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  {σ : Fin m → Fin n} (h : PermCond c c1 σ) (t : S.Tensor c)
  (b : ComponentIdx c1) :
  (basis c1).repr (permT σ h t) b =
  (basis c).repr t (fun i =>
    Fin.cast (by simp [PermCond.inv_perserve_color]) (b (h.inv σ i)))
```

def toField[\[source\]](#)

The linear map between tensors with zero indices and the underlying field k .

```
noncomputable def toField {c : Fin 0 → S.C} : S.Tensor c →l[k] k
```

lemma toField_default[\[source\]](#)

```
lemma toField_default {c : Fin 0 → S.C} :
  toField (Pure.toTensor default : S.Tensor c) = 1
```

lemma toField_pure[\[source\]](#)

```
@[simp]
lemma toField_pure {c : Fin 0 → S.C} (p : Pure S c) :
  toField (p.toTensor : S.Tensor c) = 1
```

lemma toField_basis_default[\[source\]](#)

```
@[simp]
lemma toField_basis_default {c : Fin 0 → S.C} :
  toField (basis c (@default (ComponentIdx c) Unique.instInhabited)) = 1
```

lemma toField_eq_repr[\[source\]](#)

```
lemma toField_eq_repr {c : Fin 0 → S.C} (t : Tensor S c) :
  t.toField = (basis c).repr t (fun j => Fin.elim0 j)
```

lemma toField_equivariant[\[source\]](#)

```
@[simp]
lemma toField_equivariant {c : Fin 0 → S.C} (g : G) (t : Tensor S c) :
  toField (g • t) = toField t
```

2.39 PhysLean.Relativity.Tensors.ComplexTensor.Basic

[PhysLean/Relativity/Tensors/ComplexTensor/Basic.lean](#) on GitHub.

lemma basis_up_eq[\[source\]](#)

```
lemma basis_up_eq {i : Fin 4} :
  complexLorentzTensor.basis Color.up i = Lorentz.complexContrBasisFin4 i
```

lemma basis_down_eq[\[source\]](#)

```
lemma basis_down_eq {i : Fin 4} :
  complexLorentzTensor.basis Color.down i = Lorentz.complexCoBasisFin4 i
```

lemma basis_upL_eq[\[source\]](#)

```
lemma basis_upL_eq {i : Fin 2} :
  complexLorentzTensor.basis Color.upL i = Fermion.leftBasis i
```

lemma basis_downL_eq[\[source\]](#)

```
lemma basis_downL_eq {i : Fin 2} :
  complexLorentzTensor.basis Color.downL i = Fermion.altLeftBasis i
```

lemma basis_upR_eq[\[source\]](#)

```
lemma basis_upR_eq {i : Fin 2} :
  complexLorentzTensor.basis Color.upR i = Fermion.rightBasis i
```

lemma basis_downR_eq[\[source\]](#)

```
lemma basis_downR_eq {i : Fin 2} :
  complexLorentzTensor.basis Color.downR i = Fermion.altRightBasis i
```

2.40 PhysLean.Relativity.Tensors.ComplexTensor.Lemmas

[PhysLean/Relativity/Tensors/ComplexTensor/Lemmas.lean](#) on GitHub.

lemma antiSymm_contr_symm[\[source\]](#)

```
lemma antiSymm_contr_symm {A : CT[.up, .up]} {S : CT[.down, .down]}
  (hA : {A | μ ν = - (A | ν μ)}T) (hS : {S | μ ν = S | ν μ}T) :
  {A | μ ν ⊗ S | μ ν = - A | μ ν ⊗ S | μ ν}T
```

2.41 PhysLean.Relativity.Tensors.ComplexTensor.Metrics.Basic

[PhysLean/Relativity/Tensors/ComplexTensor/Metrics/Basic.lean](#) on GitHub.

lemma coMetric_eq_fromConstPair[\[source\]](#)

```
lemma coMetric_eq_fromConstPair : η' = fromConstPair Lorentz.coMetric
```

lemma contrMetric_eq_fromConstPair[\[source\]](#)

```
lemma contrMetric_eq_fromConstPair : η = fromConstPair Lorentz.contrMetric
```

lemma leftMetric_eq_fromConstPair[\[source\]](#)

```
lemma leftMetric_eq_fromConstPair : εL = fromConstPair Fermion.leftMetric
```

lemma rightMetric_eq_fromConstPair[\[source\]](#)

```
lemma rightMetric_eq_fromConstPair : εR = fromConstPair Fermion.rightMetric
```

lemma altLeftMetric_eq_fromConstPair[\[source\]](#)

```
lemma altLeftMetric_eq_fromConstPair : εL' = fromConstPair Fermion.altLeftMetric
```

lemma altRightMetric_eq_fromConstPair[\[source\]](#)

```
lemma altRightMetric_eq_fromConstPair : εR' = fromConstPair Fermion.altRightMetric
```

lemma coMetric_eq_fromPairT[\[source\]](#)

```
lemma coMetric_eq_fromPairT : η' = fromPairT (Lorentz.coMetricVal)
```

lemma contrMetric_eq_fromPairT[\[source\]](#)

```
lemma contrMetric_eq_fromPairT : η = fromPairT (Lorentz.contrMetricVal)
```

lemma leftMetric_eq_fromPairT[\[source\]](#)

```
lemma leftMetric_eq_fromPairT : εL = fromPairT (Fermion.leftMetricVal)
```

lemma rightMetric_eq_fromPairT[\[source\]](#)

```
lemma rightMetric_eq_fromPairT : εR = fromPairT (Fermion.rightMetricVal)
```

lemma altLeftMetric_eq_fromPairT[\[source\]](#)

```
lemma altLeftMetric_eq_fromPairT : εL' = fromPairT (Fermion.altLeftMetricVal)
```

lemma altRightMetric_eq_fromPairT[\[source\]](#)

```
lemma altRightMetric_eq_fromPairT : εR' = fromPairT (Fermion.altRightMetricVal)
```

lemma coMetric_eq_complexCoBasis[\[source\]](#)

```
lemma coMetric_eq_complexCoBasis : η' =
  fromPairT (complexCoBasis (Sum.inl 0) ⊗ₜ[ℂ] complexCoBasis (Sum.inl 0))
  - fromPairT (complexCoBasis (Sum.inr 0) ⊗ₜ[ℂ] complexCoBasis (Sum.inr 0))
  - fromPairT (complexCoBasis (Sum.inr 1) ⊗ₜ[ℂ] complexCoBasis (Sum.inr 1))
  - fromPairT (complexCoBasis (Sum.inr 2) ⊗ₜ[ℂ] complexCoBasis (Sum.inr 2))
```

lemma coMetric_eq_complexCoBasisFin4[\[source\]](#)

```

lemma coMetric_eq_complexCoBasisFin4 :  $\eta'$  =
  fromPairT (complexCoBasisFin4 0  $\otimes_{\mathbb{C}}$  complexCoBasisFin4 0)
- fromPairT (complexCoBasisFin4 1  $\otimes_{\mathbb{C}}$  complexCoBasisFin4 1)
- fromPairT (complexCoBasisFin4 2  $\otimes_{\mathbb{C}}$  complexCoBasisFin4 2)
- fromPairT (complexCoBasisFin4 3  $\otimes_{\mathbb{C}}$  complexCoBasisFin4 3)

```

lemma contrMetric_eq_complexContrBasis[\[source\]](#)

```

lemma contrMetric_eq_complexContrBasis :  $\eta$  =
  fromPairT (complexContrBasis (Sum.inl 0)  $\otimes_{\mathbb{C}}$  complexContrBasis (Sum.inl 0))
- fromPairT (complexContrBasis (Sum.inr 0)  $\otimes_{\mathbb{C}}$  complexContrBasis (Sum.inr 0))
- fromPairT (complexContrBasis (Sum.inr 1)  $\otimes_{\mathbb{C}}$  complexContrBasis (Sum.inr 1))
- fromPairT (complexContrBasis (Sum.inr 2)  $\otimes_{\mathbb{C}}$  complexContrBasis (Sum.inr 2))

```

lemma contrMetric_eq_complexContrBasisFin4[\[source\]](#)

```

lemma contrMetric_eq_complexContrBasisFin4 :  $\eta$  =
  fromPairT (complexContrBasisFin4 0  $\otimes_{\mathbb{C}}$  complexContrBasisFin4 0)
- fromPairT (complexContrBasisFin4 1  $\otimes_{\mathbb{C}}$  complexContrBasisFin4 1)
- fromPairT (complexContrBasisFin4 2  $\otimes_{\mathbb{C}}$  complexContrBasisFin4 2)
- fromPairT (complexContrBasisFin4 3  $\otimes_{\mathbb{C}}$  complexContrBasisFin4 3)

```

lemma leftMetric_eq_leftBasis[\[source\]](#)

```

lemma leftMetric_eq_leftBasis :  $\varepsilon_L$  =
  - fromPairT (leftBasis 0  $\otimes_{\mathbb{C}}$  leftBasis 1)
+ fromPairT (leftBasis 1  $\otimes_{\mathbb{C}}$  leftBasis 0)

```

lemma altLeftMetric_eq_altLeftBasis[\[source\]](#)

```

lemma altLeftMetric_eq_altLeftBasis :  $\varepsilon_L'$  =
  fromPairT (altLeftBasis 0  $\otimes_{\mathbb{C}}$  altLeftBasis 1)
- fromPairT (altLeftBasis 1  $\otimes_{\mathbb{C}}$  altLeftBasis 0)

```

lemma rightMetric_eq_rightBasis[\[source\]](#)

```

lemma rightMetric_eq_rightBasis :  $\varepsilon_R$  =
  - fromPairT (rightBasis 0  $\otimes_{\mathbb{C}}$  rightBasis 1)
+ fromPairT (rightBasis 1  $\otimes_{\mathbb{C}}$  rightBasis 0)

```

lemma altRightMetric_eq_altRightBasis[\[source\]](#)

```

lemma altRightMetric_eq_altRightBasis :  $\varepsilon_R'$  =
  fromPairT (altRightBasis 0  $\otimes_{\mathbb{C}}$  altRightBasis 1)
- fromPairT (altRightBasis 1  $\otimes_{\mathbb{C}}$  altRightBasis 0)

```

lemma coMetric_eq_basis[\[source\]](#)

```

lemma coMetric_eq_basis :  $\eta'$  =
  (Tensor.basis (S := complexLorentzTensor) ![Color.down, Color.down] (fun | 0 => 0 | 1 => 0)
  )

```

```

- (Tensor.basis (S := complexLorentzTensor) ![Color.down, Color.down] (fun | 0 => 1 | 1 => 1))
- (Tensor.basis (S := complexLorentzTensor) ![Color.down, Color.down] (fun | 0 => 2 | 1 => 2))
- (Tensor.basis (S := complexLorentzTensor) ![Color.down, Color.down]
  (fun | 0 => 3 | 1 => 3))

```

lemma contrMetric_eq_basis[\[source\]](#)

```

lemma contrMetric_eq_basis : η =
  (Tensor.basis (S := complexLorentzTensor) ![Color.up, Color.up] (fun | 0 => 0 | 1 => 0))
- (Tensor.basis (S := complexLorentzTensor) ![Color.up, Color.up] (fun | 0 => 1 | 1 => 1))
- (Tensor.basis (S := complexLorentzTensor) ![Color.up, Color.up] (fun | 0 => 2 | 1 => 2))
- (Tensor.basis (S := complexLorentzTensor) ![Color.up, Color.up]
  (fun | 0 => 3 | 1 => 3))

```

lemma leftMetric_eq_basis[\[source\]](#)

```

lemma leftMetric_eq_basis : εL =
  - (Tensor.basis (S := complexLorentzTensor) ![Color.upL, Color.upL] (fun | 0 => 0 | 1 => 1)
  )
+ (Tensor.basis (S := complexLorentzTensor)
  ![Color.upL, Color.upL] (fun | 0 => 1 | 1 => 0))

```

lemma altLeftMetric_eq_basis[\[source\]](#)

```

lemma altLeftMetric_eq_basis : εL' =
  (Tensor.basis (S := complexLorentzTensor) ![Color.downL, Color.downL] (fun | 0 => 0 | 1 => 1))
- (Tensor.basis (S := complexLorentzTensor)
  ![Color.downL, Color.downL] (fun | 0 => 1 | 1 => 0))

```

lemma rightMetric_eq_basis[\[source\]](#)

```

lemma rightMetric_eq_basis : εR =
  - (Tensor.basis (S := complexLorentzTensor) ![Color.upR, Color.upR] (fun | 0 => 0 | 1 => 1)
  )
+ (Tensor.basis (S := complexLorentzTensor)
  ![Color.upR, Color.upR] (fun | 0 => 1 | 1 => 0))

```

lemma altRightMetric_eq_basis[\[source\]](#)

```

lemma altRightMetric_eq_basis : εR' =
  (Tensor.basis (S := complexLorentzTensor)
    ![Color.downR, Color.downR] (fun | 0 => 0 | 1 => 1))
- (Tensor.basis (S := complexLorentzTensor)
  ![Color.downR, Color.downR] (fun | 0 => 1 | 1 => 0))

```

lemma actionT_coMetric[\[source\]](#)

The tensor coMetric is invariant under the action of $SL(2, \mathbb{C})$.

```

lemma actionT_coMetric (g : SL(2, ℂ)) : g • η' = η'

```

lemma actionT_contrMetric[\[source\]](#)

The tensor `contrMetric` is invariant under the action of $SL(2, \mathbb{C})$.

```
lemma actionT_contrMetric (g : SL(2, ℂ)) : g • η = η
```

lemma actionT_leftMetric[\[source\]](#)

The tensor `leftMetric` is invariant under the action of $SL(2, \mathbb{C})$.

```
lemma actionT_leftMetric (g : SL(2, ℂ)) : g • εL = εL
```

lemma actionT_rightMetric[\[source\]](#)

The tensor `rightMetric` is invariant under the action of $SL(2, \mathbb{C})$.

```
lemma actionT_rightMetric (g : SL(2, ℂ)) : g • εR = εR
```

lemma actionT_altLeftMetric[\[source\]](#)

The tensor `altLeftMetric` is invariant under the action of $SL(2, \mathbb{C})$.

```
lemma actionT_altLeftMetric (g : SL(2, ℂ)) : g • εL' = εL'
```

lemma actionT_altRightMetric[\[source\]](#)

The tensor `altRightMetric` is invariant under the action of $SL(2, \mathbb{C})$.

```
lemma actionT_altRightMetric (g : SL(2, ℂ)) : g • εR' = εR'
```

2.42 PhysLean.Relativity.Tensors.ComplexTensor.OfRat

[PhysLean/Relativity/Tensors/ComplexTensor/OfRat.lean](#) on GitHub.

lemma ofRat_basis_repr_apply[\[source\]](#)

```
@[simp]
lemma ofRat_basis_repr_apply {n : ℕ} {c : Fin n → complexLorentzTensor.C}
  (f : (ComponentIdx c) → RatComplexNum)
  (b : (ComponentIdx c)) :
  (Tensor.basis c).repr (ofRat f) b = toComplexNum (f b)
```

lemma basis_eq_ofRat[\[source\]](#)

```
lemma basis_eq_ofRat {n : ℕ} {c : Fin n → complexLorentzTensor.C}
  (b : (ComponentIdx c)) :
  Tensor.basis c b
  = ofRat (fun b' => if b = b' then ⟨1, 0⟩ else ⟨0, 0⟩)
```

lemma prodT_ofRat_ofRat[\[source\]](#)

```
lemma prodT_ofRat_ofRat {n n1 : ℕ} {c : Fin n → complexLorentzTensor.C}
  (f : (ComponentIdx c) → RatComplexNum)
  {c1 : Fin n1 → complexLorentzTensor.C}
```

```
(f1 : (ComponentIdx c1) → RatComplexNum) :
(prodT (ofRat f) (ofRat f1)) =
((ofRat (fun b => f (ComponentIdx.splitEquiv b).1 *
  f1 (ComponentIdx.splitEquiv b).2)))
```

lemma contrT_ofRat_eq_sum_dropPairSection

[\[source\]](#)

```
lemma contrT_ofRat_eq_sum_dropPairSection {n : ℕ} {c : Fin (n + 1 + 1) → complexLorentzTensor.C}
{ i j : Fin (n + 1 + 1)} {h : i ≠ j ∧ complexLorentzTensor.τ (c i) = c j }
(f : (ComponentIdx c) → RatComplexNum) :
(contrT n i j h (ofRat f)) = ((ofRat (fun b =>
  (∑ x : ComponentIdx.DropPairSection b,
    f x.1 * if (x.1 i).1 = (x.1 j).1 then 1 else 0))))
```

lemma contrT_ofRat

[\[source\]](#)

```
lemma contrT_ofRat {n : ℕ} {c : Fin (n + 1 + 1) → complexLorentzTensor.C}
{ i j : Fin (n + 1 + 1)} {h : i ≠ j ∧ complexLorentzTensor.τ (c i) = c j }
(f : (ComponentIdx c) → RatComplexNum) :
(contrT n i j h (ofRat f)) = ((ofRat (fun b =>
  (∑ x : Fin (complexLorentzTensor.repDim (c i)),
    f (DropPairSection.ofFinEquiv h.1 b (x, Fin.cast (by simp [- h.2]) x))))))
```

lemma permT_ofRat

[\[source\]](#)

```
lemma permT_ofRat {n m : ℕ} {c : Fin n → complexLorentzTensor.C}
{c1 : Fin m → complexLorentzTensor.C}
{σ : Fin m → Fin n} (h : PermCond c c1 σ)
(f : ComponentIdx c → RatComplexNum) :
(permT σ h ((ofRat f))) =
((ofRat (fun b => f (fun i => Fin.cast (by simp [PermCond.inv_perserve_color])
  (b (h.inv σ i))))))
```

2.43 PhysLean.Relativity.Tensors.ComplexTensor.Units.Basic

[PhysLean/Relativity/Tensors/ComplexTensor/Units/Basic.lean](#) on GitHub.

lemma coContrUnit_eq_fromConstPair

[\[source\]](#)

```
lemma coContrUnit_eq_fromConstPair : δ' = fromConstPair Lorentz.coContrUnit
```

lemma contrCoUnit_eq_fromConstPair

[\[source\]](#)

```
lemma contrCoUnit_eq_fromConstPair : δ = fromConstPair Lorentz.contrCoUnit
```

lemma altLeftLeftUnit_eq_fromConstPair

[\[source\]](#)

```
lemma altLeftLeftUnit_eq_fromConstPair : δL' = fromConstPair Fermion.altLeftLeftUnit
```

lemma leftAltLeftUnit_eq_fromConstPair[\[source\]](#)

```
lemma leftAltLeftUnit_eq_fromConstPair :  $\delta L = \text{fromConstPair Fermion.leftAltLeftUnit}$ 
```

lemma altRightRightUnit_eq_fromConstPair[\[source\]](#)

```
lemma altRightRightUnit_eq_fromConstPair :  $\delta R' = \text{fromConstPair Fermion.altRightRightUnit}$ 
```

lemma rightAltRightUnit_eq_fromConstPair[\[source\]](#)

```
lemma rightAltRightUnit_eq_fromConstPair :  $\delta R = \text{fromConstPair Fermion.rightAltRightUnit}$ 
```

lemma coContrUnit_eq_fromPairT[\[source\]](#)

```
lemma coContrUnit_eq_fromPairT :  $\delta' = \text{fromPairT (Lorentz.coContrUnitVal)}$ 
```

lemma contrCoUnit_eq_fromPairT[\[source\]](#)

```
lemma contrCoUnit_eq_fromPairT :  $\delta = \text{fromPairT (Lorentz.contrCoUnitVal)}$ 
```

lemma altLeftLeftUnit_eq_fromPairT[\[source\]](#)

```
lemma altLeftLeftUnit_eq_fromPairT :  $\delta L' = \text{fromPairT (Fermion.altLeftLeftUnitVal)}$ 
```

lemma leftAltLeftUnit_eq_fromPairT[\[source\]](#)

```
lemma leftAltLeftUnit_eq_fromPairT :  $\delta L = \text{fromPairT (Fermion.leftAltLeftUnitVal)}$ 
```

lemma altRightRightUnit_eq_fromPairT[\[source\]](#)

```
lemma altRightRightUnit_eq_fromPairT :  $\delta R' = \text{fromPairT (Fermion.altRightRightUnitVal)}$ 
```

lemma rightAltRightUnit_eq_fromPairT[\[source\]](#)

```
lemma rightAltRightUnit_eq_fromPairT :  $\delta R = \text{fromPairT (Fermion.rightAltRightUnitVal)}$ 
```

lemma coContrUnit_eq_complexCoBasis_complexContrBasis[\[source\]](#)

```
lemma coContrUnit_eq_complexCoBasis_complexContrBasis :  $\delta' = \sum i, \text{fromPairT (complexCoBasis } i \otimes_{\mathbb{C}} \text{ complexContrBasis } i)$ 
```

lemma coContrUnit_eq_complexCoBasisFin4_complexContrBasisFin4[\[source\]](#)

```
lemma coContrUnit_eq_complexCoBasisFin4_complexContrBasisFin4 :  $\delta' = \sum i, \text{fromPairT (complexCoBasisFin4 } i \otimes_{\mathbb{C}} \text{ complexContrBasisFin4 } i)$ 
```

lemma contrCoUnit_eq_complexContrBasis_complexCoBasis[\[source\]](#)

```
lemma contrCoUnit_eq_complexContrBasis_complexCoBasis :  $\delta = \sum i, \text{fromPairT (complexContrBasis } i \otimes_{\mathbb{C}} \text{ complexCoBasis } i)$ 
```

lemma contrCoUnit_eq_complexContrBasisFin4_complexCoBasisFin4[\[source\]](#)

```
lemma contrCoUnit_eq_complexContrBasisFin4_complexCoBasisFin4 :  $\delta =$ 
   $\sum i$ , fromPairT (complexContrBasisFin4 i  $\otimes_{\mathbb{C}}$  complexCoBasisFin4 i)
```

lemma altLeftLeftUnit_eq_altLeftBasis_leftBasis[\[source\]](#)

```
lemma altLeftLeftUnit_eq_altLeftBasis_leftBasis :  $\delta L' =$ 
   $\sum i$ , fromPairT (altLeftBasis i  $\otimes_{\mathbb{C}}$  leftBasis i)
```

lemma leftAltLeftUnit_eq_leftBasis_altLeftBasis[\[source\]](#)

```
lemma leftAltLeftUnit_eq_leftBasis_altLeftBasis :  $\delta L =$ 
   $\sum i$ , fromPairT (leftBasis i  $\otimes_{\mathbb{C}}$  altLeftBasis i)
```

lemma altRightRightUnit_eq_altRightBasis_rightBasis[\[source\]](#)

```
lemma altRightRightUnit_eq_altRightBasis_rightBasis :  $\delta R' =$ 
   $\sum i$ , fromPairT (altRightBasis i  $\otimes_{\mathbb{C}}$  rightBasis i)
```

lemma rightAltRightUnit_eq_rightBasis_altRightBasis[\[source\]](#)

```
lemma rightAltRightUnit_eq_rightBasis_altRightBasis :  $\delta R =$ 
   $\sum i$ , fromPairT (rightBasis i  $\otimes_{\mathbb{C}}$  altRightBasis i)
```

lemma coContrUnit_eq_basis[\[source\]](#)

```
lemma coContrUnit_eq_basis :  $\delta' =$ 
   $\sum i$ , Tensor.basis (S := complexLorentzTensor)
    ![Color.down, Color.up] (fun | 0 => i | 1 => i)
```

lemma contrCoUnit_eq_basis[\[source\]](#)

```
lemma contrCoUnit_eq_basis :  $\delta =$ 
   $\sum i$ , Tensor.basis (S := complexLorentzTensor)
    ![Color.up, Color.down] (fun | 0 => i | 1 => i)
```

lemma altLeftLeftUnit_eq_basis[\[source\]](#)

```
lemma altLeftLeftUnit_eq_basis :  $\delta L' =$ 
   $\sum i$ , Tensor.basis (S := complexLorentzTensor)
    ![Color.downL, Color.upL] (fun | 0 => i | 1 => i)
```

lemma leftAltLeftUnit_eq_basis[\[source\]](#)

```
lemma leftAltLeftUnit_eq_basis :  $\delta L =$ 
   $\sum i$ , Tensor.basis (S := complexLorentzTensor)
    ![Color.upL, Color.downL] (fun | 0 => i | 1 => i)
```

lemma altRightRightUnit_eq_basis[\[source\]](#)

```
lemma altRightRightUnit_eq_basis : δR' =
  ∑ i, Tensor.basis (S := complexLorentzTensor)
    ![Color.downR, Color.upR] (fun | 0 => i | 1 => i)
```

lemma rightAltRightUnit_eq_basis[\[source\]](#)

```
lemma rightAltRightUnit_eq_basis : δR =
  ∑ i, Tensor.basis (S := complexLorentzTensor)
    ![Color.upR, Color.downR] (fun | 0 => i | 1 => i)
```

lemma altLeftLeftUnit_eq_ofRat[\[source\]](#)

```
lemma altLeftLeftUnit_eq_ofRat : δL' = ofRat fun f =>
  if f 0 = f 1 then 1 else 0
```

lemma leftAltLeftUnit_eq_ofRat[\[source\]](#)

```
lemma leftAltLeftUnit_eq_ofRat : δL = ofRat fun f =>
  if f 0 = f 1 then 1 else 0
```

lemma altRightRightUnit_eq_ofRat[\[source\]](#)

```
lemma altRightRightUnit_eq_ofRat : δR' = ofRat fun f =>
  if f 0 = f 1 then 1 else 0
```

lemma rightAltRightUnit_eq_ofRat[\[source\]](#)

```
lemma rightAltRightUnit_eq_ofRat : δR = ofRat fun f =>
  if f 0 = f 1 then 1 else 0
```

lemma actionT_coContrUnit[\[source\]](#)

The tensor coContrUnit is invariant under the action of $SL(2, \mathbb{C})$.

```
lemma actionT_coContrUnit (g : SL(2, ℂ)) : g • δ' = δ'
```

lemma actionT_contrCoUnit[\[source\]](#)

The tensor contrCoUnit is invariant under the action of $SL(2, \mathbb{C})$.

```
lemma actionT_contrCoUnit (g : SL(2, ℂ)) : g • δ = δ
```

lemma actionT_altLeftLeftUnit[\[source\]](#)

The tensor altLeftLeftUnit is invariant under the action of $SL(2, \mathbb{C})$.

```
lemma actionT_altLeftLeftUnit (g : SL(2, ℂ)) : g • δL' = δL'
```

lemma actionT_leftAltLeftUnit[\[source\]](#)

The tensor `leftAltLeftUnit` is invariant under the action of $SL(2, \mathbb{C})$.

```
lemma actionT_leftAltLeftUnit (g : SL(2, ℂ)) : g • δL = δL
```

lemma actionT_altRightRightUnit[\[source\]](#)

The tensor `altRightRightUnit` is invariant under the action of $SL(2, \mathbb{C})$.

```
lemma actionT_altRightRightUnit (g : SL(2, ℂ)) : g • δR' = δR'
```

lemma actionT_rightAltRightUnit[\[source\]](#)

The tensor `rightAltRightUnit` is invariant under the action of $SL(2, \mathbb{C})$.

```
lemma actionT_rightAltRightUnit (g : SL(2, ℂ)) : g • δR = δR
```

2.44 PhysLean.Relativity.Tensors.Constructors

[PhysLean/Relativity/Tensors/Constructors.lean](#) on GitHub.

def fromSingleP[\[source\]](#)

The equivalence between $S.FD.obj \{as := c\}$ and $Pure\ S \ ![c]$.

```
noncomputable def Pure.fromSingleP {c : S.C} : S.FD.obj {as := c} ≈l[k] Pure S ![c] where
```

def fromSingleT[\[source\]](#)

The equivalence between $S.FD.obj \{as := c\}$ and $S.Tensor \ ![c]$.

```
noncomputable def fromSingleT {c : S.C} : S.FD.obj {as := c} ≈l[k] S.Tensor ![c] where
```

lemma fromSingleT_symm_pure[\[source\]](#)

```
lemma fromSingleT_symm_pure {c : S.C} (p : Pure S ![c]) :  
  fromSingleT.symm p.toTensor = Pure.fromSingleP.symm p
```

lemma fromSingleT_eq_pureT[\[source\]](#)

```
lemma fromSingleT_eq_pureT {c : S.C} (x : S.FD.obj {as := c}) :  
  fromSingleT x = Pure.toTensor (fun 0 => x : Pure S ![c])
```

lemma actionT_fromSingleT[\[source\]](#)

```
lemma actionT_fromSingleT {c : S.C} (x : S.FD.obj {as := c}) (g : G) :  
  g • fromSingleT x = fromSingleT ((S.FD.obj {as := c}).p g x)
```

lemma fromSingleT_map[\[source\]](#)

```

lemma fromSingleT_map {c c1 : S.C}
  (h : ({as := c} : Discrete S.C) = {as := c1}) (x : S.FD.obj {as := c}) :
  fromSingleT (S.FD.map (eqToHom h) x) =
  permT id (by simp at h; simp [h]) (fromSingleT x)

```

lemma contrT_fromSingleT_fromSingleT[\[source\]](#)

```

lemma contrT_fromSingleT_fromSingleT {c : S.C} (x : S.FD.obj {as := c})
  (y : S.FD.obj {as := S.τ c}) :
  contrT 0 0 1 (by simp; rfl) (prodT (fromSingleT x) (fromSingleT y)) =
  (S.contr.app (Discrete.mk (c))) (x ⊗t[k] y) • (Pure.toTensor default)

```

def fromPairT[\[source\]](#)

The construction of a tensor with two indices from the tensor product (S.FD.obj (Discrete.mk c1)).V ⊗_t[k] (S.FD.obj (Discrete.mk c2)).V defined categorically.

```

noncomputable def fromPairT {c1 c2 : S.C} :
  (S.FD.obj (Discrete.mk c1)).V ⊗t[k] (S.FD.obj (Discrete.mk c2)).V →t[k] S.Tensor ![c1, c2]
where

```

lemma fromPairT_tmul[\[source\]](#)

```

lemma fromPairT_tmul {c1 c2 : S.C} (x : S.FD.obj (Discrete.mk c1))
  (y : S.FD.obj (Discrete.mk c2)) :
  fromPairT (x ⊗t[k] y) =
  permT id (And.intro Function.bijective_id (fun i => by fin_cases i <=> rfl))
  (prodT (fromSingleT x) (fromSingleT y))

```

lemma actionT_fromPairT[\[source\]](#)

```

lemma actionT_fromPairT {c1 c2 : S.C}
  (x : (S.FD.obj (Discrete.mk c1)).V ⊗t[k] (S.FD.obj (Discrete.mk c2)).V)
  (g : G) :
  g • fromPairT x = fromPairT (TensorProduct.map ((S.FD.obj (Discrete.mk c1)).p g)
    ((S.FD.obj (Discrete.mk c2)).p g) x)

```

lemma fromPairT_map_right[\[source\]](#)

```

lemma fromPairT_map_right {c1 c2 c2' : S.C} (h : c2 = c2')
  (x : (S.FD.obj (Discrete.mk c1)).V ⊗t[k] (S.FD.obj (Discrete.mk c2)).V) :
  fromPairT (TensorProduct.map LinearMap.id (S.FD.map (eqToHom (by rw [h]))).hom.hom' x) =
  permT id (by simp; intro i; fin_cases i <=> simp [h])
  (fromPairT x)

```

lemma fromPairT_comm[\[source\]](#)

```

lemma fromPairT_comm {c1 c2 : S.C}
  (x : (S.FD.obj (Discrete.mk c1)).V ⊗t[k] (S.FD.obj (Discrete.mk c2)).V) :
  fromPairT (TensorProduct.comm k _ x) =
  permT ![1, 0] (And.intro (by decide) (fun i => by fin_cases i <=> simp))
  (fromPairT x)

```

def fromSingleTContrFromPairT[\[source\]](#)

The contraction of tensors with one index with one with two indices defined categorically.

```
noncomputable def fromSingleTContrFromPairT {c c2 : S.C}
  (x : (S.FD.obj (Discrete.mk c)).V)
  (y : (S.FD.obj (Discrete.mk (S.τ c))).V ⊗[k] (S.FD.obj (Discrete.mk c2)).V) :
  S.Tensor ![c2]
```

lemma fromSingleTContrFromPairT_tmul[\[source\]](#)

```
lemma fromSingleTContrFromPairT_tmul {c c2 : S.C}
  (x : S.FD.obj (Discrete.mk c))
  (y1 : (S.FD.obj (Discrete.mk (S.τ c))).V)
  (y2 : (S.FD.obj (Discrete.mk c2)).V) :
  fromSingleTContrFromPairT x (y1 ⊗t[k] y2) =
  (S.contr.app (Discrete.mk (c))) (x ⊗t[k] y1) • fromSingleT y2
```

lemma fromSingleT_contr_fromPairT_tmul[\[source\]](#)

```
lemma fromSingleT_contr_fromPairT_tmul {c c2 : S.C}
  (x : S.FD.obj (Discrete.mk c))
  (y1 : (S.FD.obj (Discrete.mk (S.τ c))).V)
  (y2 : (S.FD.obj (Discrete.mk c2)).V) :
  contrT 1 0 1 (by simp; rfl)
  (prodT (fromSingleT x) (fromPairT (y1 ⊗t[k] y2))) =
  permT id (by simp; intro i; fin_cases i; rfl) (fromSingleTContrFromPairT x (y1 ⊗t[k] y2))
```

lemma contrT_fromSingleT_fromPairT[\[source\]](#)

```
lemma contrT_fromSingleT_fromPairT {c c2 : S.C}
  (x : S.FD.obj (Discrete.mk c))
  (y : (S.FD.obj (Discrete.mk (S.τ c))).V ⊗[k] (S.FD.obj (Discrete.mk c2)).V) :
  contrT 1 0 1 (by simp; rfl)
  (prodT (fromSingleT x) (fromPairT y)) =
  permT id (by simp; intro i; fin_cases i; rfl) (fromSingleTContrFromPairT x y)
```

def fromPairTContr[\[source\]](#)

The contraction of tensors with two indices defined categorically.

```
noncomputable def fromPairTContr {c c1 c2 : S.C}
  (x : (S.FD.obj (Discrete.mk c1)).V ⊗[k] (S.FD.obj (Discrete.mk c)).V)
  (y : (S.FD.obj (Discrete.mk (S.τ c))).V ⊗[k] (S.FD.obj (Discrete.mk c2)).V) :
  S.Tensor ![c1, c2]
```

lemma fromPairTContr_tmul_tmul[\[source\]](#)

```
lemma fromPairTContr_tmul_tmul {c c1 c2 : S.C}
  (x1 : S.FD.obj (Discrete.mk c1))
  (x2 : S.FD.obj (Discrete.mk c))
  (y1 : S.FD.obj (Discrete.mk (S.τ c)))
  (y2 : S.FD.obj (Discrete.mk c2)) :
  fromPairTContr (x1 ⊗t[k] x2) (y1 ⊗t[k] y2) =
```

```
(S.contr.app (Discrete.mk (c))) (x2 ⊗t[k] y1) • fromPairT (x1 ⊗t[k] y2)
```

lemma fromPairT_contr_fromPairT_eq_fromPairTContr_tmul

[\[source\]](#)

```
lemma fromPairT_contr_fromPairT_eq_fromPairTContr_tmul (c c1 c2 : S.C)
  (x1 : (S.FD.obj (Discrete.mk c1)).V)
  (x2 : (S.FD.obj (Discrete.mk c)).V)
  (y1 : (S.FD.obj (Discrete.mk (S.τ c))).V)
  (y2 : (S.FD.obj (Discrete.mk c2)).V) :
  contrT 2 1 2 (by simp; rfl)
    (prodT (fromPairT (x1 ⊗t[k] x2)) (fromPairT (y1 ⊗t[k] y2))) =
  permT id (by simp; intro i; fin_cases i <=> rfl)
    (fromPairTContr (x1 ⊗t[k] x2) (y1 ⊗t[k] y2))
```

lemma fromPairT_contr_fromPairT_eq_fromPairTContr

[\[source\]](#)

```
lemma fromPairT_contr_fromPairT_eq_fromPairTContr (c c1 c2 : S.C)
  (x : (S.FD.obj (Discrete.mk c1)).V ⊗t[k] (S.FD.obj (Discrete.mk c)).V)
  (y : (S.FD.obj (Discrete.mk (S.τ c))).V ⊗t[k] (S.FD.obj (Discrete.mk c2)).V) :
  contrT 2 1 2 (by simp; rfl)
    (prodT (fromPairT x) (fromPairT y)) =
  permT id (by simp; intro i; fin_cases i <=> rfl) (fromPairTContr x y)
```

lemma fromPairT_basis_repr

[\[source\]](#)

```
lemma fromPairT_basis_repr {c c1 : S.C}
  (x : (S.FD.obj (Discrete.mk c)).V ⊗t[k] (S.FD.obj (Discrete.mk c1)).V)
  (b : ComponentIdx ![c, c1]) :
  (basis ![c, c1]).repr (fromPairT x) b =
  (Basis.tensorProduct (S.basis c) (S.basis c1)).repr x (b 0, b 1)
```

lemma fromPairT_apply_basis_repr

[\[source\]](#)

```
lemma fromPairT_apply_basis_repr {c c1 : S.C}
  (b0 : Fin (S.repDim c)) (b1 : Fin (S.repDim c1)) :
  fromPairT (S.basis c b0 ⊗t[k] S.basis c1 b1) =
  Tensor.basis ![c, c1] (fun | 0 => b0 | 1 => b1)
```

def fromConstPair

[\[source\]](#)

A constant two tensor (e.g. metric and unit).

```
noncomputable def fromConstPair {c1 c2 : S.C}
  (v : ⋈_ (Rep k G) → S.FD.obj (Discrete.mk c1) ⊗ S.FD.obj (Discrete.mk c2)) :
  S.Tensor ![c1, c2]
```

lemma actionT_fromConstPair

[\[source\]](#)

Tensors formed by fromConstPair are invariant under the group action.

```
@[simp]
lemma actionT_fromConstPair {c1 c2 : S.C}
  (v : ⋈_ (Rep k G) → S.FD.obj (Discrete.mk c1) ⊗ S.FD.obj (Discrete.mk c2))
  (g : G) : g • fromConstPair v = fromConstPair v
```

lemma fromConstPair_whiskerLeft[\[source\]](#)

```
@[simp]
lemma fromConstPair_whiskerLeft {c1 c2 c2' : S.C} (h : c2 = c2')
  (v :  $\mathbb{K}_{-}$  (Rep k G)  $\longrightarrow$  S.FD.obj (Discrete.mk c1)  $\otimes$  S.FD.obj (Discrete.mk c2)) :
  fromConstPair (v  $\gg$ 
    ((S.FD.obj ({ as := c1 } : Discrete S.C)  $\triangleleft$  S.FD.map (Discrete.eqToHom (h)))) =
    permT id (And.intro (Function.bijective_id) (by simp [h])) (fromConstPair v)
```

lemma fromConstPair_braid[\[source\]](#)

```
@[simp]
lemma fromConstPair_braid {c1 c2 : S.C}
  (v :  $\mathbb{K}_{-}$  (Rep k G)  $\longrightarrow$  S.FD.obj (Discrete.mk c1)  $\otimes$  S.FD.obj (Discrete.mk c2)) :
  fromConstPair (v  $\gg$  ( $\beta_{-}$   $_{-}$ ).hom) =
  permT ![1, 0] (And.intro (by decide) (fun i => by fin_cases i <=> simp))
  (fromConstPair v)
```

def fromTripleT[\[source\]](#)

The construction of a tensor with two indices from the tensor product (S.FD.obj (Discrete.mk c1)).V $\otimes$ [k] (S.FD.obj (Discrete.mk c2)).V defined categorically.

```
noncomputable def fromTripleT {c1 c2 c3 : S.C} :
  (S.FD.obj (Discrete.mk c1)).V  $\otimes$ [k] (S.FD.obj (Discrete.mk c2)).V
   $\otimes$ [k] (S.FD.obj (Discrete.mk c3)).V  $\rightarrow_l$ [k] S.Tensor ![c1, c2, c3] where
```

lemma fromTripleT_tmul[\[source\]](#)

```
lemma fromTripleT_tmul {c1 c2 c3 : S.C} (x : S.FD.obj (Discrete.mk c1))
  (y : S.FD.obj (Discrete.mk c2)) (z : S.FD.obj (Discrete.mk c3)) :
  fromTripleT (x  $\otimes_t$ [k] y  $\otimes_t$ [k] z) =
  permT id (And.intro Function.bijective_id (fun i => by fin_cases i <=> rfl))
  (prodT (fromSingleT x) (prodT (fromSingleT y) (fromSingleT z)))
```

lemma actionT_fromTripleT[\[source\]](#)

```
lemma actionT_fromTripleT {c1 c2 c3 : S.C}
  (x : (S.FD.obj (Discrete.mk c1)).V  $\otimes$ [k] (S.FD.obj (Discrete.mk c2)).V
     $\otimes$ [k] (S.FD.obj (Discrete.mk c3)).V) (g : G) :
  g • fromTripleT x = fromTripleT (TensorProduct.map ((S.FD.obj (Discrete.mk c1)).p g)
    (TensorProduct.map ((S.FD.obj (Discrete.mk c2)).p g)
    ((S.FD.obj (Discrete.mk c3)).p g)) x)
```

lemma fromTripleT_basis_repr[\[source\]](#)

```
lemma fromTripleT_basis_repr {c c1 c2 : S.C}
  (x : (S.FD.obj (Discrete.mk c)).V  $\otimes$ [k] (S.FD.obj (Discrete.mk c1)).V
     $\otimes$ [k] (S.FD.obj (Discrete.mk c2)).V)
  (b : ComponentIdx ![c, c1, c2]) :
  (basis ![c, c1, c2]).repr (fromTripleT x) b =
  (Basis.tensorProduct (S.basis c) (Basis.tensorProduct (S.basis c1) (S.basis c2))).repr x
  (b 0, b 1, b 2)
```

lemma fromTripleT_apply_basis[\[source\]](#)

```

lemma fromTripleT_apply_basis {c c1 c2 : S.C}
  (b0 : Fin (S.repDim c)) (b1 : Fin (S.repDim c1))
  (b2 : Fin (S.repDim c2)) :
  fromTripleT (S.basis c b0 @τ[k] S.basis c1 b1 @τ[k] S.basis c2 b2) =
  Tensor.basis ![c, c1, c2] (fun | 0 => b0 | 1 => b1 | 2 => b2)

```

def fromConstTriple[\[source\]](#)

A constant three tensor (e.g. the Pauli matrices).

```

noncomputable def fromConstTriple {c1 c2 c3 : S.C}
  (v :  $\mathbb{K}_-$  (Rep k G)  $\longrightarrow$  S.FD.obj (Discrete.mk c1)  $\otimes$  S.FD.obj (Discrete.mk c2)  $\otimes$ 
    S.FD.obj (Discrete.mk c3)) :
  S.Tensor ![c1, c2, c3]

```

lemma actionT_fromConstTriple[\[source\]](#)

Tensors formed by fromConstPair are invariant under the group action.

```

@[simp]
lemma actionT_fromConstTriple {c1 c2 c3 : S.C}
  (v :  $\mathbb{K}_-$  (Rep k G)  $\longrightarrow$  S.FD.obj (Discrete.mk c1)  $\otimes$  S.FD.obj (Discrete.mk c2)  $\otimes$ 
    S.FD.obj (Discrete.mk c3))
  (g : G) : g • fromConstTriple v = fromConstTriple v

```

def fromConst[\[source\]](#)

A general constant node.

```

def fromConst {n :  $\mathbb{N}$ } {c : Fin n  $\rightarrow$  S.C} (T :  $\mathbb{K}_-$  (Rep k G)  $\longrightarrow$  S.F.obj (OverColor.mk c)) :
  Tensor S c

```

lemma actionT_fromConst[\[source\]](#)

Tensors formed by fromConst are invariant under the group action.

```

@[simp]
lemma actionT_fromConst {n :  $\mathbb{N}$ } {c : Fin n  $\rightarrow$  S.C} (T :  $\mathbb{K}_-$  (Rep k G)  $\longrightarrow$  S.F.obj (OverColor.mk
  c))
  (g : G) : g • fromConst T = fromConst T

```

2.45 PhysLean.Relativity.Tensors.Contraction.Basic

[PhysLean/Relativity/Tensors/Contraction/Basic.lean](#) on GitHub.

lemma contrT_decide[\[source\]](#)

```

lemma contrT_decide {n :  $\mathbb{N}$ } {c : Fin (n + 1 + 1)  $\rightarrow$  S.C} {i j : Fin (n + 1 + 1)}
  (hx : S.τ (c i) = c j) (hij : i  $\neq$  j := by decide) :
  i  $\neq$  j  $\wedge$  S.τ (c i) = c j

```

lemma contrT_congr[\[source\]](#)

```

lemma contrT_congr {n : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {i j : Fin (n + 1 + 1)} {hij : i ≠ j ∧ S.τ (c i) = c j}
  (i' j' : Fin (n + 1 + 1)) (t : S.Tensor c)
  (hii' : i = i' := by decide)
  (hjj' : j = j' := by decide) :
  contrT n i j hij t = permT id (And.intro (Function.bijective_id) (by subst hii' hjj'; simp)
  )
  (contrT n i' j' (by subst hii' hjj'; exact hij) t)

```

2.46 PhysLean.Relativity.Tensors.Contraction.Basis

[PhysLean/Relativity/Tensors/Contraction/Basis.lean](#) on GitHub.

def dropPair[\[source\]](#)

The ComponentIdx obtained by dropping two components.

```

def dropPair {n : ℕ} {c : Fin (n + 1 + 1) → S.C}
  (i j : Fin (n + 1 + 1)) (b : ComponentIdx c) :
  ComponentIdx (c ∘ Pure.dropPairEmb i j)

```

def DropPairSection[\[source\]](#)

Given a coordinate parameter $b : \Pi k, \text{Fin } (S.\text{repDim } ((c \circ i.\text{succAbove} \circ j.\text{succAbove}) k))$, the coordinate parameter $\Pi k, \text{Fin } (S.\text{repDim } (c k))$ which map down to b .

```

def DropPairSection {n : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {i : Fin (n + 1 + 1)} {j : Fin (n + 1 + 1)}
  (b : ComponentIdx (c ∘ Pure.dropPairEmb i j)) :
  Finset (ComponentIdx c)

```

lemma mem_iff_apply_dropPairEmb_eq[\[source\]](#)

```

lemma mem_iff_apply_dropPairEmb_eq {n : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {i j : Fin (n + 1 + 1)}
  (b : ComponentIdx (c ∘ Pure.dropPairEmb i j))
  (b' : ComponentIdx c) :
  b' ∈ DropPairSection b ↔
  ∀ m, b' (Pure.dropPairEmb i j m) = b m

```

lemma mem_self_of_dropPair[\[source\]](#)

```

@[simp]
lemma mem_self_of_dropPair {n : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {i j : Fin (n + 1 + 1)}
  (b : ComponentIdx (c)) :
  b ∈ DropPairSection (b.dropPair i j)

```

def offFin[\[source\]](#)

Given a b in $\text{ComponentIdx } (c \circ \text{Pure.dropPairEmb } i j)$ and an x in $\text{Fin } (S.\text{repDim } (c i)) \times \text{Fin } (S.\text{repDim } (c j))$, the corresponding coordinate parameter in $\text{ComponentIdx } c$.

```
def ofFin {n : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c ∘ Pure.dropPairEmb i j))
  (x : Fin (S.repDim (c i)) × Fin (S.repDim (c j))) :
  ComponentIdx c
```

lemma ofFin_apply_fst[\[source\]](#)

```
@[simp]
lemma ofFin_apply_fst {n : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c ∘ Pure.dropPairEmb i j))
  (x : Fin (S.repDim (c i)) × Fin (S.repDim (c j))) :
  ofFin hij b x i = x.1
```

lemma ofFin_apply_snd[\[source\]](#)

```
@[simp]
lemma ofFin_apply_snd {n : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c ∘ Pure.dropPairEmb i j))
  (x : Fin (S.repDim (c i)) × Fin (S.repDim (c j))) :
  ofFin hij b x j = x.2
```

lemma ofFin_mem_dropPairEmbSection[\[source\]](#)

```
lemma ofFin_mem_dropPairEmbSection {n : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c ∘ Pure.dropPairEmb i j))
  (x : Fin (S.repDim (c i)) × Fin (S.repDim (c j))) :
  ofFin hij b x ∈ DropPairSection b
```

def ofFinEquiv[\[source\]](#)

The equivalence between ContrSection b and Fin (S.repDim (c i)) × Fin (S.repDim (c (i.succAbove j))).

```
def ofFinEquiv {n : ℕ} {c : Fin n.succ.succ → S.C}
  {i j : Fin (n + 1 + 1)} (hij : i ≠ j)
  (b : ComponentIdx (c ∘ Pure.dropPairEmb i j)) :
  Fin (S.repDim (c i)) × Fin (S.repDim (c j)) ≈ DropPairSection b where
```

lemma ofFinEquiv_apply_fst[\[source\]](#)

```
@[simp]
lemma ofFinEquiv_apply_fst {n : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c ∘ Pure.dropPairEmb i j))
  (x : Fin (S.repDim (c i)) × Fin (S.repDim (c j))) :
  (ofFinEquiv hij b x).1 i = x.1
```

lemma ofFinEquiv_apply_snd[\[source\]](#)

```
@[simp]
lemma ofFinEquiv_apply_snd {n : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c ∘ Pure.dropPairEmb i j))
  (x : Fin (S.repDim (c i)) × Fin (S.repDim (c j))) :
  (ofFinEquiv hij b x).1 j = x.2
```

lemma dropPair_basisVector[\[source\]](#)

```

lemma Pure.dropPair_basisVector {n : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx c) :
  Pure.dropPair i j hij (basisVector c b) =
  basisVector (c ∘ Pure.dropPairEmb i j) fun m => b (dropPairEmb i j m)

```

lemma contrT_basis_repr_apply[\[source\]](#)

```

lemma contrT_basis_repr_apply {n : ℕ} {c : Fin (n + 1 + 1) → S.C} {i j : Fin (n + 1 + 1)}
  (h : i ≠ j ∧ S.τ (c i) = c j) (t : Tensor S c)
  (b : ComponentIdx (c ∘ Pure.dropPairEmb i j)) :
  (basis (c ∘ Pure.dropPairEmb i j)).repr (contrT n i j h t) b =
  ∑ (b' : DropPairSection b), (basis c).repr t b'.1 *
  S.castToField ((S.contr.app (Discrete.mk (c i))).hom
  (S.basis (c i) (b'.1 i) ⊗τ[k] S.basis (S.τ (c i)) (Fin.cast (by rw [h.2]) (b'.1 j)))))

```

lemma contrT_basis_repr_apply_eq_sum_fin[\[source\]](#)

```

lemma contrT_basis_repr_apply_eq_sum_fin {n : ℕ} {c : Fin (n + 1 + 1) → S.C} {i j : Fin (n + 1 + 1)}
  (h : i ≠ j ∧ S.τ (c i) = c j) (t : Tensor S c)
  (b : ComponentIdx (c ∘ Pure.dropPairEmb i j)) :
  (basis (c ∘ Pure.dropPairEmb i j)).repr (contrT n i j h t) b =
  ∑ (x1 : Fin (S.repDim (c i))), ∑ (x2 : Fin (S.repDim (c j))),
  (basis c).repr t (DropPairSection.ofFinEquiv h.1 b (x1, x2)).1 *
  S.castToField ((S.contr.app (Discrete.mk (c i))).hom
  (S.basis (c i) x1 ⊗τ[k] S.basis (S.τ (c i)) (Fin.cast (by rw [h.2]) x2)))

```

2.47 PhysLean.Relativity.Tensors.Contraction.Products

[PhysLean/Relativity/Tensors/Contraction/Products.lean](#) on GitHub.

lemma contrT_prodT_snd[\[source\]](#)

```

lemma contrT_prodT_snd {n n1 : ℕ} {c : Fin (n + 1 + 1) → S.C}
  {c1 : Fin n1 → S.C}
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j ∧ S.τ (c i) = c j)
  (t : Tensor S c) (t1 : Tensor S c1) :
  (contrT _
    (finSumFinEquiv (m := n1) (Sum.inr i))
    (finSumFinEquiv (m := n1) (Sum.inr j))
    (by simp [hij, - finSumFinEquiv_apply_right, finSumFinEquiv.injective.eq_iff]) <|
  prodT t1 t) =
  ((permT id (PermCond.on_id_symm (Pure.dropPairEmb_permCond_prod i j hij))) <|
  (prodT t1 (contrT n i j hij t)))

```

2.48 PhysLean.Relativity.Tensors.Contraction.Pure

[PhysLean/Relativity/Tensors/Contraction/Pure.lean](#) on GitHub.

lemma dropPairEmb_self_apply[\[source\]](#)

```

lemma dropPairEmb_self_apply (i : Fin (n + 1 + 1)) (m : Fin n) :
  dropPairEmb i i m = if m.1 < i.1 then ⟨m.1, by omega⟩ else ⟨m.1 + 2, by omega⟩

```

lemma dropPairEmb_eq_succAbove_succAbove[\[source\]](#)

```
lemma dropPairEmb_eq_succAbove_succAbove (i : Fin (n + 1 + 1)) (j : Fin (n + 1)) :
  dropPairEmb i (i.succAbove j) = i.succAbove ∘ j.succAbove
```

lemma dropPairEmb_injective[\[source\]](#)

```
lemma dropPairEmb_injective {n : ℕ}
  (i j : Fin (n + 1 + 1)) : Function.Injective (dropPairEmb i j)
```

lemma dropPairEmb_eq_iff_eq[\[source\]](#)

```
@[simp]
lemma dropPairEmb_eq_iff_eq {n : ℕ}
  (i j : Fin (n + 1 + 1)) (m1 m2 : Fin n) :
  dropPairEmb i j m1 = dropPairEmb i j m2 ↔ m1 = m2
```

lemma dropPairEmb_leq_iff_leq[\[source\]](#)

```
@[simp]
lemma dropPairEmb_leq_iff_leq {n : ℕ}
  (i j : Fin (n + 1 + 1)) (m1 m2 : Fin n) :
  dropPairEmb i j m1 ≤ dropPairEmb i j m2 ↔ m1 ≤ m2
```

lemma dropPairEmb_lt_iff_lt[\[source\]](#)

```
@[simp]
lemma dropPairEmb_lt_iff_lt {n : ℕ}
  (i j : Fin (n + 1 + 1)) (m1 m2 : Fin n) :
  dropPairEmb i j m1 < dropPairEmb i j m2 ↔ m1 < m2
```

lemma dropPairEmb_monotone[\[source\]](#)

```
@[simp]
lemma dropPairEmb_monotone {n : ℕ} (i j : Fin (n + 1 + 1)) :
  Monotone (dropPairEmb i j)
```

lemma dropPairEmb_eq_orderEmb0fFin[\[source\]](#)

```
lemma dropPairEmb_eq_orderEmb0fFin {n : ℕ}
  (i j : Fin (n + 1 + 1)) (hij : i ≠ j) :
  dropPairEmb i j = (Finset.orderEmb0fFin {i, j})c
  (by rw [Finset.card_compl]; simp [Finset.card_pair hij]))
```

lemma dropPairEmb_neq_fst[\[source\]](#)

```
@[simp]
lemma dropPairEmb_neq_fst (i j : Fin (n + 1 + 1)) (m : Fin n) :
  ¬ dropPairEmb i j m = i
```

lemma dropPairEmb_neq_snd[\[source\]](#)

```
@[simp]
lemma dropPairEmb_neq_snd (i j : Fin (n + 1 + 1)) (m : Fin n) :
  ¬ dropPairEmb i j m = j
```

lemma dropPairEmbPre_eq_orderIsoOfFin[\[source\]](#)

```
lemma dropPairEmbPre_eq_orderIsoOfFin (i j : Fin (n + 1 + 1)) (hij : i ≠ j) (m : Fin (n + 1 + 1))
  (hm : m ≠ i ∧ m ≠ j) :
  dropPairEmbPre i j hij m hm =
  (Finset.orderIsoOfFin {i, j}c (by rw [Finset.card_compl]; simp [Finset.card_pair hij])).
  symm
  ⟨m, by simp [hm]⟩
```

lemma dropPairEmbPre_dropPairEmb[\[source\]](#)

```
@[simp]
lemma dropPairEmbPre_dropPairEmb (i j : Fin (n + 1 + 1)) (hij : i ≠ j)
  (m : Fin n) :
  dropPairEmbPre i j hij (dropPairEmb i j m) (by simp) = m
```

lemma dropPairOfMap_id[\[source\]](#)

```
@[simp]
lemma dropPairOfMap_id {n1 : ℕ} (i j : Fin (n1 + 1 + 1)) (hij : i ≠ j) :
  dropPairOfMap i j hij id (Function.bijective_id) = id
```

2.49 PhysLean.Relativity.Tensors.Dual

[PhysLean/Relativity/Tensors/Dual.lean](#) on GitHub.

def fromDualMap[\[source\]](#)

The linear map taking a tensor based on the color $S.\tau c$ to a tensor based on the color c , defined by contraction with the metric tensor.

```
noncomputable def fromDualMap {c : S.C} : S.Tensor ![S.τ c] →l[k] S.Tensor ![c] where
```

lemma fromDualMap_apply[\[source\]](#)

```
lemma fromDualMap_apply {c : S.C} (t : S.Tensor ![S.τ c]) :
  fromDualMap t = permT id (by simp; intro i; fin_cases i; rfl)
  (contrT 1 1 2 (by simp; rfl) (prodT (metricTensor c) t))
```

def toDualMap[\[source\]](#)

The linear map taking a tensor based on the color c to a tensor based on the color $S.\tau c$, defined by contraction with the metric tensor.

```
noncomputable def toDualMap {c : S.C} : S.Tensor ![c] →l[k] S.Tensor ![S.τ c] where
```

lemma toDualMap_apply[\[source\]](#)

```

lemma toDualMap_apply {c : S.C} (t : S.Tensor ![c]) :
  toDualMap t = permT id (by
    simp; intro i; fin_cases i; rfl) (contrT 1 1 2 (by
      change _  $\wedge$  S. $\tau$  (S. $\tau$  c) = c
      simp) (prodT (metricTensor (S. $\tau$  c)) t))

```

lemma toDualMap_fromDualMap[\[source\]](#)

```

@[simp]
lemma toDualMap_fromDualMap {c : S.C} (t : S.Tensor ![S. $\tau$  c]) :
  toDualMap (fromDualMap t) = t

```

lemma fromDualMap_eq_permT_toDualMap[\[source\]](#)

```

lemma fromDualMap_eq_permT_toDualMap {c : S.C} (t : S.Tensor ![S. $\tau$  c]) :
  fromDualMap t = permT id (by simp) (toDualMap t)

```

lemma toDualMap_eq_permT_fromDualMap[\[source\]](#)

```

lemma toDualMap_eq_permT_fromDualMap {c : S.C} (t : S.Tensor ![c]) :
  toDualMap t = (fromDualMap (permT id (by simp) t))

```

lemma fromDualMap_toDualMap[\[source\]](#)

```

@[simp]
lemma fromDualMap_toDualMap {c : S.C} (t : S.Tensor ![c]) :
  fromDualMap (toDualMap t) = t

```

def toDual[\[source\]](#)

The linear equivalence between $S.\text{Tensor } ![c]$ and $S.\text{Tensor } ![S.\tau c]$ formed by contracting with metric tensors.

```

noncomputable def toDual {c : S.C} : S.Tensor ![c]  $\approx_l[k]$  S.Tensor ![S. $\tau$  c]

```

lemma toDual_equivariant[\[source\]](#)

```

lemma toDual_equivariant {c : S.C} (g : G) (t : S.Tensor ![c]) :
  toDual (g • t) = g • toDual t

```

lemma repDim_ τ [\[source\]](#)

```

@[simp]
lemma repDim_ $\tau$  {c : S.C} [StrongRankCondition k] :
  S.repDim (S. $\tau$  c) = S.repDim c

```

2.50 PhysLean.Relativity.Tensors.Elab

[PhysLean/Relativity/Tensors/Elab.lean](#) on GitHub.

def indexExprIsNum[\[source\]](#)

Bool which is true if an index is a num.

```
def indexExprIsNum (stx : Syntax) : Bool
```

def indexToNum[\[source\]](#)

If an index is a num - the underlying natural number.

```
def indexToNum (stx : Syntax) : TermElabM Nat
```

def indexToIdent[\[source\]](#)

When an index is not a num, the corresponding ident.

```
def indexToIdent (stx : Syntax) : TermElabM Ident
```

def indexPosEq[\[source\]](#)

Takes a pair $a\ b : \mathbb{N} \times \text{TSyntax `indexExpr}$. If $a.1 < b.1$ and $a.2 = b.2$ then outputs some $(a.1, b.1)$, otherwise none.

```
def indexPosEq (a b : TSyntax `indexExpr × ℕ) : TermElabM (Option (ℕ × ℕ))
```

def indexToDual[\[source\]](#)

Bool which is true if an index is of the form $\tau(i)$ that is, to be dualized.

```
def indexToDual (stx : Syntax) : Bool
```

def evalAdjustPos[\[source\]](#)

Adjusts a list $\text{List } \mathbb{N}$ by subtracting from each natural number the number of elements before it in the list which are less than itself. This is used to form a list of pairs which can be used for evaluating indices.

```
def evalAdjustPos (l : List ℕ) : List ℕ
```

def getEvalPos[\[source\]](#)

For list of indexExpr e.g. $[\alpha, 3, \beta, 2, \gamma]$, `getEvalPos` returns a list of pairs $\mathbb{N} \times \mathbb{N}$ related to indices which are numbers. The second element of each pair is the number corresponding to that index. The first element is the position of that number in the list of indices when all other numbered indices before it are removed. Thus for the example given `getEvalPos` outputs $[(1, 3), (2, 2)]$.

```
def getEvalPos (ind : List (TSyntax `indexExpr)) : TermElabM (List (ℕ × ℕ))
```

def getContrPos[\[source\]](#)

For list of indexExpr e.g. $[\alpha, 3, \beta, \alpha, 2, \gamma]$, `getContrPos` first removes all indices which are numbers (e.g. $[\alpha, \beta, \alpha, \gamma]$). It then outputs pairs (a, b) in $\mathbb{N} \times \mathbb{N}$ of

positions of this list with $a < b$ such that the index at a is equal to the index at b . It checks whether or not an element is contracted more than once.

```
def getContrPos (ind : List (TSyntax `indexExpr)) : TermElabM (List (ℕ × ℕ))
```

def withoutContrEval

[\[source\]](#)

The list of indices after contraction or evaluation.

```
def withoutContrEval (ind : List (TSyntax `indexExpr)) : TermElabM (List (TSyntax `indexExpr))
```

def toPairs

[\[source\]](#)

Takes a list and puts consecutive elements into pairs. e.g. $[0, 1, 2, 3]$ becomes $[(0, 1), (2, 3)]$.

```
def toPairs (l : List ℕ) : List (ℕ × ℕ)
```

def contrListAdjust

[\[source\]](#)

Adjusts a list $List (ℕ × ℕ)$ by subtracting from each natural number the number of elements before it in the list which are less than itself. This is used to form a list of pairs which can be used for contracting indices.

```
def contrListAdjust (l : List (ℕ × ℕ)) : List (ℕ × ℕ)
```

def getPermutation

[\[source\]](#)

Given two lists of indices, all of which are indent, returns the $List (ℕ)$ representing the how one list permutes into the other.

```
def getPermutation (l1 l2 : List (TSyntax `indexExpr)) : TermElabM (List (ℕ))
```

def stringToTerm

[\[source\]](#)

The construction of an expression corresponding to the type of a given string once parsed.

```
def stringToTerm (str : String) : TermElabM Term
```

def getPermutationTerm

[\[source\]](#)

Given two lists of indices returns the permutation between them based on `finMapToEquiv`.

```
def getPermutationTerm (l1 l2 : List (TSyntax `indexExpr)) : TermElabM Term
```

def getNumIndicesExact

[\[source\]](#)

For syntax of the form T where T is `S.F.obj (OverColor.mk c)` this returns the value of `TensorSpecies.numIndices T`. That is, the exact number of indices associated with that tensor.

```
def getNumIndicesExact (stx : Syntax) : TermElabM ℕ
```

def getAllIndices[\[source\]](#)

For syntax of the form $T \mid \alpha \beta \ 2 \ \beta$, `getAllIndices` returns a list $[\alpha, \beta, 2, \beta]$ of all `indexExpr`.

```
def getAllIndices (stx : Syntax) : TermElabM (List (TSyntax `indexExpr))
```

def getProdIndices[\[source\]](#)

The function `getProdIndices` is defined for the following syntax: 1. For e.g. $T \mid \alpha \beta \ 2 \ \beta$, it returns all uncontracted and unevaluated indices e.g. $[\alpha]$ 2. For e.g. $T1 \mid \alpha \beta \ 2 \ \beta \otimes T2 \mid \alpha \gamma \ \delta \ \delta$ it returns all unevaluated indices which are not contracted in either tensor e.g. $[\alpha, \alpha, \gamma]$. 3. For e.g. $(T1 \mid \alpha \beta \ 2 \ \beta \otimes T2 \mid \alpha \gamma \ \delta \ \delta) \otimes T3 \mid \gamma$ it does 2 recursively e.g. $[\gamma, \gamma]$

```
partial def getProdIndices (stx : Syntax) : TermElabM (List (TSyntax `indexExpr))
```

def getIndicesFull[\[source\]](#)

Returns the remaining indices of a tensor expression after contraction and evaluation. Thus every index in the output of `getIndicesFull` is ident and there are no duplicates. Examples are: 1. $T \mid \alpha \beta \ 2 \ \beta$ gives $[\alpha]$ 2. $T1 \mid \alpha \beta \ 2 \ \beta \otimes T2 \mid \alpha \gamma \ \delta \ \delta$ gives $[\gamma]$ 3. $(T1 \mid \alpha \beta \ 2 \ \beta \otimes T2 \mid \alpha \gamma \ \delta \ \delta) \otimes T3 \mid \gamma$ gives $[]$ 4. $T1 \mid \alpha \beta \ 2 \ \beta + T2 \mid \alpha \ 4 \ \delta \ \delta$ gives $[\alpha]$

```
partial def getIndicesFull (stx : Syntax) : TermElabM (List (TSyntax `indexExpr))
```

def getIndicesLeft[\[source\]](#)

Gets the indices associated with the LHS of an addition.

```
def getIndicesLeft (stx : Syntax) : TermElabM (List (TSyntax `indexExpr))
```

def getIndicesRight[\[source\]](#)

Gets the indices associated with the RHS of an addition.

```
def getIndicesRight (stx : Syntax) : TermElabM (List (TSyntax `indexExpr))
```

def getIndicesLeftEq[\[source\]](#)

Gets the indices associated with the LHS of an equality.

```
def getIndicesLeftEq (stx : Syntax) : TermElabM (List (TSyntax `indexExpr))
```

def getIndicesRightEq[\[source\]](#)

Gets the indices associated with the RHS of an equality.

```
def getIndicesRightEq (stx : Syntax) : TermElabM (List (TSyntax `indexExpr))
```

def nodeTermMap[\[source\]](#)

For a term of the form T where T is $S.F.obj$ (`OverColor.mk c`), `tensorTermToTensorTree` returns the term corresponding to the `tensorNode T`

```
def nodeTermMap (T : Term) : Term
```

def evalTermMap[\[source\]](#)

Given a list l of pairs $\mathbb{N} \times \mathbb{N}$ and a term T corresponding to a tensor tree, for each (a, b) in l , `evalSyntax` applies `TensorTree.eval a b` to T recursively. Here a is the position of the index to be evaluated and b is the value it is evaluated to.

For example, if l is $[(1, 2), (1, 4)]$ and T is a tensor tree then `evalSyntax l T` is `TensorTree.eval 1 4 (TensorTree.eval 1 2 T)`.

The list l is expected to be the output of `getEvalPos`.

```
def evalTermMap (l : List (N × N)) (T : Term) : Term
```

def contrTermMap[\[source\]](#)

For each element of $l : \text{List } (\mathbb{N} \times \mathbb{N})$ applies `TensorTree.contr` to the given term.

```
def contrTermMap (n : N) (l : List (N × N)) (T : Term) : Term
```

def prodTermMap[\[source\]](#)

The syntax associated with a product of tensors.

```
def prodTermMap (T1 T2 : Term) : Term
```

def negTermMap[\[source\]](#)

The syntax associated with negation of tensors.

```
def negTermMap (T1 : Term) : Term
```

def smulTermMap[\[source\]](#)

The syntax associated with the scalar multiplication of tensors.

```
def smulTermMap (c T : Term) : Term
```

def actionTermMap[\[source\]](#)

The syntax associated with the group action of tensors.

```
def actionTermMap (c T : Term) : Term
```

def addTermMap[\[source\]](#)

The syntax for a equality of tensor trees.

```
def addTermMap (P : Term) (T1 T2 : Term) : TermElabM Term
```

def equalTermMap[\[source\]](#)

The syntax for a equality of tensor trees.

```
def equalTermMap (P : Term) (T1 T2 : Term) : TermElabM Term
```

def syntaxFull[\[source\]](#)

Takes a syntax corresponding to a tensor expression and turns it into a term corresponding to a tensor tree.

```
partial def syntaxFull (stx : Syntax) : TermElabM Term
```

def elaborateTensorNode[\[source\]](#)

Takes a syntax corresponding to a tensor expression and turns it into an expression corresponding to a tensor tree.

```
def elaborateTensorNode (stx : Syntax) : TermElabM Expr
```

2.51 PhysLean.Relativity.Tensors.MetricTensor

[PhysLean/Relativity/Tensors/MetricTensor.lean](#) on GitHub.

lemma metricTensor_invariant[\[source\]](#)

```
@[simp]
lemma metricTensor_invariant {c : S.C} (g : G) :
  g • metricTensor c = metricTensor c
```

lemma permT_fromPairTContr_metric_metric[\[source\]](#)

```
lemma permT_fromPairTContr_metric_metric {c : S.C} :
  permT ![1, 0] (And.intro (by decide) (fun i => by fin_cases i <=> rfl))
  (fromPairTContr ((S.metric.app (Discrete.mk c)).hom (1 : k))
  ((S.metric.app (Discrete.mk (S.τ c))).hom (1 : k))) =
  (unitTensor c)
```

lemma fromPairTContr_metric_metric_eq_permT_unit[\[source\]](#)

```
lemma fromPairTContr_metric_metric_eq_permT_unit {c : S.C} :
  fromPairTContr ((S.metric.app (Discrete.mk c)).hom (1 : k))
  ((S.metric.app (Discrete.mk (S.τ c))).hom (1 : k)) =
  permT ![1, 0] (And.intro (by decide) (fun i => by fin_cases i <=> rfl))
  (unitTensor c)
```

lemma contrT_metricTensor_metricTensor[\[source\]](#)

The contraction of the metric tensor with its dual gives the unit tensor. This is the de-categorification of $S.contr_metric$.

```
@[simp]
lemma contrT_metricTensor_metricTensor {c : S.C} :
  contrT 2 1 2 (by simp; rfl) (prodT (metricTensor c) (metricTensor (S.τ c))) =
```

```
permT ![1, 0] (And.intro (by decide) (fun i => by fin_cases i <=> rfl))
(unitTensor c)
```

lemma contrT_metricTensor_metricTensor_eq_dual_unit

[\[source\]](#)

```
lemma contrT_metricTensor_metricTensor_eq_dual_unit {c : S.C} :
  contrT 2 1 2 (by simp; rfl) (prodT (metricTensor c) (metricTensor (S.τ c))) =
    permT ![0, 1] (And.intro (by decide) (fun i => by
      fin_cases i
      · change S.τ (S.τ c) = c
      · simp
      · rfl))
    (unitTensor (S.τ c))
```

lemma contrT_dual_metricTensor_metricTensor

[\[source\]](#)

```
@[simp]
lemma contrT_dual_metricTensor_metricTensor {c : S.C} :
  contrT 2 1 2 (by change _ ^ S.τ (S.τ c) = c; simp)
    (prodT (metricTensor (S.τ c)) (metricTensor c)) =
    permT id (by simp; intro i; fin_cases i <=> rfl) (unitTensor c)
```

2.52 PhysLean.Relativity.Tensors.OfInt

[PhysLean/Relativity/Tensors/OfInt.lean](#) on GitHub.

abbrev TensorInt

[\[source\]](#)

A tensor with integer components with respect to the basis.

```
abbrev TensorInt {n : ℕ} (c : Fin n → S.C)
```

def toTensor

[\[source\]](#)

The element of S .Tensor c created from a tensor TensorInt S c .

```
noncomputable def toTensor {n : ℕ} {c : Fin n → S.C} (f : TensorInt S c) :
  S.Tensor c
```

lemma basis_repr_apply

[\[source\]](#)

```
lemma basis_repr_apply {n : ℕ} {c : Fin n → S.C}
  (f : TensorInt S c) (b : ComponentIdx c) :
  (Tensor.basis c).repr (toTensor f) b = Int.cast (f b)
```

lemma basis_eq_tensorInt

[\[source\]](#)

```
lemma basis_eq_tensorInt {n : ℕ} {c : Fin n → S.C}
  (b : ComponentIdx c) :
  Tensor.basis c b = TensorInt.toTensor (fun b' => if b = b' then 1 else 0)
```

2.53 PhysLean.Relativity.Tensors.Product

[PhysLean/Relativity/Tensors/Product.lean](#) on GitHub.

def splitEquiv

[\[source\]](#)

The equivalence between $\text{ComponentIdx } (\text{Sum.elim } c \ c1 \circ \uparrow \text{finSumFinEquiv.symm})$ and $\text{ComponentIdx } c \times \text{ComponentIdx } c1$ formed by products.

```
def ComponentIdx.splitEquiv {n1 n2 : ℕ} {c : Fin n1 → S.C} {c1 : Fin n2 → S.C} :
  ComponentIdx (Sum.elim c c1 ∘ ↑finSumFinEquiv.symm) = ComponentIdx c × ComponentIdx c1
where
```

lemma prodLeftMap_id

[\[source\]](#)

@[simp]

```
lemma prodLeftMap_id {n2 n : ℕ} :
  prodLeftMap (n := n) n2 id = id
```

lemma prodRightMap_id

[\[source\]](#)

@[simp]

```
lemma prodRightMap_id {n2 n : ℕ} :
  prodRightMap (n := n) n2 id = id
```

lemma prodP_component

[\[source\]](#)

```
lemma Pure.prodP_component {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  (p : Pure S c) (p1 : Pure S c1)
  (b : ComponentIdx (Sum.elim c c1 ∘ ↑finSumFinEquiv.symm)) :
  (p.prodP p1).component b = p.component (ComponentIdx.splitEquiv b).1 *
  p1.component (ComponentIdx.splitEquiv b).2
```

lemma prodT_basis_repr_apply

[\[source\]](#)

```
lemma prodT_basis_repr_apply {n m : ℕ} {c : Fin n → S.C} {c1 : Fin m → S.C}
  (t : Tensor S c) (t1 : Tensor S c1)
  (b : ComponentIdx (Sum.elim c c1 ∘ ↑finSumFinEquiv.symm)) :
  (basis (Sum.elim c c1 ∘ ↑finSumFinEquiv.symm)).repr (prodT t t1) b =
  (basis c).repr t (ComponentIdx.splitEquiv b).1 *
  (basis c1).repr t1 (ComponentIdx.splitEquiv b).2
```

lemma prodP_zero_right_permCond

[\[source\]](#)

```
lemma Pure.prodP_zero_right_permCond {n} {c : Fin n → S.C}
  {c1 : Fin 0 → S.C} : PermCond c (Sum.elim c c1 ∘ ↑finSumFinEquiv.symm) id
```

lemma prodP_zero_right

[\[source\]](#)

```
lemma Pure.prodP_zero_right {n} {c : Fin n → S.C}
  {c1 : Fin 0 → S.C} (p : Pure S c) (p0 : Pure S c1) :
  prodP p p0 = permP id (prodP_zero_right_permCond) p
```

lemma prodT_default_right[\[source\]](#)

```

lemma prodT_default_right {n} {c : Fin n → S.C}
  {c1 : Fin 0 → S.C} (t : S.Tensor c) :
  prodT t (Pure.toTensor default : S.Tensor c1) =
  permT id (Pure.prodP_zero_right_permCond) t

```

2.54 PhysLean.Relativity.Tensors.RealTensor.Basic

[PhysLean/Relativity/Tensors/RealTensor/Basic.lean](#) on GitHub.

lemma contrT_basis_repr_apply_eq_dropPairSection[\[source\]](#)

```

lemma contrT_basis_repr_apply_eq_dropPairSection {n d: ℕ}
  {c : Fin (n + 1 + 1) → (realLorentzTensor d).C}
  {i j : Fin (n + 1 + 1)} (h : i ≠ j ∧ (realLorentzTensor d).τ (c i) = c j)
  (t : ℝT(d, c)) (b : ComponentIdx (c ∘ Pure.dropPairEmb i j)) :
  (basis (c ∘ Pure.dropPairEmb i j)).repr (contrT n i j h t) b =
  ∑ (x : b.DropPairSection),
  ((basis c).repr t x.1) *
  if (x.1 i).1 = (x.1 j).1 then 1 else 0

```

lemma contrT_basis_repr_apply_eq_fin[\[source\]](#)

```

lemma contrT_basis_repr_apply_eq_fin {n d: ℕ} {c : Fin (n + 1 + 1) → (realLorentzTensor d).C}
  {i j : Fin (n + 1 + 1)}
  {h : i ≠ j ∧ (realLorentzTensor d).τ (c i) = c j}
  (t : ℝT(d, c)) (b : ComponentIdx (c ∘ Pure.dropPairEmb i j)) :
  (basis (c ∘ Pure.dropPairEmb i j)).repr (contrT n i j h t) b =
  ∑ (x : Fin (1 + d)),
  ((basis c).repr t
    (DropPairSection.ofFinEquiv h.1 b (Fin.cast (by simp) x, Fin.cast (by simp) x)))

```

2.55 PhysLean.Relativity.Tensors.RealTensor.Metrics.Basic

[PhysLean/Relativity/Tensors/RealTensor/Metrics/Basic.lean](#) on GitHub.

lemma coMetric_eq_fromConstPair[\[source\]](#)

```

lemma coMetric_eq_fromConstPair {d : ℕ} :
  η' d = fromConstPair (Lorentz.preCoMetric d)

```

lemma contrMetric_eq_fromConstPair[\[source\]](#)

```

lemma contrMetric_eq_fromConstPair {d : ℕ} :
  η d = fromConstPair (Lorentz.preContrMetric d)

```

lemma coMetric_eq_fromPairT[\[source\]](#)

```

lemma coMetric_eq_fromPairT {d : ℕ} :
  η' d = fromPairT (Lorentz.preCoMetricVal d)

```

lemma contrMetric_eq_fromPairT[\[source\]](#)

```
lemma contrMetric_eq_fromPairT {d : ℕ} :
  η d = fromPairT (Lorentz.preContrMetricVal d)
```

lemma actionT_coMetric[\[source\]](#)

The tensor coMetric is invariant under the action of LorentzGroup d.

```
@[simp]
lemma actionT_coMetric {d : ℕ} (g : LorentzGroup d) :
  g • η' d = η' d
```

lemma actionT_contrMetric[\[source\]](#)

The tensor contrMetric is invariant under the action of LorentzGroup d.

```
@[simp]
lemma actionT_contrMetric {d} (g : LorentzGroup d) : g • η d = η d
```

2.56 PhysLean.Relativity.Tensors.RealTensor.ToComplex

[PhysLean/Relativity/Tensors/RealTensor/ToComplex.lean](#) on GitHub.

def colorToComplex[\[source\]](#)

The map from colors of real Lorentz tensors to complex Lorentz tensors.

```
def colorToComplex (c : realLorentzTensor.C) : complexLorentzTensor.Color
```

semiformal_result toComplex[\[source\]](#)

The semilinear map from real Lorentz tensors to complex Lorentz tensors.

Semiformal implementation note: Probably the easist way to define this is through basis.

```
semiformal_result "7RJQJ" toComplex (c : Fin n → realLorentzTensor.C) :
  ℝT(3, c) →sl [Complex.ofRealHom] ℂT(colorToComplex ∘ c)
```

informal_lemma toComplex_injective[\[source\]](#)

The map toComplex is injective.

```
informal_lemma toComplex_injective where
  deps := []
  tag := "7RK CJ"
```

informal_lemma toComplex_equivariant[\[source\]](#)

The map toComplex is equivariant.

```
informal_lemma toComplex_equivariant where
  deps := []
  tag := "7RKDY"
```

informal_lemma permT_toComplex[\[source\]](#)*The map toComplex commutes with permT.*

```
informal_lemma permT_toComplex where
  deps := [``permT]
  tag := "7RKA6"
```

informal_lemma prodT_toComplex[\[source\]](#)*The map toComplex commutes with prodT.*

```
informal_lemma prodT_toComplex where
  deps := [``prodT]
  tag := "7RKFF"
```

informal_lemma contrT_toComplex[\[source\]](#)*The map toComplex commutes with contrT.*

```
informal_lemma contrT_toComplex where
  deps := [``contrT]
  tag := "7RKFR"
```

informal_lemma evalT_toComplex[\[source\]](#)*The map toComplex commutes with evalT.*

```
informal_lemma evalT_toComplex where
  deps := [``evalT]
  tag := "7RKGK"
```

2.57 PhysLean.Relativity.Tensors.RealTensor.Vector.Basic[PhysLean/Relativity/Tensors/RealTensor/Vector/Basic.lean](#) on GitHub.**lemma toCoord_pure**[\[source\]](#)

```
lemma toCoord_pure {d : ℕ} (p : Pure (realLorentzTensor d) ![.up]) (i : Fin 1 ⊗ Fin d) :
  toCoord p.toTensor i = ((Lorentz.contrBasisFin d).repr (p 0)) (indexEquiv.symm i 0)
```

lemma toCoord_basis_apply[\[source\]](#)

```
lemma toCoord_basis_apply {d : ℕ} (μ : Fin (1 + d)) (v : Fin 1 ⊗ Fin d) :
  toCoord (Tensor.basis (S := realLorentzTensor d) ![Color.up] (fun _ => Fin.cast (by simp)
    μ)) v
  = (Finsupp.single (finSumFinEquiv.symm μ) 1) v
```

lemma basis_repr_apply[\[source\]](#)

```
lemma basis_repr_apply {d : ℕ} (p : Vector d)
  (b : ComponentIdx (S := realLorentzTensor d) ![Color.up]) :
  (Tensor.basis (S := realLorentzTensor d) ![Color.up]).repr p b =
  p (finSumFinEquiv.symm (b 0))
```

abbrev spatialPart[\[source\]](#)

Extract spatial components from a Lorentz vector, returning them as a vector in Euclidean space.

```
abbrev spatialPart {d : ℕ} (v : Vector d) : EuclideanSpace ℝ (Fin d)
```

lemma spatialPart_basis_natAdd[\[source\]](#)

```
@[simp]
lemma spatialPart_basis_natAdd {d : ℕ} (i : Fin d) (j : Fin d) :
  spatialPart (Tensor.basis (S := realLorentzTensor d) ![Color.up] (fun _ =>
    Fin.cast (by simp) (Fin.natAdd 1 i))) j =
    (Finsupp.single (Sum.inr i : Fin 1 ⊗ Fin d) 1) (Sum.inr j)
```

lemma spatialPart_basis_castAdd[\[source\]](#)

```
@[simp]
lemma spatialPart_basis_castAdd {d : ℕ} (i : Fin d) :
  spatialPart (Tensor.basis (S := realLorentzTensor d) ![Color.up] (fun _ =>
    Fin.cast (by simp) (Fin.castAdd d (0 : Fin 1)))) i = 0
```

abbrev timeComponent[\[source\]](#)

Extract time component from a Lorentz vector

```
abbrev timeComponent {d : ℕ} (v : Vector d) : ℝ
```

lemma timeComponent_basis_natAdd[\[source\]](#)

```
@[simp]
lemma timeComponent_basis_natAdd {d : ℕ} (i : Fin d) :
  timeComponent (Tensor.basis (S := realLorentzTensor d) ![Color.up] (fun _ =>
    Fin.cast (by simp) (Fin.natAdd 1 i))) = 0
```

lemma timeComponent_basis_castAdd[\[source\]](#)

```
@[simp]
lemma timeComponent_basis_castAdd {d : ℕ} :
  timeComponent (Tensor.basis (S := realLorentzTensor d) ![Color.up] (fun _ =>
    Fin.cast (by simp) (Fin.castAdd d (0 : Fin 1)))) = 1
```

2.58 PhysLean.Relativity.Tensors.RealTensor.Vector.Causality.Basic

[PhysLean/Relativity/Tensors/RealTensor/Vector/Causality/Basic.lean](#) on GitHub.

def isFutureDirected[\[source\]](#)

In Minkowski spacetime with $(+—)$ signature, we can define future-directed vectors as having positive time components (by convention)

```
def isFutureDirected {d : ℕ} (v : Vector d) : Prop
```

def isPastDirected[\[source\]](#)

In Minkowski spacetime with $(+ -)$ signature, we can define past-directed vectors as having negative time components (by convention)

```
def isPastDirected {d : ℕ} (v : Vector d) : Prop
```

2.59 PhysLean.Relativity.Tensors.RealTensor.Vector.Causality.LightLike

[PhysLean/Relativity/Tensors/RealTensor/Vector/Causality/LightLike.lean](#) on GitHub.

lemma causalCharacter_zero[\[source\]](#)

```
@[simp]
```

```
lemma causalCharacter_zero {d : ℕ} : causalCharacter (0 : Vector d) =  
  CausalCharacter.lightLike
```

lemma causallyPrecedes_refl[\[source\]](#)

Causally preceding is reflexive

```
@[simp]
```

```
lemma causallyPrecedes_refl {d : ℕ} (p : Vector d) : causallyPrecedes p p
```

lemma lightlike_eq_spatial_norm_of_eq_time[\[source\]](#)

For two lightlike vectors with equal time components, their spatial parts have equal Euclidean norms

```
lemma lightlike_eq_spatial_norm_of_eq_time {d : ℕ} {v w : Vector d}  
  (hv : causalCharacter v = .lightLike) (hw : causalCharacter w = .lightLike)  
  (h_time : timeComponent v = timeComponent w) :  
  ⟨⟨spatialPart v, spatialPart v⟩⟩_ℝ = ⟨⟨spatialPart w, spatialPart w⟩⟩_ℝ
```

lemma lightlike_spatial_parallel_implies_proportional[\[source\]](#)

If two lightlike vectors have parallel spatial components, their temporal components must also be proportional, which implies the entire vectors are proportional

```
lemma lightlike_spatial_parallel_implies_proportional {d : ℕ} {v w : Vector d}  
  (hv : causalCharacter v = .lightLike) (hw : causalCharacter w = .lightLike)  
  (h_spatial_parallel : ∃ (r : ℝ), v = r • w) :  
  ∃ (r : ℝ), |v (Sum.inl 0)| = |r| * |w (Sum.inl 0)|
```

2.60 PhysLean.Relativity.Tensors.RealTensor.Vector.Causality.TimeLike

[PhysLean/Relativity/Tensors/RealTensor/Vector/Causality/TimeLike.lean](#) on GitHub.

lemma timelike_neg_time_component_product[\[source\]](#)

For timelike vectors with negative time components, their time components multiply to give a positive number

```
@[simp]
```

```
lemma timelike_neg_time_component_product {d : ℕ} (v w : Vector d)
```

```
(hv_neg : v (Sum.inl 0) < 0) (hw_neg : w (Sum.inl 0) < 0) :
v (Sum.inl 0) * w (Sum.inl 0) > 0
```

lemma timelike_time_dominates_space[\[source\]](#)

For timeLike vectors in Minkowski space, the inner product of the spatial part is less than the square of the time component

```
lemma timelike_time_dominates_space {d : ℕ} {v : Vector d}
(hv : causalCharacter v = .timeLike) :
⟨⟨spatialPart v, spatialPart v⟩⟩_ℝ < (timeComponent v) * (timeComponent v)
```

lemma time_component_ne_zero_of_timelike[\[source\]](#)

For nonzero timelike vectors, the time component is nonzero

```
@[simp]
lemma time_component_ne_zero_of_timelike {d : ℕ} {v : Vector d}
(hv : causalCharacter v = .timeLike) :
v (Sum.inl 0) ≠ 0
```

lemma timelike_time_component_ne_zero[\[source\]](#)

For timelike vectors, the time component is nonzero

```
@[simp]
lemma timelike_time_component_ne_zero {d : ℕ} {v : Vector d}
(hv : causalCharacter v = .timeLike) :
timeComponent v ≠ 0
```

lemma timelike_iff_time_lt_space[\[source\]](#)

A vector is timelike if and only if its time component squared is less than the sum of its spatial components squared

```
lemma timelike_iff_time_lt_space {d : ℕ} {v : Vector d} :
causalCharacter v = .timeLike ↔
⟨⟨spatialPart v, spatialPart v⟩⟩_ℝ < v (Sum.inl 0) * v (Sum.inl 0)
```

lemma timeComponent_squared_pos_of_timelike[\[source\]](#)

Time component squared is positive for timelike vectors

```
@[simp]
lemma timeComponent_squared_pos_of_timelike {d : ℕ} {v : Vector d}
(hv : causalCharacter v = .timeLike) :
0 < (timeComponent v)^2
```

lemma timelike_spatial_lt_time_squared[\[source\]](#)

For timelike vectors, the spatial norm squared is strictly less than the time component squared

```

lemma timelike_spatial_lt_time_squared {d : ℕ} {v : Vector d}
  (hv : causalCharacter v = .timeLike) :
  ⟨⟨spatialPart v, spatialPart v⟩⟩_ℝ < (timeComponent v)^2

```

2.61 PhysLean.Relativity.Tensors.TensorSpecies.Basic

[PhysLean/Relativity/Tensors/TensorSpecies/Basic.lean](#) on GitHub.

lemma basis_congr

[\[source\]](#)

```

lemma basis_congr {c c1 : S.C} (h : c = c1) (i : Fin (S.repDim c)) :
  S.basis c i = S.FD.map (eqToHom (by simp [h])) (S.basis c1 (Fin.cast (by simp [h]) i))

```

lemma basis_congr_repr

[\[source\]](#)

```

lemma basis_congr_repr {c c1 : S.C} (h : c = c1) (i : Fin (S.repDim c))
  (t : S.FD.obj (Discrete.mk c)) :
  (S.basis c).repr t i = (S.basis c1).repr (S.FD.map (eqToHom (by simp [h])) t)
  (Fin.cast (by simp [h]) i)

```

2.62 PhysLean.Relativity.Tensors.UnitTensor

[PhysLean/Relativity/Tensors/UnitTensor.lean](#) on GitHub.

lemma unit_app_eq_dual_unit_app

[\[source\]](#)

```

lemma unit_app_eq_dual_unit_app (c : S.C) :
  (S.unit.app (Discrete.mk c)) = (S.unit.app ({ as := S.τ c }))) >>
  (β_ (S.FD.obj ({ as := S.τ (S.τ c) }))) (S.FD.obj ({ as := S.τ c }))) .hom >>
  ((S.FD.obj ({ as := S.τ c } : Discrete S.C) <Δ S.FD.map (Discrete.eqToHom (by simp))))

```

lemma unitTensor_eq_permT_dual

[\[source\]](#)

The unit tensor is symmetric on dualing the color.

```

lemma unitTensor_eq_permT_dual (c : S.C) :
  unitTensor c = permT ![1, 0] (And.intro (by decide) (fun i => by fin_cases i <=> simp))
  (unitTensor (S.τ c))

```

lemma dual_unitTensor_eq_permT_unitTensor

[\[source\]](#)

```

lemma dual_unitTensor_eq_permT_unitTensor (c : S.C) :
  unitTensor (S.τ c) = permT ![1, 0] (And.intro (by decide) (fun i => by fin_cases i <=> simp
  ))
  (unitTensor c)

```

lemma unit_fromSingleTContrFromPairT_eq_fromSingleT

[\[source\]](#)

```

lemma unit_fromSingleTContrFromPairT_eq_fromSingleT {c : S.C} (x : S.FD.obj (Discrete.mk c)) :
  fromSingleTContrFromPairT x ((S.unit.app (Discrete.mk c)).hom (1 : k)) =
  fromSingleT x

```

lemma contrT_single_unitTensor[\[source\]](#)

This lemma represents the de-categorification of $S.contr_unit$.

```
@[simp]
lemma contrT_single_unitTensor {c : S.C} (x : Tensor S ![c]) :
  contrT 1 0 1 (by simp; rfl) (prodT x (unitTensor c)) =
  permT id (by simp; intro i; fin_cases i; rfl) x
```

lemma contrT_unitTensor_dual_single[\[source\]](#)

```
lemma contrT_unitTensor_dual_single {c : S.C} (x : Tensor S ![S.τ c]) :
  contrT 1 1 2 (by simp; rfl) (prodT (unitTensor c) x) =
  permT id (by simp; intro i; fin_cases i; rfl) x
```

lemma unitTensor_invariant[\[source\]](#)

```
@[simp]
lemma unitTensor_invariant {c : S.C} (g : G) :
  g • unitTensor c = unitTensor c
```

2.63 PhysLean.StringTheory.FTheory.SU5U1.Charges

[PhysLean/StringTheory/FTheory/SU5U1/Charges.lean](#) on GitHub.

inductive CodimensionOneConfig[\[source\]](#)

The distinct codimension one configurations of the zero-section (σ_0) relativity to the additional rational section (σ_{1S}).

```
inductive CodimensionOneConfig
| same : CodimensionOneConfig
| nearestNeighbor : CodimensionOneConfig
| nextToNearestNeighbor : CodimensionOneConfig
```

def allowedBarFiveCharges[\[source\]](#)

The allowed $U(1)$ -charges of matter in the 5-bar representation of $SU(5)$ given a $CodimensionOneConfig$.

```
def allowedBarFiveCharges : CodimensionOneConfig → Finset ℤ
| same => {-3, -2, -1, 0, 1, 2, 3}
| nearestNeighbor => {-14, -9, -4, 1, 6, 11}
| nextToNearestNeighbor => {-13, -8, -3, 2, 7, 12}
```

instance (I : CodimensionOneConfig) → Fintype I.allowedBarFiveCharges | same =>... [\[source\]](#)

```
instance : (I : CodimensionOneConfig) → Fintype I.allowedBarFiveCharges
| same => inferInstance
| nearestNeighbor => inferInstance
| nextToNearestNeighbor => inferInstance
```

def allowedTenCharges[\[source\]](#)

The allowed $U(1)$ -charges of matter in the 10d representation of $SU(5)$ given a $CodimensionOneConfig$.

```
def allowedTenCharges : CodimensionOneConfig → Finset ℤ
| same => {-3, -2, -1, 0, 1, 2, 3}
| nearestNeighbor => {-12, -7, -2, 3, 8, 13}
| nextToNearestNeighbor => {-9, -4, 1, 6, 11}
```

instance (I : CodimensionOneConfig) → Fintype I.allowedTenCharges | same => infe...[\[source\]](#)

```
instance : (I : CodimensionOneConfig) → Fintype I.allowedTenCharges
| same => inferInstance
| nearestNeighbor => inferInstance
| nextToNearestNeighbor => inferInstance
```

2.64 PhysLean.StringTheory.FTheory.SU5U1.Examples

[PhysLean/StringTheory/FTheory/SU5U1/Examples.lean](#) on GitHub.

def mkOneTenFourFiveBar[\[source\]](#)

The construction of matter content given a $CodimensionOneConfig$ from a value of M , N and the charges of the representations.

The charge of qHu is the charge of the 5 not the 5-bar representation.

This parameterization comes from Section 3.1.2 of [arXiv:1507.05961](#).

```
def mkOneTenFourFiveBar (I : CodimensionOneConfig) (M : ChiralityFlux) (N : HyperChargeFlux)
(q10 qHu qHd q51 q52 : ℤ)
(hq10 : q10 ∈ I.allowedTenCharges := by decide)
(hqHu : - qHu ∈ I.allowedBarFiveCharges := by decide)
(hqHd : qHd ∈ I.allowedBarFiveCharges := by decide)
(hq51 : q51 ∈ I.allowedBarFiveCharges := by decide)
(hq52 : q52 ∈ I.allowedBarFiveCharges := by decide)
(h5 : ∀ a ∈ ({(M, N, (q51, hq51)), (3 - M, - N, (q52, hq52))}) :
  Multiset (QuantaBarFive I)), a.M = 0 → a.N ≠ 0 := by decide)
(h10 : ∀ a ∈ ({(3, 0, (q10, hq10))}) :
  Multiset (QuantaTen I)), a.M = 0 → a.N ≠ 0 := by decide)
(hd5 : DistinctChargedBarFive (I := I) {(M, N, (q51, hq51)), (3 - M, - N, (q52, hq52))}
  (-qHu, hqHu) (qHd, hqHd) := by decide)
(hd10 : DistinctChargedTen (I := I) {(3, 0, (q10, hq10))} := by decide) :
MatterContent I where
```

def caseI14a[\[source\]](#)

An example of matter content with one 10d representation and 4 5-bar representations. This corresponds to example I.1.4.a in table 1 of [arXiv:1507.05961](#).

```
def caseI14a : MatterContent .same
```

def caseI14b[\[source\]](#)

An example of matter content with one 10d representation and 4 5-bar representations. This corresponds to example I.1.4.b in table 1 of arXiv:1507.05961.

```
def caseI14b : MatterContent .same
```

def caseI14c

[\[source\]](#)

An example of matter content with one 10d representation and 4 5-bar representations. This corresponds to one-version of example I.1.4.c in table 1 of arXiv:1507.05961.

```
def caseI14c : MatterContent .nearestNeighbor
```

def caseI14c'

[\[source\]](#)

An example of matter content with one 10d representation and 4 5-bar representations. This corresponds to one-version of example I.1.4.c in table 1 of arXiv:1507.05961.

```
def caseI14c' : MatterContent .nearestNeighbor
```

def caseI14d

[\[source\]](#)

An example of matter content with one 10d representation and 4 5-bar representations. This corresponds to example I.1.4.d in table 1 of arXiv:1507.05961.

```
def caseI14d : MatterContent .same
```

def caseI14e

[\[source\]](#)

An example of matter content with one 10d representation and 4 5-bar representations. This corresponds to example I.1.4.e in table 1 of arXiv:1507.05961.

```
def caseI14e : MatterContent .nearestNeighbor
```

def caseI14f

[\[source\]](#)

An example of matter content with one 10d representation and 4 5-bar representations. This corresponds to example I.1.4.f in table 1 of arXiv:1507.05961.

```
def caseI14f : MatterContent .nextToNearestNeighbor
```

def caseI24a

[\[source\]](#)

An example of matter content with two 10d representation and 4 5-bar representations. This corresponds to one of the two versions of I.2.4.a in table 8 of arXiv:1507.05961.

```
def caseI24a : MatterContent .same where
```

def caseI24a'

[\[source\]](#)

An example of matter content with two 10d representation and 4 5-bar representations. This corresponds to one of the two versions of the I.2.4.a in table 8 of arXiv:1507.05961.

```
def caseI24a' : MatterContent .same where
```

def caseI24b

[\[source\]](#)

An example of matter content with two 10d representation and 4 5-bar representations. This corresponds to one of the four versions of I.2.4.b in table 8 of arXiv:1507.05961.

```
def caseI24b : MatterContent .same where
```

def caseI24b'

[\[source\]](#)

An example of matter content with two 10d representation and 4 5-bar representations. This corresponds to one of the four versions of I.2.4.b in table 8 of arXiv:1507.05961.

```
def caseI24b' : MatterContent .same where
```

def caseI24b''

[\[source\]](#)

An example of matter content with two 10d representation and 4 5-bar representations. This corresponds to one of the four versions of I.2.4.b in table 8 of arXiv:1507.05961.

```
def caseI24b'' : MatterContent .same where
```

def caseI24b'''

[\[source\]](#)

An example of matter content with two 10d representation and 4 5-bar representations. This corresponds to one of the four versions of I.2.4.b in table 8 of arXiv:1507.05961.

```
def caseI24b''' : MatterContent .same where
```

def caseI34a

[\[source\]](#)

An example of matter content with three 10d representations and 4 5-bar representations. This corresponds to example I.3.4.a of arXiv:1507.05961 (Eq. A12).

```
def caseI34a : MatterContent .same where
```

def allCases

[\[source\]](#)

The finite set of all examples of MatterContent currently defined in PhysLean.

```
def allCases : Finset (Σ I, MatterContent I)
```

lemma allCases_anomalyFree

[\[source\]](#)

```
lemma allCases_anomalyFree : ∀ C ∈ allCases, C.2.AnomalyFree
```

lemma allCases_validMatterSpectrum[\[source\]](#)

```
lemma allCases_validMatterSpectrum : ∀ C ∈ allCases, C.2.ValidMatterSpectrum
```

lemma allCases_muTermU1Constrained[\[source\]](#)

```
lemma allCases_muTermU1Constrained : ∀ C ∈ allCases, C.2.MuTermU1Constrained
```

lemma allCases_RParityU1Constrained[\[source\]](#)

```
lemma allCases_RParityU1Constrained : ∀ C ∈ allCases, C.2.RParityU1Constrained
```

lemma allCases_protonDecayU1Constrained[\[source\]](#)

```
lemma allCases_protonDecayU1Constrained : ∀ C ∈ allCases, C.2.ProtonDecayU1Constrained
```

lemma allCases_hasATopYukawa[\[source\]](#)

```
lemma allCases_hasATopYukawa : ∀ C ∈ allCases, C.2.HasATopYukawa
```

lemma not_allCases_hasABottomYukawa[\[source\]](#)

Not all the examples have a bottom Yukawa.

```
lemma not_allCases_hasABottomYukawa : ¬ ∀ C ∈ allCases, C.2.HasABottomYukawa
```

2.65 PhysLean.StringTheory.FTheory.SU5U1.Matter

[PhysLean/StringTheory/FTheory/SU5U1/Matter.lean](#) on GitHub.

abbrev ChiralityFlux[\[source\]](#)

A type for the chirality flux of matter. This is induced by G_4 -flux. This is often denoted M .

```
abbrev ChiralityFlux : Type
```

abbrev HyperChargeFlux[\[source\]](#)

A type for the hypercharge flux associated with matter curves. This is often denote N .

```
abbrev HyperChargeFlux : Type
```

abbrev QuantaBarFive[\[source\]](#)

The type of quanta associated with matter content in the 5-bar representation.

```
abbrev QuantaBarFive (I : CodimensionOneConfig) : Type
```

abbrev QuantaBarFive.M[\[source\]](#)*The ChiralityFlux quanta of a 5-bar representation.***abbrev** QuantaBarFive.M {I : CodimensionOneConfig} (a : QuantaBarFive I) : ChiralityFlux**abbrev QuantaBarFive.N**[\[source\]](#)*The HyperChargeFlux quanta of a 5-bar representation.***abbrev** QuantaBarFive.N {I : CodimensionOneConfig} (a : QuantaBarFive I) : HyperChargeFlux**abbrev QuantaBarFive.q**[\[source\]](#)*The extra U(1) charge of a 5-bar representation.***abbrev** QuantaBarFive.q {I : CodimensionOneConfig} (a : QuantaBarFive I) :
I.allowedBarFiveCharges**abbrev QuantaTen**[\[source\]](#)*The type of quanta associated with matter content in the 10d representation.***abbrev** QuantaTen (I : CodimensionOneConfig) : Type**abbrev QuantaTen.M**[\[source\]](#)*The ChiralityFlux quanta of a 10d representation.***abbrev** QuantaTen.M {I : CodimensionOneConfig} (a : QuantaTen I) : ChiralityFlux**abbrev QuantaTen.N**[\[source\]](#)*The HyperChargeFlux quanta of a 10d representation.***abbrev** QuantaTen.N {I : CodimensionOneConfig} (a : QuantaTen I) : HyperChargeFlux**abbrev QuantaTen.q**[\[source\]](#)*The extra U(1) charge of a 10d representation.***abbrev** QuantaTen.q {I : CodimensionOneConfig} (a : QuantaTen I) :
I.allowedTenCharges**def DistinctChargedBarFive**[\[source\]](#)*The proposition on Multiset (QuantaBarFive I), and two I.allowedBarFiveCharges denoted qHu and qHd which is true if none of the (underlying) charges are equal.***def** DistinctChargedBarFive {I : CodimensionOneConfig}
(quantaBarFiveMatter : Multiset (QuantaBarFive I))
(qHu : I.allowedBarFiveCharges) (qHd : I.allowedBarFiveCharges) : Prop

instance Decidable (DistinctChargedBarFive quantaBarFiveMatter qHu qHd)

[\[source\]](#)

```
instance {I : CodimensionOneConfig}
  (quantaBarFiveMatter : Multiset (QuantaBarFive I))
  (qHu : I.allowedBarFiveCharges) (qHd : I.allowedBarFiveCharges) :
  Decidable (DistinctChargedBarFive quantaBarFiveMatter qHu qHd)
```

def DistinctChargedTen

[\[source\]](#)

The proposition on a Multiset (QuantaTen I) which is true if non of the underlying charges are equal.

```
def DistinctChargedTen {I : CodimensionOneConfig}
  (quantaTen : Multiset (QuantaTen I)) : Prop
```

instance Decidable (DistinctChargedTen quantaTen)

[\[source\]](#)

```
instance {I : CodimensionOneConfig}
  (quantaTen : Multiset (QuantaTen I)) :
  Decidable (DistinctChargedTen quantaTen)
```

structure MatterContent

[\[source\]](#)

The matter content, assumed to sit in the 5-bar or 10d representation of SU(5).

```
@[ext]
structure MatterContent (I : CodimensionOneConfig) where
  quantaBarFiveMatter : Multiset (QuantaBarFive I)
  quantaTen : Multiset (QuantaTen I)
  qHu : I.allowedBarFiveCharges
  qHd : I.allowedBarFiveCharges
  chirality_charge_not_both_zero_bar_five_matter :
    ∀ a ∈ quantaBarFiveMatter, (a.M = 0 → a.N ≠ 0)
  chirality_charge_not_both_zero_ten : ∀ a ∈ quantaTen, (a.M = 0 → a.N ≠ 0)
  distinctly_charged_quantaBarFiveMatter : DistinctChargedBarFive quantaBarFiveMatter qHu qHd
  distinctly_charged_quantaTen : DistinctChargedTen quantaTen
```

instance DecidableEq (MatterContent I)

[\[source\]](#)

The type MatterContent I has a decidable equality.

```
instance : DecidableEq (MatterContent I)
```

def quantaBarFive

[\[source\]](#)

The QuantaBarFive of all 5-bar representations including the up and down Higgses. The chirality fluxes of the up and down Higgses are taken to be zero, whilst their hypercharge flux is taken to be -1 and +1 respectively, this choice is related to doublet-triplet splitting.

```
def quantaBarFive : Multiset (QuantaBarFive I)
```

lemma chirality_charge_not_both_zero_bar_five[\[source\]](#)

```
lemma chirality_charge_not_both_zero_bar_five :
  ∀ a ∈ M.quantaBarFive, (a.M = 0 → a.N ≠ 0)
```

lemma quantaBarFive_chiralityFlux_two_le_count_zero[\[source\]](#)

```
lemma quantaBarFive_chiralityFlux_two_le_count_zero :
  2 ≤ (M.quantaBarFive.map (QuantaBarFive.M)).count 0
```

lemma quantaBarFive_chiralityFlux_two_le_filter_zero_card[\[source\]](#)

```
lemma quantaBarFive_chiralityFlux_two_le_filter_zero_card :
  2 ≤ ((M.quantaBarFive.map (QuantaBarFive.M)).filter (fun x => x = 0)).card
```

lemma quantaBarFiveMatter_map_q_noDup[\[source\]](#)

```
lemma quantaBarFiveMatter_map_q_noDup :
  (M.quantaBarFiveMatter.map (QuantaBarFive.q)).Nodup
```

lemma quantaBarFive_map_q_noDup[\[source\]](#)

```
lemma quantaBarFive_map_q_noDup : (M.quantaBarFive.map (QuantaBarFive.q)).Nodup
```

lemma quantaBarFive_map_q_card_le_seven[\[source\]](#)

```
lemma quantaBarFive_map_q_card_le_seven :
  (M.quantaBarFive.map (QuantaBarFive.q)).card ≤ 7
```

lemma quantaBarFive_card_le_seven[\[source\]](#)

```
lemma quantaBarFive_card_le_seven : M.quantaBarFive.card ≤ 7
```

def GaugeAnomalyMSSM[\[source\]](#)

The gauge anomalies coming from the SM gauge group.

These correspond to the conditions - $\sum_a M_a - \sum_i M_i = 0$ - $\sum_a N_a = 0$ - $\sum_i N_i = 0$

where the sums are over the matter content in the 5-bar and 10d representations.

Ref: See equation (20) and (21) of arXiv:1401.5084.

```
def GaugeAnomalyMSSM : Prop
```

instance Decidable (GaugeAnomalyMSSM M)[\[source\]](#)

```
instance : Decidable (GaugeAnomalyMSSM M)
```

def GaugeAnomalyU1MSSM[\[source\]](#)

The mixed $U(1)$ -MSSM gauge anomaly.

This condition corresponds to

$$\sum_a q_a N_a + \sum_i q_i N_i = 0$$

Ref: See equation (22) of arXiv:1401.5084.

```
def GaugeAnomalyU1MSSM : Prop
```

```
instance Decidable (GaugeAnomalyU1MSSM  $\mathcal{M}$ )
```

[\[source\]](#)

```
instance : Decidable (GaugeAnomalyU1MSSM  $\mathcal{M}$ )
```

```
def GaugeAnomalyU1YU1U1
```

[\[source\]](#)

The mixed $U(1)Y$ - $U(1)$ - $U(1)$ gauge anomaly.

This condition corresponds to

$$3 * \sum_a q_a q_a N_a + \sum_i q_i q_i N_i = 0$$

Ref: See equation (23) of arXiv:1401.5084.

```
def GaugeAnomalyU1YU1U1 : Prop
```

```
instance Decidable (GaugeAnomalyU1YU1U1  $\mathcal{M}$ )
```

[\[source\]](#)

```
instance : Decidable (GaugeAnomalyU1YU1U1  $\mathcal{M}$ )
```

```
def AnomalyFree
```

[\[source\]](#)

The condition on matter content for it to be anomaly free.

```
def AnomalyFree : Prop
```

```
instance Decidable (AnomalyFree  $\mathcal{M}$ )
```

[\[source\]](#)

```
instance : Decidable (AnomalyFree  $\mathcal{M}$ )
```

2.66 PhysLean.StringTheory.FTheory.SU5U1.NoExotics.Basic

[PhysLean/StringTheory/FTheory/SU5U1/NoExotics/Basic.lean](#) on GitHub.

```
def ThreeChiralFamilies
```

[\[source\]](#)

The condition on the matter content for there to exist three chiral families.

This corresponds to the conditons that: $-\sum_a M_a = 3 - \sum_i M_i = 3 - 0 \leq M_a - 0 \leq M_i$

Ref: Equation (26) of arXiv:1401.5084.

```
def ThreeChiralFamilies : Prop
```

instance Decidable $\mathcal{M}$.ThreeChiralFamilies

[\[source\]](#)

instance : Decidable $\mathcal{M}$.ThreeChiralFamilies

def NoExotics

[\[source\]](#)

The condition on the matter content for there to be no exotics in the spectrum.

This corresponds to the conditions that: - $\sum_a N_a = 0$ - $\sum_i N_i = 0$ - $M_a \leq N_a \leq M_a - M_i - 1 \leq N_i \leq 3$

Ref: Equation (27) and (28) of arXiv:1401.5084.

def NoExotics : Prop

instance Decidable $\mathcal{M}$.NoExotics

[\[source\]](#)

instance : Decidable $\mathcal{M}$.NoExotics

def ThreeLeptonDoublets

[\[source\]](#)

The condition on the matter content for there to be three lepton doublets with exactly one pair of Higgs.

This corresponds to the conditions that: - $\sum_i |M_i + N_i| = 5$

Ref: Equation (29) of arXiv:1401.5084.

def ThreeLeptonDoublets : Prop

instance Decidable $\mathcal{M}$.ThreeLeptonDoublets

[\[source\]](#)

instance : Decidable $\mathcal{M}$.ThreeLeptonDoublets

def ValidMatterSpectrum

[\[source\]](#)

The condition on the matter content for it to produce a valid spectrum.

def ValidMatterSpectrum : Prop

instance Decidable $\mathcal{M}$.ValidMatterSpectrum

[\[source\]](#)

instance : Decidable $\mathcal{M}$.ValidMatterSpectrum

2.67 PhysLean.StringTheory.FTheory.SU5U1.NoExotics.FiveBar

[PhysLean/StringTheory/FTheory/SU5U1/NoExotics/FiveBar.lean](#) on GitHub.

lemma quantaBarFive_chiralityFlux_nonneg'

[\[source\]](#)

The chirality flux associated with a 5-bar representation must be non-negative.

lemma quantaBarFive_chiralityFlux_nonneg' {a : QuantaBarFive I}
(h3 : $\mathcal{M}$.ThreeChiralFamilies) (h : a ∈ $\mathcal{M}$.quantaBarFive) : 0 ≤ a.M

lemma quantaBarFive_chiralityFlux_nonneg[\[source\]](#)

The chirality flux associated with a 5-bar representation must be non-negative.

```
lemma quantaBarFive_chiralityFlux_nonneg {a : ChiralityFlux}
  (h3 :  $\mathcal{M}$ .ThreeChiralFamilies) (h : a ∈ ( $\mathcal{M}$ .quantaBarFive).map QuantaBarFive.M) : 0 ≤ a
```

lemma quantaBarFive_chiralityFlux_le_three[\[source\]](#)

The chirality flux associated with a 5-bar representation must be less than or equal to three.

```
lemma quantaBarFive_chiralityFlux_le_three {a : QuantaBarFive I}
  (h3 :  $\mathcal{M}$ .ThreeChiralFamilies) (h : a ∈  $\mathcal{M}$ .quantaBarFive) : a.M ≤ 3
```

lemma quantaBarFive_chiralityFlux_filter_non_zero_sum[\[source\]](#)

```
lemma quantaBarFive_chiralityFlux_filter_non_zero_sum (h3 :  $\mathcal{M}$ .ThreeChiralFamilies) :
  (( $\mathcal{M}$ .quantaBarFive.map (QuantaBarFive.M)).filter (fun x => ¬ x = 0)).sum = 3
```

lemma quantaBarFive_chiralityFlux_filter_non_zero_card_le_three[\[source\]](#)

The number of 5-bar representations with non-zero chirality flux is less than or equal to three.

```
lemma quantaBarFive_chiralityFlux_filter_non_zero_card_le_three (h3 :  $\mathcal{M}$ .ThreeChiralFamilies) :
  (( $\mathcal{M}$ .quantaBarFive.map (QuantaBarFive.M)).filter (fun x => ¬ x = 0)).card ≤ 3
```

lemma pos_of_mem_quantaBarFive_chiralityFlux_filter_non_zero[\[source\]](#)

```
lemma pos_of_mem_quantaBarFive_chiralityFlux_filter_non_zero {a : ChiralityFlux}
  (h3 :  $\mathcal{M}$ .ThreeChiralFamilies)
  (h : a ∈ (( $\mathcal{M}$ .quantaBarFive.map (QuantaBarFive.M)).filter (fun x => ¬ x = 0))) :
  0 < a
```

lemma quantaBarFive_chiralityFlux_filter_non_zero_card_mem[\[source\]](#)

```
lemma quantaBarFive_chiralityFlux_filter_non_zero_card_mem (h3 :  $\mathcal{M}$ .ThreeChiralFamilies) :
  (( $\mathcal{M}$ .quantaBarFive.map (QuantaBarFive.M)).filter (fun x => ¬ x = 0)).card ∈
  ({1, 2, 3} : Finset ℕ)
```

lemma quantaBarFive_chiralityFlux_filter_non_zero_mem[\[source\]](#)

The multiset of non-zero chirality fluxes of matter content in the 5-bar representation satisfying ThreeChiralFamilies must either be - {1, 1, 1} - {1, 2} - {3}

```
lemma quantaBarFive_chiralityFlux_filter_non_zero_mem (h3 :  $\mathcal{M}$ .ThreeChiralFamilies) :
  ( $\mathcal{M}$ .quantaBarFive.map (QuantaBarFive.M)).filter (fun x => ¬ x = 0)
  ∈ ({1, 1, 1}, {1, 2}, {3}) : Finset (Multiset ChiralityFlux)
```

lemma quantaBarFive_zero_chiralityFlux_abs_sum_le_five[\[source\]](#)

```
lemma quantaBarFive_zero_chiralityFlux_abs_sum_le_five (h3L :  $\mathcal{M}$ .ThreeLeptonDoublets) :
  (Multiset.map (fun a => |a.M + a.N|) ( $\mathcal{M}$ .quantaBarFive.filter (fun x => x.M = 0))).sum ≤ 5
```

lemma quantaBarFive_zero_chiralityFlux_card_le_five[\[source\]](#)

The number of 5d representations with chirality flux equal to zero is less than or equal to five.

```
lemma quantaBarFive_zero_chiralityFlux_card_le_five (h3L :  $\mathcal{M}$ .ThreeLeptonDoublets) :
  (( $\mathcal{M}$ .quantaBarFive.filter (fun x => x.M = 0))).card ≤ 5
```

lemma quantaBarFive_zero_chiralityFlux_card_mem[\[source\]](#)

```
lemma quantaBarFive_zero_chiralityFlux_card_mem (h3L :  $\mathcal{M}$ .ThreeLeptonDoublets) :
  (( $\mathcal{M}$ .quantaBarFive.map (QuantaBarFive.M)).filter (fun x => x = 0)).card ∈
  ({0, 1, 2, 3, 4, 5} : Finset  $\mathbb{N}$ )
```

lemma quantaBarFive_zero_chiralityFlux_mem[\[source\]](#)

```
lemma quantaBarFive_zero_chiralityFlux_mem (h3L :  $\mathcal{M}$ .ThreeLeptonDoublets) :
  ( $\mathcal{M}$ .quantaBarFive.map (QuantaBarFive.M)).filter (fun x => x = 0) ∈
  ({replicate 5 0, replicate 4 0, replicate 3 0, replicate 2 0} :
   Finset (Multiset ChiralityFlux))
```

lemma quantaBarFive_card_le_eight[\[source\]](#)

The number of 5-bar representations is less than or equal to eight. Note the existences of quantaBarFive_card_le_seven. The proof of this weaker result does not rely on the assumptions of no-duplicate charges.

```
lemma quantaBarFive_card_le_eight (h3 :  $\mathcal{M}$ .ThreeChiralFamilies) (h3L :  $\mathcal{M}$ .ThreeLeptonDoublets) :
  :
   $\mathcal{M}$ .quantaBarFive.card ≤ 8
```

lemma quantaBarFive_chiralityFlux_mem[\[source\]](#)

The multiset of chirality fluxes of matter content in the 5 bar representation satisfying ThreeChiralFamilies and ThreeLeptonDoublets must either be - {1, 1, 1} - {1, 2} - {3} with zero to five chirality fluxes equal to zero.

```
lemma quantaBarFive_chiralityFlux_mem (h3 :  $\mathcal{M}$ .ThreeChiralFamilies) (h3L :  $\mathcal{M}$ .
  ThreeLeptonDoublets) :
   $\mathcal{M}$ .quantaBarFive.map QuantaBarFive.M ∈
  ({{1, 1, 1, 0, 0, 0, 0}, {1, 1, 1, 0, 0, 0, 0}, {1, 1, 1, 0, 0, 0, 0},
   {1, 2, 0, 0, 0, 0, 0}, {1, 2, 0, 0, 0, 0, 0}, {1, 2, 0, 0, 0, 0, 0}, {1, 2, 0, 0, 0, 0, 0},
   {3, 0, 0, 0, 0, 0, 0}, {3, 0, 0, 0, 0, 0, 0}, {3, 0, 0, 0, 0, 0, 0}} :
   Finset (Multiset ChiralityFlux))
```

lemma quantaBarFive_three_le_card[\[source\]](#)

The number of 5-bar representations (including the Higgs) is greater than or equal to three.

```
lemma quantaBarFive_three_le_card (h3 :  $\mathcal{M}$ .ThreeChiralFamilies) (h3L :  $\mathcal{M}$ .ThreeLeptonDoublets) :
  :
  3 ≤  $\mathcal{M}$ .quantaBarFive.card
```

2.68 PhysLean.StringTheory.FTheory.SU5U1.NoExotics.Ten

[PhysLean/StringTheory/FTheory/SU5U1/NoExotics/Ten.lean](#) on GitHub.

lemma quantaTen_chiralityFlux_nonneg'

[\[source\]](#)

The chirality flux associated with a 10d representation must be non-negative.

```
lemma quantaTen_chiralityFlux_nonneg' {a : QuantaTen I}
  (h3 :  $\mathcal{M}$ .ThreeChiralFamilies) (h : a ∈  $\mathcal{M}$ .quantaTen) : 0 ≤ a.M
```

lemma quantaTen_chiralityFlux_nonneg

[\[source\]](#)

The chirality flux associated with a 5-bar representation must be non-negative.

```
lemma quantaTen_chiralityFlux_nonneg {a : ChiralityFlux}
  (h3 :  $\mathcal{M}$ .ThreeChiralFamilies) (h : a ∈ ( $\mathcal{M}$ .quantaTen).map QuantaTen.M) : 0 ≤ a
```

lemma quantaTen_chiralityFlux_ne_zero

[\[source\]](#)

Due to the condition of having no exotics in the spectrum, the chirality flux of a 10d representation must be non-zero. This also uses the condition that the hypercharge flux and the chirality flux cannot both be zero.

```
lemma quantaTen_chiralityFlux_ne_zero {a : ChiralityFlux}
  (he :  $\mathcal{M}$ .NoExotics) (h : a ∈ ( $\mathcal{M}$ .quantaTen).map QuantaTen.M) : ¬ a = 0
```

lemma quantaTen_chiralityFlux_pos

[\[source\]](#)

The chirality flux associated with a 5-bar representation must be non-negative.

```
lemma quantaTen_chiralityFlux_pos {a : ChiralityFlux} (he :  $\mathcal{M}$ .NoExotics)
  (h3 :  $\mathcal{M}$ .ThreeChiralFamilies) (h : a ∈ ( $\mathcal{M}$ .quantaTen).map QuantaTen.M) : 0 < a
```

lemma quantaTen_chiralityFlux_filter_ne_zero_eq_self

[\[source\]](#)

```
lemma quantaTen_chiralityFlux_filter_ne_zero_eq_self (he :  $\mathcal{M}$ .NoExotics) :
  ( $\mathcal{M}$ .quantaTen.map (QuantaTen.M)).filter (fun x => ¬ x = 0) =
   $\mathcal{M}$ .quantaTen.map (QuantaTen.M)
```

lemma quantaTen_chiralityFlux_le_three

[\[source\]](#)

The chirality flux associated with a 10d representation must be less than or equal to three.

```
lemma quantaTen_chiralityFlux_le_three {a : QuantaTen I}
  (h3 :  $\mathcal{M}$ .ThreeChiralFamilies) (h : a ∈  $\mathcal{M}$ .quantaTen) : a.M ≤ 3
```

lemma quantaTen_chiralityFlux_card_le_three

[\[source\]](#)

```
lemma quantaTen_chiralityFlux_card_le_three (he :  $\mathcal{M}$ .NoExotics)
  (h3 :  $\mathcal{M}$ .ThreeChiralFamilies) :
  ( $\mathcal{M}$ .quantaTen.map (QuantaTen.M)).card ≤ 3
```

lemma quantaTen_chiralityFlux_card_mem[\[source\]](#)

```
lemma quantaTen_chiralityFlux_card_mem (he :  $\mathcal{M}$ .NoExotics)
  (h3 :  $\mathcal{M}$ .ThreeChiralFamilies) :
  (( $\mathcal{M}$ .quantaTen.map (QuantaTen.M))).card  $\in$ 
  ({1, 2, 3} : Finset  $\mathbb{N}$ )
```

lemma quantaTen_card_le_three[\[source\]](#)

The number of 10d representations is less than or equal to three.

```
lemma quantaTen_card_le_three (he :  $\mathcal{M}$ .NoExotics)
  (h3 :  $\mathcal{M}$ .ThreeChiralFamilies) :  $\mathcal{M}$ .quantaTen.card  $\leq$  3
```

lemma quantaTen_chiralityFlux_mem[\[source\]](#)

The multiset of chirality fluxes of matter content in the 10d representation satisfying NoExotics and ThreeChiralFamilies must either be - {1, 1, 1} - {1, 2} - {3}

```
lemma quantaTen_chiralityFlux_mem (he :  $\mathcal{M}$ .NoExotics) (h3 :  $\mathcal{M}$ .ThreeChiralFamilies) :
   $\mathcal{M}$ .quantaTen.map (QuantaTen.M)
   $\in$  ({1, 1, 1}, {1, 2}, {3}) : Finset (Multiset ChiralityFlux)
```

2.69 PhysLean.StringTheory.FTheory.SU5U1.PhenoConstraints.Basic

[PhysLean/StringTheory/FTheory/SU5U1/PhenoConstraints/Basic.lean](#) on GitHub.

def chargeMuTerm[\[source\]](#)

The overall charge of the term μ $5H_u$ $5\bar{H}_d$

```
def chargeMuTerm (qHu qHd : I.allowedBarFiveCharges) :  $\mathbb{Z}$ 
```

def chargeW1Term[\[source\]](#)

The charges of the term $W^{1_{ijk}} 10^i 10^j 10^k 5\bar{M}^l$.

```
def chargeW1Term (q5 : Multiset I.allowedBarFiveCharges) (q10 : Multiset I.allowedTenCharges) :
  Multiset  $\mathbb{Z}$ 
```

def chargeBetaTerm[\[source\]](#)

The charges of the term $\beta_i 5\bar{M}^i 5H_u$.

```
def chargeBetaTerm (q5bar : Multiset I.allowedBarFiveCharges) (qHu : I.allowedBarFiveCharges) :
  Multiset  $\mathbb{Z}$ 
```

def chargeLambdaTerm[\[source\]](#)

The charges of the term $\lambda_{ijk} 5\bar{M}^i 5\bar{M}^j 10^k$.

```
def chargeLambdaTerm (q5bar : Multiset I.allowedBarFiveCharges)
  (q10 : Multiset I.allowedTenCharges) : Multiset  $\mathbb{Z}$ 
```

def chargeK1Term[\[source\]](#)

The charges of the term $K^1_{ijk} 10^i 10^j 5M^k$.

```
def chargeK1Term (q5bar : Multiset I.allowedBarFiveCharges)
  (q10 : Multiset I.allowedTenCharges) : Multiset ℤ
```

def chargeW4Term[\[source\]](#)

The charges of the term $W^4_i 5^{\sim}M^i 5^{\sim}Hd 5Hu 5Hu$.

```
def chargeW4Term (q5bar : Multiset I.allowedBarFiveCharges)
  (qHd : I.allowedBarFiveCharges) (qHu : I.allowedBarFiveCharges) : Multiset ℤ
```

def chargeK2Term[\[source\]](#)

The charges of the term $K^2_i 5^{\sim}Hu 5^{\sim}Hd 10^i$

```
def chargeK2Term (q10 : Multiset I.allowedTenCharges)
  (qHu : I.allowedBarFiveCharges) (qHd : I.allowedBarFiveCharges) :
  Multiset ℤ
```

def chargeW2Term[\[source\]](#)

The charges of the term $W^2_{ijk} 10^i 10^j 10^k 5^{\sim}Hd$.

```
def chargeW2Term (q10 : Multiset I.allowedTenCharges)
  (qHd : I.allowedBarFiveCharges) : Multiset ℤ
```

def chargeYukawaTop[\[source\]](#)

The charges associated with the term $\lambda^t_{ij} 10^i 10^j 5Hu$

```
def chargeYukawaTop (q10 : Multiset I.allowedTenCharges)
  (qHu : I.allowedBarFiveCharges) : Multiset ℤ
```

def chargeYukawaBottom[\[source\]](#)

The charges associated with the term $\lambda^b_{ij} 10^i 5^{\sim}M^j 5^{\sim}Hd^{\sim}$.

```
def chargeYukawaBottom (q5bar : Multiset I.allowedBarFiveCharges)
  (q10 : Multiset I.allowedTenCharges) (qHd : I.allowedBarFiveCharges) : Multiset ℤ
```

def phenoCharges[\[source\]](#)

A string containing the $U(1)$ -charges associated with interaction terms.

```
def phenoCharges : String
```

def MuTermU1Constrained[\[source\]](#)

A proposition which is true when the μ -term ($5Hu 5^{\sim}Hd$) does not obey the additional $U(1)$ symmetry in the model, and is therefore constrained.

```
def MuTermU1Constrained : Prop
```

instance Decidable $\mathcal{M}$.MuTermU1Constrained[\[source\]](#)

```
instance : Decidable  $\mathcal{M}$ .MuTermU1Constrained
```

def RParityU1Constrained[\[source\]](#)

A proposition which is true when the R-parity violating terms all do not obey the additional $U(1)$ symmetry in the model, and are therefore constrained. This corresponds to the terms: $-\beta_i \tilde{5}^i \tilde{5}^i H_u - \lambda_{ijk} \tilde{5}^i \tilde{5}^j 10^k - W^2_{ijk} 10^i 10^j 10^k \tilde{5}^i H_d - W^4_i \tilde{5}^i \tilde{5}^i H_d 5H_u 5H_u - K^1_{ijk} 10^i 10^j 5M^k - K^2_i \tilde{5}^i H_u \tilde{5}^i H_d 10^i$

```
def RParityU1Constrained : Prop
```

instance Decidable $\mathcal{M}$.RParityU1Constrained[\[source\]](#)

```
instance : Decidable  $\mathcal{M}$ .RParityU1Constrained
```

def ProtonDecayU1Constrained[\[source\]](#)

A proposition which is true when the terms in the super-potential and the Kahler potential contributing to proton decay do not obey the additional $U(1)$ symmetry in the model, and are therefore constrained. This corresponds to the terms $-\tilde{W}^1_{ijkl} 10^i 10^j 10^k \tilde{5}^l - \lambda_{ijk} \tilde{5}^i \tilde{5}^j 10^k - W^2_{ijk} 10^i 10^j 10^k \tilde{5}^i H_d - K^1_{ijk} 10^i 10^j 5M^k$

```
def ProtonDecayU1Constrained : Prop
```

instance Decidable $\mathcal{M}$.ProtonDecayU1Constrained[\[source\]](#)

```
instance : Decidable  $\mathcal{M}$ .ProtonDecayU1Constrained
```

def HasATopYukawa[\[source\]](#)

The condition on the matter content for there to exist at least one copy of the coupling $-\lambda^t_{ij} 10^i 10^j 5H_u$

```
def HasATopYukawa ( $\mathcal{M}$  : MatterContent I) : Prop
```

instance Decidable $\mathcal{M}$.HasATopYukawa[\[source\]](#)

```
instance : Decidable  $\mathcal{M}$ .HasATopYukawa
```

def HasABottomYukawa[\[source\]](#)

The condition on the matter content for there to exist at least one copy of the coupling $-\lambda^b_{ij} 10^i \tilde{5}^j \tilde{5}^i H_d$

```
def HasABottomYukawa ( $\mathcal{M}$  : MatterContent I) : Prop
```

instance Decidable $\mathcal{M}$.HasABottomYukawa[\[source\]](#)

```
instance : Decidable  $\mathcal{M}$ .HasABottomYukawa
```

2.70 `scripts.MetaPrograms.check_dup_tags`

[scripts/MetaPrograms/check_dup_tags.lean](#) on GitHub.

def tagsFromTODOs [\[source\]](#)

```
def tagsFromTODOs : MetaM (Array String)
```

def tagsFromInformal [\[source\]](#)

```
unsafe def tagsFromInformal : CoreM (Array String)
```

def tagsFromSemiformal [\[source\]](#)

```
unsafe def tagsFromSemiformal : CoreM (Array String)
```

def tagDuplicateTest [\[source\]](#)

```
unsafe def tagDuplicateTest : MetaM Unit
```

2.71 `scripts.MetaPrograms.local_stats`

[scripts/MetaPrograms/local_stats.lean](#) on GitHub.

def numDefs [\[source\]](#)

```
def numDefs (dir : String) : CoreM Nat
```

def numLemmas [\[source\]](#)

```
def numLemmas (dir : String) : CoreM Nat
```

def getStats [\[source\]](#)

```
def getStats (dir : String) : MetaM String
```

2.72 `scripts.MetaPrograms.make_tag`

[scripts/MetaPrograms/make_tag.lean](#) on GitHub.

def digitChar [\[source\]](#)

```
def digitChar (n : Nat) : Char
```

def toDigitsCore [\[source\]](#)

```
def toDigitsCore (base : Nat) : Nat → Nat → List Char → List Char
| 0, _, ds => ds
| fuel+1, n, ds =>
  let d
```

def toDigits[\[source\]](#)

```
def toDigits (base : Nat) (n : Nat) : List Char
```

2.73 scripts.MetaPrograms.TODO_to_yaml

[scripts/MetaPrograms/TOD0_to_yaml.lean](#) on GitHub.

def ofSemiFormal[\[source\]](#)

```
def FullTODOInfo.ofSemiFormal (t : WantedInfo) : FullTODOInfo
```

def allSemiInformal[\[source\]](#)

```
unsafe def allSemiInformal : CoreM (Array FullTODOInfo)
```